

Architetture Multimediali

Simone Paolo Petta

a.a. 2024-2025

Contents

1	Introduzione	7
1.1	Comunicazione	8
1.2	Comunicazione Digitale	9
1.3	Convergenza digitale	10
1.4	Media VS Documents	10
1.5	Media VS Modeling	10
1.6	Media VS Use	10
1.7	Media VS Funzionalità	11
1.8	MIR - Multimedia Information Retrieval	11
1.8.1	Esigenza esplorativa	11
1.8.2	Typical MIR Architecture	12
1.9	Recupero informativo	12
1.10	Audio	12
1.11	Quality of service	12
2	RDBMS	13
2.1	OODBMS - Object-Oriented Databases	14
2.2	ORDBMS - Object-relational DBMS	14
2.2.1	Tipi di dati	15
2.2.2	Array	18
2.2.3	Multiset	19
2.2.4	Metodi	20
2.2.5	Tabella tipata	22
2.2.6	Reference types	24
2.2.7	Ereditarietà	25
2.3	Dal design concettuale al modello object-relational	27
2.3.1	Modello ER	28
2.3.2	UML	28
2.3.3	Associazioni	28
3	Oracle	32
3.1	Data type	33
3.2	SQL Developer	33
3.3	Object types	34
3.4	Metodi	34
3.4.1	Metodi member	34

3.4.2	Metodi costruttori	34
3.5	Tabelle	35
3.6	PLSQL	35
3.6.1	Blocco di istruzioni	36
3.6.2	Gestione eccezioni	36
3.6.3	Cursore	37
3.7	Esempio creazione tipo di dato	38
3.8	Tabella tipata	39
3.8.1	Accesso	40
3.9	Tipi riferimento	40
3.10	Collezione	42
3.10.1	VARRAY	42
3.10.2	Nested Table	43
3.11	Ereditarietà	45
3.12	Vista	47
3.13	Esempio	48
4	Text Retrieval	53
4.1	Creazione di un archivio di documenti testuali	55
4.1.1	Rappresentare semanticamente un documento	56
4.1.2	Indicizzazione	56
4.1.3	Analisi lessicale	57
4.1.4	Rimozione delle stop word	57
4.1.5	Stemming	57
4.1.6	Indice inverso	58
4.2	Recovery	59
4.3	Modelling	59
4.4	Classici	60
4.4.1	Booleano	61
4.4.2	Modello vettoriale	61
4.5	ORACLE Text	65
4.5.1	Main Features	65
4.5.2	Query particolari	67
4.5.3	Valutazione	82
5	Modello Probabilistico	83
5.1	Eventi ed errori	84
5.2	Rischio	85
5.3	Probabilistic Retrieval Model	86
5.3.1	Stima rilevanza documento	87
5.3.2	BM25	88
5.3.3	Cord-factor	89
5.3.4	TF/IDF	89

6	Solr	93
6.1	Score calculation	94
6.2	Schema	95
6.2.1	Field Types	95
6.2.2	Dynamic Fields	96
6.3	Indexing	96
6.4	Query Parsers	96
6.5	Hello World!	97
6.5.1	Query	97
6.5.2	Copy Field	98
6.5.3	Parametri	98
6.5.4	And/Or	99
6.5.5	Faceting	99
6.5.6	Pivot	100
6.6	Using a Solr Analyzer for Text Analysis	100
6.6.1	Classic Analyzer	100
6.6.2	Keyword Analyzer	100
6.6.3	Simple Analyzer	101
6.6.4	Standard Analyzer	101
6.6.5	Stop Analyzer	101
6.6.6	Space Analyzer	101
6.7	Tokenizzatore	101
7	Question Answering	102
7.1	Architettura di un QA	104
7.1.1	Question analysis	105
7.1.2	Query formulation	106
7.1.3	IR And Document Processing	107
7.2	Dialog Flow	109
7.2.1	Parametro	109
7.2.2	Fullfillment	109
7.2.3	Dialog Flow CX	111
7.2.4	Ton of voice	112
7.3	Generative AI	112
7.3.1	ChatGPT	112
7.3.2	Embeddings Models	113
7.3.3	Text similarity models	113
7.3.4	Modelli ricerca testuali	113
7.4	RAG- Retrival Augmented Generation	113
7.4.1	Cheshire Cat	114
8	Digital Images	117
8.1	Risoluzione	118
8.2	Profondità di colore	118
8.2.1	Scala di grigi	118
8.3	Dithering	119
8.4	Colori	119

8.4.1	8 bit color information	120
8.5	GIF	120
8.6	JPEG	120
8.7	PNG	120
8.8	TIFF	120
8.9	EXIF	121
8.10	PS	121
8.11	Human perception of color	121
8.11.1	Codifica del colore	121
8.11.2	Tinta	123
8.11.3	Saturazione	124
8.11.4	Luminosità	124
8.11.5	HSV	125
8.11.6	CIE	125
8.12	Image Retrival	126
8.12.1	Forma	127
8.12.2	Texture	127
8.12.3	Colore	127
8.12.4	Surrogati delle immagini	128
8.12.5	Database di immagini	128
8.12.6	Similarità	129
8.12.7	Feature Vector	130
8.12.8	Precisione e ricerca parziale	131
8.12.9	Metodo trasformatzionale	131
8.12.10	Image DBMS	132
8.13	Oracle Multimedia	132
8.13.1	Storage	132
8.13.2	Storage system	133
8.13.3	Tipo di dato OrdImage	134
8.13.4	ORDSource	134
8.14	Blob	134
8.15	ORDImage	140
8.16	IRS	149
8.16.1	Semantic Gap	149
8.16.2	MPE System	149
8.16.3	Semantic Retrival	149
8.16.4	Query by semantic example	151
8.16.5	Query in natural language	151
8.16.6	What next?	152
9	Digital Audio	153
9.1	Campionamento e Quantizzazione	154
9.2	Teorema di Nyquist	155
9.3	Quantizzazione	156
9.3.1	Merito dei dB	157
9.3.2	Signal to noise ratio (SNR)	157

9.3.3	Signal to Quantization Noise Ratio (SQNR)	157
9.4	Linear and Non-linear Quantization	157
9.4.1	Storage of audio in file in Kb	158
9.5	Digital Audio Format	158
9.6	Compressione	159
9.6.1	Masking	159
9.7	MPEG	159
9.8	MIDI	160
10	Digital Video	161
10.1	Frame	162
10.2	Codifica	162
10.2.1	La risoluzione video	162
10.2.2	Aspect Ratio	162
10.2.3	Interlacciamento	162
10.3	Compressione	163
10.3.1	Ridondanza spaziale	163
10.3.2	Ridondanza temporale	164
10.3.3	Motion Vector	164
10.3.4	Motion Estimation (ME)	165
10.3.5	Motion Compensation (MC)	165
10.3.6	Frame I/P/B	166
10.3.7	MPEG	166
10.3.8	Slice	166
10.3.9	Profilo	167
10.4	Stabilizzazione	167
10.5	Retrival	167
10.5.1	Audio	168
10.5.2	Oracle	169
11	Media Integration	172
11.1	Sincronizzazione	173
11.1.1	Media Integration in MM Presentation	173
11.1.2	SMIL	174
11.1.3	MPEG7	174
12	Quality of Service	182
12.1	Multimedia Service Classes	184
12.2	MPEG System Streams	185
12.2.1	Packet Jitter	186
12.2.2	Perdita dei pacchetti	187
12.3	The windows of scarcity	187
12.4	Gestione QoS	188
12.4.1	Negoziiazione	189
12.4.2	Traffic shaping	190
12.4.3	Flow Specification	193
12.4.4	Procedura di negoziazione	193

12.4.5	Reservation delle risorse	193
12.4.6	Scheduling	193
12.4.7	Stream adaptation	194
12.5	IP Protocols	194
12.5.1	Approcci di Qos	194
12.6	Human computer interaction	197

Chapter 1

Introduzione

Parleremo di multimedia e comunicazione digitale. Parleremo di rappresentazione, codifica, memorizzazione e trasmissione multimediale. Tutto quello che riguarda la multimedialità.

Un sistema multimediale è quello che gestisce un'informazione complessa ed eterogenea, a differenza di quello che succede nelle basi di dati normali. Non ci sono istruzioni codificate per recuperare le immagini, per gestire l'informazione non strutturata è necessario modellare l'archivio che gestisce le informazioni non strutturate e poi fare delle query apposite, ovviamente sono operazioni più complesse rispetto ad utilizzare un normale DBMS.

Cos'è un'informazione multimediale? ci sono tantissime risposte anche opposte tra di loro in base a chi la dà:

- Un venditore di PC direbbe che un PC ha un microprocessore che capisce istruzioni multimediali
- un consumer direbbe che una TV ha centinaia di canali digitali
- Da un punto di vista scientifico quando parliamo di applicazioni che includono diversi tipi di media

Le discipline che sono coinvolte quando parliamo di sistemi multimediali sono diversi, aspetti di grafica, interazione uomo macchina..

Componenti di un'informazione multimediale:

- Contesti: qualsiasi contesto che utilizzi più di un media. Di per sé oggi qualsiasi sistema è multimediale, anche una pagina web con delle immagini.
- Videoconferenze
- Applicazioni per la modifica multimediale, non dobbiamo pensare solo a sistemi di fruizione ma sistemi in cui possiamo fare editing.
- reversing engineering

1.1 Comunicazione

Quando parliamo di sistemi interattivi non solo dobbiamo modellare informazioni multimediali in maniera efficace, ma è importante mettere l'accento sulla comunicazione tramite informazione multimediale.

Devo stare attento a diverse cose, non solo riguardante la trasmissione, c'è quella che viene chiamata la qualità percepita, quando creo il progetto multimediale lo faccio in base a determinate condizioni fornendo al sistema una serie di contenuti multimediali. Ma quello che vado a creare siamo sicuri che verrà percepito dall'utente finale come ce lo avevo in mente io?

Problematiche:

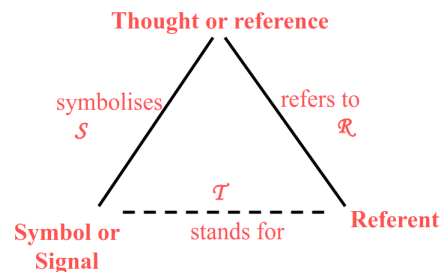
- Tecniche: basilari di trasmissione, dobbiamo evitare ritardi, pause, problemi di sincronizzazione audio-video

- Esigenze dell'utente: queste esigenze che ha l'utente sono fornite in modo adeguato all'utente?

Dobbiamo essere in grado di accontentare l'utente in entrambi i casi, la soddisfazione non è facile, l'aspetto tecnico lo possiamo risolvere tramite alcune tecniche. La soddisfazione è complicata perchè devo capire cosa vuole l'utente, ad esempio creeremo un archivio di immagini e ad un certo punto l'utente avrà una richiesta informatica (tutte le foto con tramonto), si farà dell'estrazione di feature visuali per trovare le immagini.

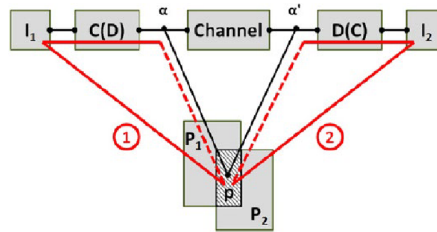
1.2 Comunicazione Digitale

Ci sono questi problemi di comprensione tra l'utente e la macchina, viene formalizzato nella teoria della comunicazione digitale.



Dal punto di vista formale hanno cercato di formalizzare questo concetto di comprensione, questo studio ha origine nella semiotica e nelle scienze del linguaggio, tant'è che il triangolo semantico è stato definito da ricercatori di scienze delle comunicazioni, schematizza come interagisce l'utente con la macchina. C'è un concetto di emissione e ricezione del messaggio, un messaggio è un insieme di segnali che nella nostra testa hanno un concetto ben preciso (tramonto) che si riferisce (referente) a qualcosa che ho già visto nel passato. Questa ricezione dev'essere simile anche per il ricevente, anche la macchina deve ricevere il segnale ed avere un concetto che deve riferirsi ad un elemento concreto.

Questa comunicazione è un modello di questo tipo:



ai due estremi abbiamo due interlocutori i quali mettono i messaggi su un canale di comunicazione che ha un concetto ben preciso, questo riferimento deve essere ben simili a quello che ha programmato la macchina (l'altro estremo). Se i due referenti non si sovrappongono (quello che mi aspetto io e quello che mi viene dato) si genera un problema di incomprensione.

Esempio di incomprensione, mettiamo di voler scrivere su una chat un messaggio che viene scritto, codificato e viene trasmesso. Ho quindi trasmesso un messaggio. Dal punto di vista della comunicazione non è detto che quello che ho spedito è quello che riceverà l'utente finale a causa della formattazione UNICODE. Nel mondo multimediale ci possono essere queste incomprensioni, perché la codifica interna di un messaggio posso fruirlo in vari modi.

1.3 Convergenza digitale

Si fa tutto con il telefono, c'è un'integrazione digitale. Da un punto di vista economico è interessante ma con questi problemi che potrebbero sorgere da un punto di vista tecnico.

1.4 Media VS Documents

Parleremo di diversi tipi di medium, testo, immagine, audio, video. Anche il testo lo affronteremo come oggetto multimediale, testo non strutturato che va strutturato, ad esempio che parla di qualcosa e noi vorremo trovare ad esempio tutti i testi che parlano di qualcosa in un DB.

1.5 Media VS Modeling

Studieremo due modelli importanti per la modellazione testuale. Poi vedremo il concetto di immagine per il recupero informativo di immagini. Poi di audio, vedremo l'implementazione e design di agenti conversazionali. Video.

1.6 Media VS Use

Vedremo il media anche dal punto di vista dell'uso. Come usiamo il sistema multimediale?

- Static Media: informazioni che non cambiano nel tempo

- Dynamic media: informazioni che cambiano nel tempo
- Time-based media: informazioni che cambiano nel tempo esono soggetti a limiti temporali

1.7 Media VS Funzionalità

- Codifica
- Decodifica
- Standardizzazione -> tag per descrivere i progetti multimediali (non grande successo)

1.8 MIR - Multimedia Information Retrieval

Ci dedicheremo al recupero di informazioni dai media. MIR sono i media e la tecnologia di storing che permettono di costruire un'architettura multimediale.

Vorremo fare ricerche in queste architetture basate su feature sintattiche.

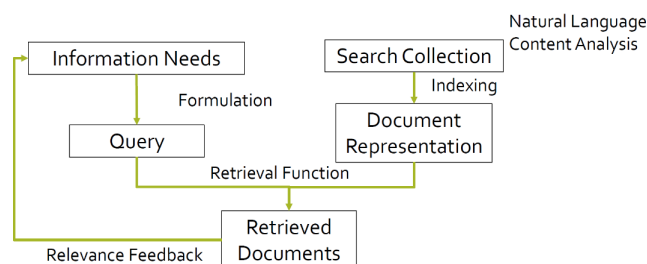
1.8.1 Esigenza esplorativa

Devo poter esplorare vari media per rispondere all'esigenza informativa dell'utente, l'utente cerca una domanda e noi dobbiamo fare in modo di restituire qualcosa che lo soddisfi.

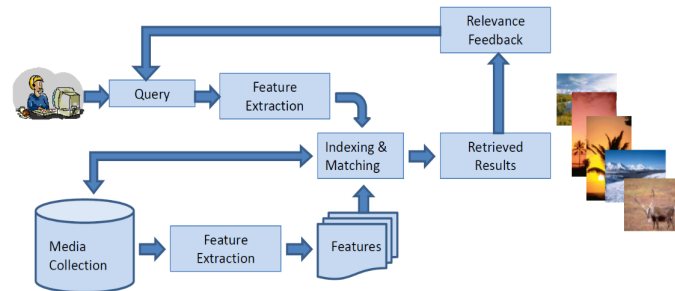
Ho un esigenza informatica, formulo la query ed a questo punto ho una serie di informazioni che devo prendere. A differenza dei modelli relazionali la risposta non sarà mai deterministica. Ci darà una percentuale di successo.

Si parla di big-data.

Esempio, la query è visuale, gli do la monnalisa e vogliamo che ci restituisca tutto quello che ci assomiglia. Devo avere una collezione di immagini, la query, da questa query estraggo informazioni visuali che devono essere metchate con quelle del db per avere una collezione di immagini finali.



1.8.2 Typical MIR Architecture



query, estrazione di feature, un insieme di db con feature, matching delle features per dare una risposta all'utente finale.

affronteremo problemi:

- Storage
- Recupero informativo

1.9 Recupero informativo

query by example, facciamo estrazione delle feature. La strategia riguarderà i colori, le texture e le forme, ma non basterà.

1.10 Audio

E' la stessa cosa, vedremo come interrogare un agente per avere una risposta adeguata al contesto.

1.11 Quality of service

Parleremo dal punto di vista tecnico cosa vuol dire trasferire informazione multimediale. Devo trovare il compromesso migliore per l'utente finale.

Chapter 2

RDBMS

Dobbiamo cambiare l'approccio per gestire le informazioni, quello che vedremo è quello object-relational, esistono in realtà anche altri sistemi per la gestione di informazioni, i no-sql.

E' un modello che cerca di superare il modello relazionale. Per il modello relazionale ci sono tools consolidati, i DBMS che hanno delle ottime performance, sono estremamente utili per avere una gestione accurata delle transizioni come a supporto di un architettura client-server.

Perchè a noi non basta il modello relazionale? ci sono un insieme limitato di dati predefiniti, sono strutture molto flat che non permettono di gestire dato non strutturato, non sono facilmente utilizzabili per rappresentare grafi o strutture dati particolari. Inoltre c'è il problema della modellazione delle associazioni, in quanto potrebbe essere necessario creare molte tabelle ridondanti.

C'è anche questo disaccoppiamento tra lo schema logico e quello che viene implementato, non sempre i modelli concettuali sono fatti per essere adattati facilmente al modello logico e tocca fare delle fasi di ristrutturazione che non danno una fotografia coerente del contesto che stiamo modellando e molto spesso questo crea un disallineamento tra il DBMS e il linguaggio di programmazione. Infatti i RDBMS non sono progettati per sfruttare la tecnologia object-oriented.

2.1 OODBMS - Object-Oriented Databases

Sono stati definiti dei modelli object oriented dei dati, anche nel mondo multimediale ha i suoi vantaggi. Il problema è che dal punto di vista commerciale non abbiamo OODBMS che siano così di ampio utilizzo.

I vantaggi sono legati ad una diretta rappresentazione di oggetti complessi, ho un matching preciso con i linguaggi di programmazione.

Il problema è che non ci sono dei modelli standard e ben definiti.

I modelli relazionali sono usati quando è importante supportare le performance, la sicurezza e soprattutto quando bisogna maneggiare dati semplici. Il modello object oriented non ha avuto un grande uso, nonostante questo sono efficienti nel supportare qualche applicazione che usa dati complessi.

2.2 ORDBMS - Object-relational DBMS

Oggi le cose sono cambiate, pensare di non dover gestire informazione multimediale è impensabile. Se un'azienda ha sempre utilizzato un modello difficilmente lo cambierà in poco tempo. Quindi cambiare completamente il mondo relazionale risulta difficile, perciò i sistemi commerciali più importanti hanno preferito utilizzare il loro motore relazionale (modello) estendendolo con concetti presi dal mondo object-oriented.

Andiamo ad estendere il modello integrando dei dati e concetti più complessi che riguardano il mondo object-oriented.

Non è l'unica cosa che è stata presa dal mondo del object relational ma abbiamo anche il concetto di collezione, di ereditarietà, di metodi (addirittura che ci permetteranno di estrarre informazione visuale dall'immagine). Inoltre abbiamo anche il concetto di tuple, ovvero di collezioni di valori non atomici.

Tipi:

- Complessi
- reference
- user-defined data types

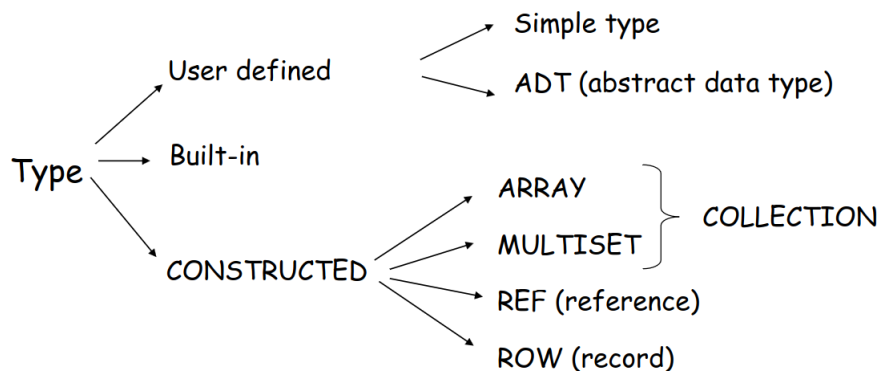
Esistono metodi per modellare l'aspetto dinamico delle istanze dei tipi definiti dall'utente. Esiste anche il concetto di ereditarietà tra i tipi e le tabelle.

L'SQL adotta un insieme di costrutti che sono tipiche del mondo object relational.

C'è uno standard, poi dato che i DBMS nascono relazionali e si estendono non tutti riescono ad assimilare lo standard ed implementarlo a 360 gradi.

2.2.1 Tipi di dati

Sappiamo che nel modello relazionale i tipi sono semplici (stringa, numero, booleano...), sono quelli chiamati built in (predefiniti).



Per ciascun tipo di dato built in ci sono una serie di operazioni predefinite.

Possiamo creare un dato definito dall'utente estendendo un tipo di dato predefinito.

Il tipo di dato che possiamo creare è di diverso tipo:

- Semplice, ad esempio voglio il tipo di dato Euro
- Articolato

la sintassi per creare un tipo di dato semplice:

```
1 CREATE TYPE <name> AS <builtin type>
```

Esempio:

```
1 CREATE TYPE id_employee AS INTEGER
2
3 CREATE TABLE employees(
4     id        id_employee ,
5     name      VARCHAR(50)
6     age       INTEGER
7     id_manager id_employee)
```

Bisogna però stare attenti alle operazioni che si fanno con questi tipi perchè potrebbero esserci dei problemi logici, abbiamo visto l'esempio del confronto tra dollari ed euro.

```
1 CREATE TYPE euro AS NUMBER(8,2);
2 CREATE TYPE dollar AS NUMBER(8,2);
3
4 CREATE TABLE European_Sales(
5     n_customer    NUMBER,
6     n_order       NUMBER,
7     total         euro)
8
9 CREATE TABLE USA_Sales(
10    n_customer    NUMBER,
11    n_order       NUMBER,
12    total         dollar)
13
14 SELECT ERP.n_customer, ERP.n_order
15 FROM European_Sales ERP, USA_Sales USA
16 WHERE ERP.n_order = USA.n_order
17 AND ERP.total > USA.total; --> ERROR!!!
```

Per risolvere questa cosa bisogna creare dei casting, mi dice cosa devo fare quando c'è un confronto o assegnamento tra due dati confrontabili, la signature function è la signature (l'interfaccia) della funzione che andrà ad implementare il confronto.

```
1 CREATE CAST (<source type> AS <target type>)
2 WITH <function signature>
3 [AS ASSIGNMENT]
```

Una funzione è sempre definita in questo modo:

```
1 FUNCTION <name> (<source type>) RETURNS <target type>
2 ... code ...
```

Quando facciamo assegnamenti o confronti con dati semplici fa sempre il cast implicito trasformando in tipi semplici, se invece avessi creato il mio cast lo avrebbe fatto anche senza che lo avessi scritto.

```
1 SELECT name
2 FROM employees
3 WHERE id_manager = CAST(123 AS id_employee);
```

Tornando all'esempio di prima euro-dollaro devo creare la funzione di cast, che prende in input un parametro euro e lo trasforma in dollaro. Deve essere attivata ogni volta che c'è il confronto euro dollaro. Attenzione che se dovessi fare confronti dollaro-euro dovrei crearne una nuova.

```
1 CREATE or replace FUNCTION Euro2Dollar(e euro)
2     RETURN Dollar
3     BEGIN
4         SET g = e*1.2;
5         RETURN(g);
6     END;
7 CREATE CAST(Euro AS Dollar)
8 WITH Euro2Dollar(Euro)
9 AS ASSIGNMENT
```

Notiamo che creo prima la funzione che effettivamente mi fa la conversione e poi la uso nel cast.

Complex values

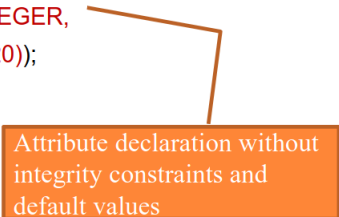
Come detto ci serve gestire dati complicati:

- Collezioni (tuple)
- Dato strutturato

Una delle prime soluzioni era quella di usare un tipo di dato chiamato ROW che simulasse il concetto di record.

```
CREATE TABLE Customer
(codC DECIMAL(4) PRIMARY KEY,
name VARCHAR(20),
surname VARCHAR(20),
tel CHAR(15),
dateB DATE,
residence ROW(street VARCHAR(20),
               no VARCHAR(10),
               postal_code INTEGER,
               city VARCHAR(20));

INSERT INTO Customer
VALUES (1245, "Mario", "Rossi",
       "02 453628", "10/11/1970",
       ROW("Via Roma", "3/12",
          16100, "Genova"))
```



Attribute declaration without integrity constraints and default values

È stato inventato l'abstract data type che è meglio del tipo raw.

Il vantaggio:

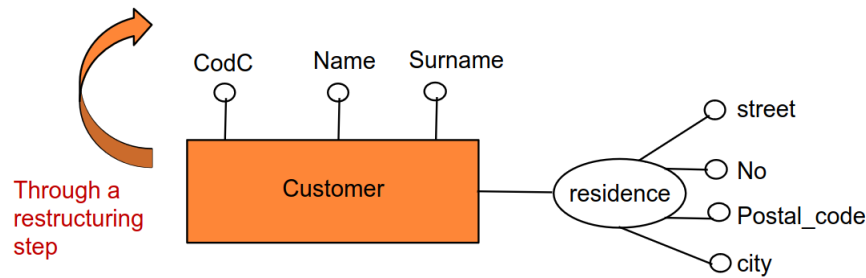
- Schema ben definito, quindi possiamo riutilizzare i dati che definiamo
- Sono legati all'utilizzo
- Hanno grande flessibilità nel definire associazioni tra tuple

includono:

- uno o più attributi
- uno o più metodi

Esempio: Mettiamo di avere il classico schema relazionale con un'entità customer con un attributo che è una composizione di attributi, non possiamo tenere questa cosa e va ristrutturata. Quello che vogliamo fare è creare un tipo di dato che rappresenti questa struttura dati, tramite il create type :

```
1 CREATE TYPE <type name>
2 AS OBJECT <attribute definitions>
3 NOT FINAL
```



Nel nostro esempio:

```
1 CREATE TYPE t_residence AS OBJECT
2 (city VARCHAR(20) DEFAULT 'milano',
3 street VARCHAR(20),
4 no INTEGER,
5 postal_cose NUMBER DEFAULT '20135')
6 NOT FINAL
```

Posso creare un tipo di dato all'interno del quale un attributo è un tipo di dato creato precedentemente:

```
1 CREATE TYPE t_customer AS OBJECT
2 (codC NUMBER(4),
3 name VARCHAR(20),
4 surname VARCHAR(20),
5 tel CHAR(15),
6 dateB DATE,
7 residence t_residence)
8 NOT FINAL;
```

La tabella in questo esempio verrebbe popolata così:

codC	name	surname	...	residence			
				street	no	p_c	city
6610	anna	rossi	...	via scribanti	16	20135	milano
6635	paola	bianchi	...	via dodecaneso	35	20135	milano
6642	marco	verdi	...	Via lagustena	35	20133	milano

Ovviamente un oggetto lo devo istanziare, quindi quando vado a creare l'oggetto devo usare il suo costruttore (potrebbe essere un costruttore che crea un oggetto nullo) con il `NEW t_residence()`.

2.2.2 Array

È una collezione di una dimensione definita di tipo di dato che vogliamo. L'array è un insieme numerato, predefinito, con una dimensione specificata a livello di istanza di particolari elementi.

```
1 <component_type> ARRAY [<dimension>]
```

Non è possibile definire dei constraint su una collezione. Ovviamente devo creare l'array tramite uno specifico costruttore, dopo di che ho un insieme di operazioni come

la concatenazione, la cardinalità, l'accesso a un elemento, l'assegnamento e operazioni di comparazione.

```
1 CREATE TABLE Customer(  
2     codC NUMBER (4) PRIMARY KEY,  
3     name VARCHAR(20),  
4     surname VARCHAR(20),  
5     tel CHAR(20) ARRAY [3],  
6     dateB DATE,  
7     residence t_residence);  
  
1 INSERT INTO Customer VALUES  
2 (6635,'paola', 'banchi', ARRAY[], '15-Mag-1970', 'vai');  
3  
4 SELECT tel [1]  
5 FROM Customer;  
6  
7 SELECT name, surnam  
8 FROM Customer  
9 WHERE tel[2]="333771994";  
10  
11 UPDATE Customer  
12 SET tel = ARRAY["026473456","3336745383"]  
13 WHERE codC = 6642;
```

Posso creare una tabella relazionale definendo un array predefinito di 3 elementi. Quando creo una riga anche se l'array è vuoto viene creato un array di 3 elementi. Per fare il set devo usare l'istruzione SET.

2.2.3 Multiset

Collezione di elementi non ordinati, c'è la possibilità di duplicazione, a differenza degli array è di cardinalità non specificata.

```
1 <component_type> MULTISSET
```

Esempio:

```
1 CREATE TABLE Customer(  
2     codC NUMBER (4) PRIMARY KEY,  
3     name VARCHAR(20),  
4     surname VARCHAR(20),  
5     tel CHAR(15) MULTISSET,  
6     dateB DATE,  
7     residence t_residence);
```

Ci sono un insieme di funzioni che posso fare sui multiset:

```
1 <expr1>, <expr2> of type MULTISSET  
2 <expr1> [NOT] MEMBER [OF]  
3 <expr2>  
4 <expr1> SUBMULTISSET <expr2>  
5 <expr> IS A SET
```

Posso popolare il multiset tramite una subquery, basta inserire la query nel costruttore del multiset:

```
1 MULTISET [SELECT phone FROM customers1 WHERE city = 'Milan']
```

Funzionalità:

- CARDINALITY(value)
- SET(value): prende il multiset e ne elimina i duplicati
- ELEMENT(value): value deve essere un multiset di cardinalità uno e viene recuperato il suo elemento
- UNNEST(value) AS correlated-name: per accedere ad un multiset devo creare una tabella fittizia collegata alla collezione, che è una tabella di un'unica colonna. Per accedere a questa tabella fittizia viene utilizzata la funzione di UNNEST. Una volta che ho la tabella posso farci quello che voglio.

```
1 SELECT SUM(t.c)
2 FROM UNNEST(MULTISET(2,3,5,7)) AS t(c)
```

Tutte le altre operazioni sono le classiche operazioni insiemistiche per poter fondere collezioni, fare intersezioni e così via:

- COLLECT: trasforma i valori di un gruppo in un MULTISET
- FUSION: aggrega i valori di due MULTISET
- INTERSECTION: intersezione di due MULTISET
- MEMBER: ci dice se un elemento fa parte di un insieme

La cosa particolare dei multiset è che sono una collezione, anche di oggetti, e la funzione UNNEST che ci permette di accedere agli elementi della collezione.

2.2.4 Metodi

Una caratteristica degli abstract data type è che posso avere dei metodi. Per quanto riguarda la creazione del tipo di dato viene richiesta la definizione della signature (interfaccia del metodo).

```
1 CREATE TYPE t_film AS
2 (titolo VARCHAR(30),
3 regista VARCHAR(20),
4 anno DECIMAL(4),
5 genere CHAR(15),
6 valutaz NUMERIC(3,2))
7 NOT FINAL
8 INSTANCE METHOD assegna_val() RETURNS t_film;
```

Di base viene utilizzato, per evitare di usare linguaggi specifici, un linguaggio SQL con estensione al mondo procedurale. Non è altro che un linguaggio molto semplice (PLSQL) che estende l'SQL con costrutti di un linguaggio procedurale.

Ci sono metodi di che possono essere invocati su ogni istanza dell'oggetto usando la dot notation, statici che devono essere preceduti dal simbolo "::" e i metodi costruttori che vengono invocati tramite la keyword NEW.

La sintassi dei metodi di istanza:

```
1 [INSTANCE | STATIC | CONSTRUCTOR ]
2 METHOD <method name> <parameter list >
3 [RETURNS <return type>]
4 [SELF AS RESULT]
5 [< method characteristics >];
```

SELF AS RESULT è la clausola che si chiama quando ci sono i metodi costruttore, il risultato è l'oggetto stesso.

Il body della funzione si crea con CREATE INSTANCE METHOD, dopo la FOR dico qual'è il tipo di dato in cui ho dichiarato la signature:

```
1 CREATE [INSTANCE | STATIC | CONSTRUCTOR]
2 METHOD <method name>
3 <parameter list>
4 [RETURNS <return type>]
5 FOR <name of structured type>
6 <method body>;
```

Esempio:

```
1 CREATE TYPE t_film AS
2   (titolo VARCHAR(30),
3    regista VARCHAR(20),
4    anno DECIMAL (4),
5    genere CHAR(15),
6    valutaz NUMERIC(3,2))
7 NOT FINAL
8 CONSTRUCTOR METHOD t_film
9   (titolo VARCHAR(30),
10    regista VARCHAR(20),
11    anno DECIMAL(4),
12    genere CHAR(15),
13    valutaz NUMERIC(3,2))
14 RETURNS t_film
15 SELF AS RESULT,
16 INSTANCE METHOD assegna_val()
17   RETURNS t_film;
```

Una volta che ho creato i metodi, sia a livello di signature che di body, posso utilizzarli nelle classiche operazioni SQL. Innanzitutto instancio un oggetto nuovo passando i parametri al costruttore.

Per chiamare una funzione di istanza faccio un update della tabella, vado a prendere l'oggetto.

```
1 INSERT INTO Video
2 VALUES (1131, NEW t_film('la fabbrica di cioccolato', 'tim
3 burton',2005,'fantastico',4.00), 'd');
4
5 UPDATE Video
6 SET film = film.assegnaVal()
7 WHERE colloc = 1131
```

Un altro esempio in cui creiamo una funzione guadagno che restituisce la differenza tra il prezzo di vendita ed il prezzo di acquisto:

```
1 CREATE TYPE t_libro AS
2     (titolo CHAR(20),
3      p_vendita DECIMAL(9,2),
4      p_acquisto DECIMAL(9,2))
5 NOT FINAL
6 INSTANCE METHOD guadagno()
7 RETURNS DECIMAL(9,2);
8
9 CREATE INSTANCE METHOD guadagno()
10 RETURNS DECIMAL(9,2)
11 FOR t_libro
12 BEGIN
13     RETURN SELF.p_vendita - SELF.p_acquisto;
14 END;
15
16 CREATE TABLE biblioteca
17     (codL# INTEGER,
18     libro t_libro);
19
20 SELECT libro.guadagno( )
21 FROM biblioteca
22 WHERE libro.titolo() =
23     "La Divina Commedia";
```

2.2.5 Tabella tipata

Una tabella tipata è una tabella che viene costruita sopra un tipo di dato astratto. Vado a generare una tabella costruendola sopra un tipo di dato astratto, ha delle peculiarità che la rendono molto più vantaggiosa di una tabella relazionale classica.

La tabella tipata è una tabella che ha una grossa caratteristica, non ha la necessità di una primary key. Creare una tabella tipata su un tipo è un mapping 1 a 1, ha tante colonne quanto gli attributi del tipo. Non devo dichiarare una primary key perchè ogni riga della tabella è un oggetto ed ogni oggetto ha un suo identificativo interno nel sistema. Nulla toglie però che in una tabella tipata io possa creare una chiave primaria. Definire in questo caso una chiave primaria è per prevenire errori futuri, tipo la duplicazione della targa.

Identifier:

- E' creato dal sistema
- E' indipendente dallo stato della tupla
- Non può essere aggiornato
- Non ha significato semantico

Primary key:

- É sempre gestito dall'utente

- È definito sopra un valore o un set di attributi
- È aggiornata dall'applicazione
- Ha significato semantico

Posso creare diverse tabelle tipate sullo stesso tipo di dato.

Come viene dichiarata:

```

1 CREATE TABLE <table_name>
2 OF < structured type name >
3 [(
4   [< attribute options >]
5   [< constraints >]
6   [< identification >]
7 )]
```

- <structured type name>: è il nome del tipo astratto su cui è costruita la tabella
- <identification>: specifica informazioni riguardo la generazione degli identificatori. La sintassi è REF IS <column name>, il nuovo attributo apparirà come primo attributo nello schema della tabella e potrà essere usato come qualsiasi altro attributo

Nell'esempio dichiariamo primary key titolo e regista in modo tale che nel dominio applicativo siano univoci. L'unica cosa obbligatoria è identificare la colonna dove ci sarà l'identificatore dell'oggetto (me lo dice lo standard ma Oracle non me lo chiede).

```

1 CREATE TYPE t_film AS
2   (titolo VARCHAR(30),
3    regista VARCHAR(20),
4    anno DECIMAL(4),
5    genere CHAR(15),
6    valutaz NUMERIC(3,2))
7 NOT FINAL;
8
9 CREATE TABLE Film OF t_Film
10   (anno WITH OPTIONS NOT NULL
11    genere WITH OPTIONS NOT NULL
12    PRIMARY KEY (titolo, regista)
13    REF IS idF);
```

Per poter accedere a tuple di una tabella tipata devo usare un tipo di dato nuovo, chiamato dato di riferimento, che è una sorta di puntatore per poter accedere a degli oggetti di una tabella tipata. La colonna film contiene dei puntatori a degli oggetti di una tabella tipata t_film.

```

1 CREATE TABLE Video
2   (colloc DECIMAL(4) PRIMARY KEY,
3    film REF(t_film),
4    tipo CHAR NOT NULL DEFAULT "d"
5 );
```

2.2.6 Reference types

Si usa la clausola REF:

```
1 REF (<structured type>) [SCOPE <table_name>]
```

Creo il tipo di film, che è un tipo strutturato:

```
1 CREATE TYPE t_film AS
2   (titolo VARCHAR(30),
3    regista VARCHAR(20),
4    anno DECIMAL(4),
5    genere CHAR(15),
6    valutaz NUMERIC(3,2))
7 NOT FINAL;
```

Creo la tabella tipata t_film, con la clausola WITH OPTIONS inserisco dei constraint di integrità inoltre assegno ad ogni tuple in Film un identificatore tramite l'attributo idF:

```
1 CREATE TABLE Film OF t_film
2   (anno WITH OPTIONS NOT NULL,
3    genere WITH OPTIONS NOT NULL,
4    PRIMARY KEY (titolo, regista),
5    REF IS idF);
```

A questo punto posso inserire oggetti nella tabella, che non è il classico INSERT INTO, ma crea un oggetto tramite il costruttore e lo mette come riga della tabella tipata.

La clausola SCOPE, permette di istanziare puntatori sulla tabella tipata che definisco dopo lo scope, limito l'accesso ai puntatori di una specifica tabella tipata.

La clausola SCOPE posso usarla anche in scope di creazione di tipo di dato, ho un attributo che è un riferimento, questo non è vincolante, è solo un preferito.

Mettiamo di creare un video nuovo in una tabella relazionale classica, non posso però sapere qual'è il riferimento, potrei farlo andando a fare un set successivo recuperando l'identificatore del film che è collocato in 1130.

In realtà si possono comprimere le due istruzioni in una INSERT INTO unica.

```
1 INSERT INTO Video VALUES (1130, NULL, "d")
2
3 UPDATE Video
4 SET film = (SELECT idF FROM Film
5             WHERE titolo = "big fish" AND regista = "tim burton")
6 WHERE colloc = 1130;
7
8 or
9
10 INSERT INTO Video
11 SELECT 1130, idF, "d" FROM Film
12 WHERE titolo = "big fish" AND regista = "tim burton";
```

Posso creare anche una sorta di recursione, metto come attributo di un oggetto un riferimento a se stessa.

```
1 CREATE TYPE t_film1 AS
2   (titolo VARCHAR(30),
3    regista VARCHAR(20),
4    anno DECIMAL(4),
```

```

5     genere CHAR(15),
6     valutaz NUMERIC(3,2),
7     rifacimento_di REF(t_film1)
8 NOT FINAL;
9
10 CREATE TABLE Film1 OF t_film1
11     (rifacimento_di WITH OPTIONS SCOPE Film1,
12      PRIMARY KEY (titolo, regista)
13      REF IS idF);

```

Il tipo riferimento è fondamentale perchè elimina il concetto di chiave esterna, tant'è che non si dice più una navigazione tramite associazione ma tramite riferimento. Si usano delle particolari funzioni che si chiamano Deref, che mi serve per trasformare l'oggetto in una tabella di una singola riga, navigo da un'espressione con un reference type ad una referenced tuple di una tabella.

```

1 SELECT film
2 FROM Video
3 WHERE colloc = 1130;
4
5 SELECT Deref(film)
6 FROM Video
7 WHERE colloc = 1130;
8
9 SELECT Deref(film).titolo
10 FROM Video
11 WHERE colloc = 1130;
12
13 SELECT film -> titolo
14 FROM Video
15 WHERE colloc = 1130;

```

Notiamo che in questo esempio è come se fosse fatto un join implicito tra la tabella Video e Film, è come se facessimo Video.film = Film.idF. Corrisponde a questa query:

```

1 SELECT Film.titolo
2 FROM Video, Film
3 WHERE Video.film = Film.idF and colloc = 1130;

```

2.2.7 Ereditarietà

Ci sono due tipi di ereditarietà:

- Ereditarietà tra tipi
- Ereditarietà tra tabelle

Tra tipi è la classica ereditarietà che abbiamo nel mondo della programmazione ad oggetti.

A livello di sintassi metto INSTANTIABLE cioè che posso creare istanze anche del padre, UNDER significa da chi sto ereditando.

```

1 CREATE TYPE t_cliente AS
2     (codCli DECIMAL(4),

```

```

3     nome VARCHAR(20),
4     telefono CHAR(15),
5     dataN DATE,
6     residenza VARCHAR(20))
7 INSTANTIABLE
8 NOT FINAL;
9
10 CREATE TYPE t_standard
11 UNDER t_cliente AS
12     (ptiMancanti INTEGER)
13 NOT FINAL;
14
15 CREATE TYPE t_VIP
16 UNDER t_cliente AS
17     (bonus INTEGER)
18 NOT FINAL;

```

Tra tabelle, se t1 è padre di t2 la tabella tipata di t1 deve essere padre della tabella tipata t2. Mantiene il concetto di ereditarietà tra tipi anche tra tabelle, lo schema di una tabella tipata deve sempre coincidere con lo schema del tipo della riga.

```

1 CREATE TABLE Cliente OF t_cliente
2 (REF IS idC
3 PRIMARY KEY(codCli)
4 UNIQUE(dataN,nome,cognome));
5 CREATE TABLE Standard
6 OF t_Standard
7 UNDER Cliente;
8 CREATE TABLE VIP
9 OF t_VIP
10 UNDER Cliente;

```

La particolarità è che la tabella tipata deve rispettare l'ereditarietà, una tabella tipata figlia deve essere costruita su un tipo figlio.

Ha la sua utilità perchè mi permette di espandere delle interrogazioni. Le clausole vengono espanse anche alle tabelle figlie.

Attenzione che la stessa cosa funziona anche con le update e le delete, si propaga la delete dal padre ai figli.

Posso evitare la propagazione con delle particolari clausole, c'è la TREAT function che viene usata per andare a recuperare un attributo che è specifico del figlio.

```

1 SELECT name, TREAT(VALUE(c) AS t_vip).bonus FROM Cliente c;

```

Methods e ereditarietà

Si può fare l'override di un metodo usando la keyword OVERRIDING.

Esempio:

```

1 CREATE TYPE t_VIP
2 UNDER t_cliente AS
3     (bonus INTEGER)
4 OVERRIDING INSTANCE
5 METHOD punteggio() RETURNS INTEGER
6 NOT FINAL

```

```

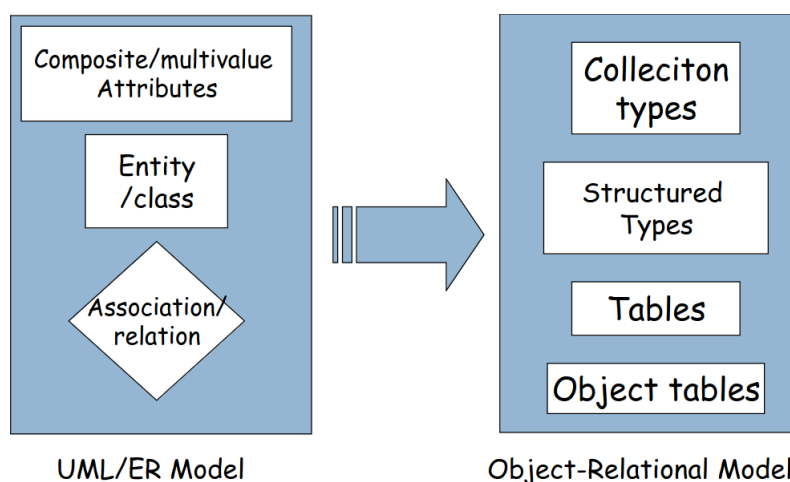
7
8 CREATE TYPE t_standard
9 UNDER t_cliente AS
10     (ptiMancanti INTEGER)
11 OVERRIDING INSTANCE
12 METHOD punteggio() RETURNS INTEGER
13 NOT FINAL
14
15 CREATE INSTANCE
16     METHOD punteggio()
17 RETURNS INTEGER
18 FOR t_VIP
19 BEGIN
20     RETURN SELF.bonus;
21 END
22
23 CREATE INSTANCE
24     METHOD punteggio()
25 RETURNS INTEGER
26 FOR t_standard
27 BEGIN
28     RETURN SELF.PtiMancanti;
29 END

```

2.3 Dal design concettuale al modello object-relational

La modellazione tra modello concettuale a modello logico relazione è molto preciso e consolidato, in questo caso abbiamo da fare una fase di ristrutturazione perchè abbiamo delle cose che non sono modellabili (attributi multivalore, ereditarietà, associazioni molti a molti...).

Per quanto riguarda il modello object relational le cose non sono così fisse. Si parte sempre da un modello o ER o UML, solitamente in questo mondo si preferisce avere una modellazione UML perchè supporta anche il concetto di classi e metodi. Nel caso di modello ER parliamo di entità nel caso di modello UML parliamo di classi.



Devo decidere quando creare delle collezioni di tipi, quando creare tabelle relazionali

e quando tabelle tipate. Abbiamo diverse scelte a livello di costrutto da dover utilizzare.

2.3.1 Modello ER

Nel modello ER non ho più bisogno di fare ristrutturazione, posso modellare tutto. Se c'è una attributo composito questo diventa un object type, la stessa cosa la faccio con gli attributi multivalore (più di un numero di telefono) mappandoli come un collection type (generalmente userò un multiset per essere libero sulla cardinalità).

Se ho delle entità:

- Se c'è una gerarchia la trasformo in una gerarchia, faccio una gerarchia dei tipi e eventualmente delle tabelle
- Non ho gerarchia, devo chiedermi quando creare una tabella relazionale e quando una tabella tipata (quindi con relativo tipo). È la parte più soggettiva della modellazione, se ho dei riferimenti all'entità è bene modellarli con i tipi riferimento (i video devono riferirsi ad un film, la tabella riferita deve essere una tabella tipata). È bene creare una tabella tipata anche quando è rilevante per un altro dominio e quando è un entry point per delle query, ad esempio l'anagrafe di una persona in un comune perchè tutte le query si proietteranno su questa tabella, questo perchè le select sono più semplici da fare in quanto rimuovo i join.

2.3.2 UML

- Ogni tipo composito diventa un object type
- Ogni tipo multivalore (list) diventa un collection type (multiset)

Ogni classe :

- Se non ha metodi e non appartiene ad una gerarchia di ereditarietà: viene creata una tabella, gli attributi vengono rappresentati come in ER
- Se ha metodi e/o appartiene ad una gerarchia di ereditarietà: viene creata una object table, come prima se è una tabella riferita, rilevante o entry point è bene che ci sia una tabella tipata

2.3.3 Associazioni

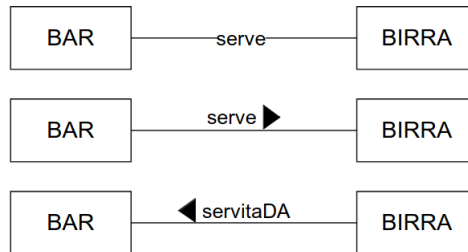
Creare tabelle tipate va a definire come poi mappiamo le associazioni, vogliamo navigazione con join o con riferimenti?

Generalmente vorremmo sempre una modellazione tramite riferimento.

L'aspetto negativo di riferimento è il concetto di bidirezionalità, perchè con i riferimenti possiamo leggere la relazione da destra verso sinistra e viceversa. Quando vogliamo mantenere la bidirezionalità bisogna sempre usare una funzione (scritta da noi) per mantenere la consistenza del database.

Esempio: vogliamo modellare il sito web in cui vogliamo presentare tutti i PUB di Milano e quello che questi sono in grado di offrire. Da un lato potremmo volere quale birra

serve un bar e dall'altro una birra è servita da quali bar. In questi casi gli entry point sono opposti. Scegliere il verso di lettura non è banale, perchè il verso di lettura implicherà un'implementazione diversa ed un modello OR diverso. Potrei modellare entrambe le direzioni ma ci sarebbe un sovraccarico esagerato.



Una birra è servita da un solo bar oppure una birra può essere servita da più bar? In questo caso mettiamo NO, quindi una singola birra è servita da diversi bar. Quindi devo evitare duplicazioni:

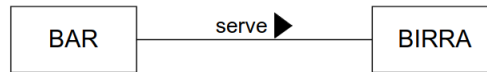
- Creo il tipo tbirra
- Creo la tabella tipata (object table) Birra a partire dal tipo tbirra
- Viene creato un attributi nella tabella BAR per rappresentare il fatto che la birra sia servita "serve":
 - REF tbirra SCOPE Birra (quando un bar serve una singola birra)
 - REF tbirra MULTISSET SCOPE Birra(quando il bar serve tante birre)

```

1 CREATE TYPE tbirra AS ( attributi di birra );
2 CREATE TABLE Birra OF tbirra(...);
3 CREATE TABLE Bar(..attributi di bar...
4     serve REF tbirra MULTISSET
5     SCOPE Birra)

```

Questo è il caso in cui un bar serve diverse birre:



idB	nome		serve
74	Gigi		MULTISET[#1,#2,#3]
86	Sport		MULTISET[#1,#4]

Bar

idB	nome	
#1	Adelscott	
#2	Moretti	
#3	Chemay	
#4	Desert	

Birre

Vediamo degli esempi di query:

dammi le birre che serve il bar, questa query segue il verso di lettura, attenzione che devo usare l'unnest:

```
1 SELECT idB -> nome
2 FROM Bar b, UNNEST(b.serve) T(idB)
3 WHERE b.nome='Sport'
```

trova i bar che servono la birra Adelscott, questa query va nel verso opposto e può essere molto onerosa, perchè se abbiamo tanti multiset devo scansionarli tutti:

```
1 SELECT b.nome
2 FROM bar b, UNNEST(b.serve) T(idB)
3 WHERE idB -> nome='Adelscott'
```

Nel caso in cui una birra venisse servita da un solo bar non avrebbe senso creare riferimenti, creerei una un attributo nella tabella Bar che è un multiset di tipi tbirra.

```
1 CREATE TYPE tbirra AS (...attributes for beer...);
2 CREATE TABLE Bar(..attributes for bar...
3     serve tbirra MULTISET)
```

In questa rappresentazione se un bar serve diverse birre non c'è modo di evitare la duplicazione dell'informazione:

Anche in questo caso vediamo degli esempi di query: trova il nome delle birre che serve il bar the Bar Sport:

```
1 SELECT T.nome
2 FROM Bar b, UNNEST(b.serve) T(nome, ...)
3 WHERE b.nome='Sport'
```

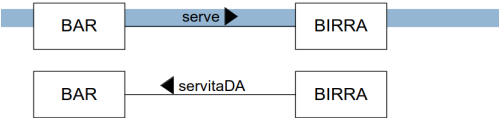
trova i bar che servono la birra 'Adelscott':

```
1 SELECT b.nome
2 FROM bar b, UNNEST(b.serve) T(nome, ...)
3 WHERE T.nome='Adelscott'
```

id	nome		serve
74	Gigi		nome
			Adelscott
			Moretti
			Chemay
86	Sport		nome
			Horne
			Desert

Bar

La scelta di mantenere tutti e due i versi non viene mai fatta, quello che viene fatto solitamente è scegliere il senso di lettura a priori.



idB	nome		serve
#74	Gigi		MULTISET[#1,#2,#3]
#86	Sport		MULTISET[#1,#3,#4]

Bar

idB	nome		servitaDa
#1	Adelscott		MULTISET[#74,#86]
#2	Moretti		MULTISET[#74]
#3	Chemay		MULTISET[#74,#86]
#4	Desert		MULTISET[#86]

Birre

Chapter 3

Oracle

Utilizzeremo il DBMS Oracle 19c. Perchè è un DBMS object relational molto utilizzato e ha un modulo multimedia che è stato fatto apposta per poter storicizzare ed interrogare oggetti multimedia, ed è estremamente potente soprattutto nella sua parte text mining.

SQL:2003	Oracle
Row types	no
Structured types	Object types
Reference types	Reference types
Collection types (MULTISET)	Collection types (VARRAY, NESTED TABLE)
Typed relations	Object tables
Type Inheritance	Type Inheritance
Relation inheritance	substitutability of the types
Methods	Methods

Sulle slide ci sono delle indicazioni di massima sul motore, ma non lo vedremo nel dettaglio. Quello che ci interessa è il concetto di schema, un accesso ad Oracle significa che all'interno del server viene creato uno schema a nostro nome, all'interno di questo schema abbiamo una visibilità su quello che abbiamo creato. Lo schema è un'implementazione all'interno del table space in cui creiamo una serie di oggetti che ci serviranno (tabelle, tipi, procedure, funzioni, viste...). Tutti gli schemi hanno la possibilità di vedere alcune tabelle particolari, gestiti da utenti creati di default, vedremo alcuni di questi utenti perchè dovremo usare alcune tabelle.

Alcuni dizionari dati sono visibili da tutti altri solo da chi ha alcuni ruoli. Esistono anche i privilegi ad esempio possiamo accedere a dei dati, modificarli o anche a fare grant (dare i privilegi a qualcun altro). Il ruolo è invece una collezione di privilegi. Noi nel nostro account abbiamo il ruolo di Dbadministrator.

3.1 Data type

Abbiamo i tipi built in. Abbiamo alcuni particolari oggetti multimediali. Dati di conversione, regole, componenti XML e i tipi di dati definiti dall'utente.

3.2 SQL Developer

Per accedere ad Oracle possiamo usare due client, SQL Developer, oppure un'estensione di visual studio.

In SQL Developer dobbiamo innanzitutto creare una nuova connessione (più verde), il nostro nome utente è <nome utente>, ruolo lasciamo quello predefinito, la password è <password>, i dettagli di connessione ip: <indirizzo del server> (funziona solo sulle macchine della rete di ateneo) per accedere da fuori (anche Eduroam) un altro indirizzo, <indirizzo del server>, la porta è sempre 1521, il sid è l'ascoltatore di Oracle (listener) oracle19 cioè stiamo contattando il server chiamando il listener oracle19.

3.3 Object types

Il nome dei dati generati dall'utente è diverso ma sono uguali.

Sintassi:

```
1 CREATE [OR REPLACE] TYPE <type name>
2 [AS OBJECT | UNDER <super-type name >]
3 ([[<attributes-list>],[<methods>]])
4 [[NOT] FINAL | [NOT] INSTANTIABLE];
```

Non è permessa la clausola di DEFAULT nella creazione dei tipi.

3.4 Metodi

Useremo il PLSQL, ma in realtà si può usare qualsiasi linguaggio. Esistono tipi:

- Istanza/Member
- Statici
- Costruttori

3.4.1 Metodi member

Sono usati per accedere ai valore di un istanza di un oggetto (hanno il parametro SELF implicito):

```
1 CREATE OR REPLACE TYPE BODY person_type AS
2     MEMBER FUNCTION get_areacode RETURN VARCHAR2 IS
3     BEGIN
4         RETURN SUBSTR(SELF.phone, 1, 3);
5     END get_areacode;
6 END;
7
8 SELECT c.contact.get_areacode()
9 FROM contacts c;
```

3.4.2 Metodi costruttori

Ogni object type ha un costruttore implicito definito dal sistema, possiamo dichiararne uno esplicito, può anche essere sovrascritto con OVERRIDE:

```
1 [STATIC|MEMBER] [
2 PROCEDURE <name> ({<par> <type par>[,]}*) |
3 FUNCTION <name> ({<par> <type par>[,]}*) RETURN <returned type>
4
5 CONSTRUCTOR FUNCTION <object type> (
6     [SELF IN OUT <object type>[,] [{<par> <type par>[,]}*])
7 RETURN SELF AS RESULT
8
9 [MAP|ORDER] MEMBER
10 FUNCTION <name> ({<par> <type par>[,]}*) RETURN <returned type>
```

Esiste il concetto di commit, creiamo una cosa e poi dobbiamo fare la commit per crearla effettivamente, si mette un \ alla fine.

```
1 CREATE TYPE person_typ AS OBJECT
2   ( idno NUMBER,
3     first_name VARCHAR2(20),
4     last_name VARCHAR2(25),
5     email VARCHAR2(25),
6     phone VARCHAR2(20),
7     MAP MEMBER FUNCTION get_idno RETURN NUMBER,
8     MEMBER PROCEDURE display_details (SELF IN OUT NOCOPY
9       person_typ));
10 /
11 CREATE TYPE BODY person_typ AS
12   MAP MEMBER FUNCTION get_idno RETURN NUMBER IS
13   BEGIN
14     RETURN idno;
15   END;
16   MEMBER PROCEDURE display_details ( SELF IN OUT NOCOPY person_typ )
17   IS
18   BEGIN
19     -- use the PUT_LINE procedure of the DBMS_OUTPUT package to
20     display details
21     DBMS_OUTPUT.PUT_LINE(TO_CHAR(idno) || ' ' || first_name || ' '
22       || last_name);
23     DBMS_OUTPUT.PUT_LINE(email || ' ' || phone);
24   END;
25 END;
```

3.5 Tabelle

Gli oggetti possono essere messi in due tipi di tabelle:

- Object tables: ogni riga è un oggetto
- Relational tables: segue il modello relazionale

```
1 CREATE TABLE tablename OF typename
2   ({<attrname> NOT NULL[,]}*
3   [,PRIMARY KEY (attrname {,attrname })]);
```

I CONSTRAINT devo metterli nella dichiarazione delle tabelle.

3.6 PLSQL

È un linguaggio procedurale definito da Oracle, ci permette di implementare dei metodi e delle procedure in modo molto semplice e veloce.

3.6.1 Blocco di istruzioni

Abbiamo una procedura che inizia con `begin` e termina con `end`, possiamo gestire delle eccezioni e dichiarare variabili.

```
1 --blocco senza nome
2 declare
3 ...
4 begin
5 ...
6 end;
```

Le stored procedure sono delle funzioni che vengono create nello schema Oracle.

```
1 -- stored procedure
2 create or replace procedure proc_name
3 as
4 -- dichiarazione di variabili
5 begin
6 ...
7 exception
8 ...
9 end;
```

I costrutti di controllo sono quelli tipici della programmazione procedurale:

```
1 if var1 > 10 then
2     var2 := var1 + 20;
3 end if;
4
5 if var1 > 10 then
6     var2 := var1 + 20;
7 else
8     var2 := var1 * var1;
9 end if;
10
11 cnt := 1;
12 loop
13     cnt := cnt + 1;
14     if cnt >= 100 then exit;
15     end if;
16     ...
17     ...
18 end loop;
19     ...
20
21 for cnt in 1 .. 100 loop
22     insert into T1 values ('loop', cnt);
23 end loop;
```

3.6.2 Gestione eccezioni

Sono molto utili perchè ci permettono di catturare delle tipiche eccezioni del mondo DB. Esempi di errori predefiniti che possono essere gestiti sono:

- `no_data_found`

- too_many_rows
- dup_val_on_index
- value_error

```

1 CREATE PROCEDURE double ( original IN VARCHAR2,
2                           new_string OUT VARCHAR2 )
3 AS
4 BEGIN
5     new_string := original || original;
6     EXCEPTION WHEN VALUE_ERROR THEN
7         dbms_output.put_line('Output buffer not long enough.');
```

3.6.3 Cursore

É la cosa particolare. Il cursore ci serve per parsare il risultato di una query. All'interno di una funzione potrei fare una query e la query mi risponde più di una riga, in questo caso devo trovare il modo per scorrere la risposta e utilizzarla nel codice. Per scorrere la risposta viene utilizzato il concetto di cursore, diventa il puntatore, che una volta letto scala e passa alle successive righe, finchè non ne trova più.

É uno strumento molto importante che troviamo spesso negli script PLSQL.

É possibile dichiarare un cursore, ogni volta che lo dichiaro è sempre legato ad una query (che non viene eseguita in questo momento):

```

1 CURSOR c1 IS SELECT empno,ename FROM emp WHERE sal>2000;
2 CURSOR c2 IS SELECT * FROM dept WHERE deptno = 10;
```

Il cursore si apre con l'operazione OPEN:

```

1 DECLARE
2     CURSOR c1 IS SELECT empno, ename FROM emp WHERE sal>2000;
3     ...
4 BEGIN
5     OPEN c1;
6     ...
7 END;
```

Scorro tramite l'operazione FETCH che ci permette di leggere la riga puntata dal cursore. Ogni volta che faccio la fetch i valori della riga puntata devo metterli in variabili associate ai valori delle celle della riga (5 celle in 5 variabili):

```

1 LOOP
2     FETCH c1 INTO variabili;
3     EXIT WHEN c1%NOTFOUND;
4     -- si processa la tupla corrente
5 END LOOP;
```

É buona chiudere il cursore con CLOSE.

Nell'esempio andiamo a leggere dalla SELECT le due variabili puntate dal cursore, ne leggo finché ce n'è e metto quello che ho letto in un'altra tabella:

```

1 DECLARE
2     my_sal empl.salary%TYPE; --var del tipo della colonna
3     my_ename empl.ename%TYPE;
4     CURSOR c1 IS SELECT ename,salary
5                     FROM empl
6                     WHERE salary > 2000;
7
8 BEGIN
9     OPEN c1;
10    LOOP FETCH c1 INTO my_ename,my_sal;
11    EXIT WHEN c1%NOTFOUND;
12    INSERT INTO RichEmpl values (my_ename,my_sal);
13    DBMS_OUTPUT.PUT_LINE('NAME = ' || my_ename || ', SALARY = ' ||
14                          my_sal);
15    END LOOP;
16 END; /

```

Il FOR automaticamente fa il FETCH ad ogni passaggio.

3.7 Esempio creazione tipo di dato

Creiamo un tipo di dato con la sintassi tipica di Oracle, AS OBJECT, e una procedura che non fa niente se non una stampa a video.

```

1 CREATE TYPE person_typ AS OBJECT
2     ( idno NUMBER,
3       first_name VARCHAR2(20),
4       last_name VARCHAR2(25),
5       email VARCHAR2(25),
6       phone VARCHAR2(20),
7       MAP MEMBER FUNCTION get_idno RETURN NUMBER,
8       MEMBER PROCEDURE display_details (SELF IN OUT NOCOPY
9         person_typ));
10 /
11 CREATE TYPE BODY person_typ AS
12     MAP MEMBER FUNCTION get_idno RETURN NUMBER IS
13     BEGIN
14         RETURN idno;
15     END;
16     MEMBER PROCEDURE display_details ( SELF IN OUT NOCOPY person_typ )
17     IS
18     BEGIN
19         -- use the PUT_LINE procedure of the DBMS_OUTPUT package to
20         -- display details
21         DBMS_OUTPUT.PUT_LINE(TO_CHAR(idno) || ' ' || first_name || ' ' ||
22                               last_name);
23         DBMS_OUTPUT.PUT_LINE(email || ' ' || phone);
24     END;
25 END;
26 /

```

3.8 Tabella tipata

C'è un minimo di differenza rispetto alla sintassi standard. Non possiamo dichiarare una colonna REF. Quindi per creare una tabella tipata è semplice faccio:

```
1 CREATE TABLE tablename OF typename
2   ({<attrname,> NOT NULL[,]}*
3   [,PRIMARY KEY (attrname {,attrname })]);
```

Con tutti i vincoli e le condizioni che non posso dichiarare a livello di tipo ma posso farlo a livello di tabella tipata (NOT NULL, PRIMARY KEY...).

Ad esempio creiamo la tabella tipata sull'oggetto fatto precedentemente aggiungendo i CONSTRAINT che prima non potevo mettere:

```
1 CREATE TABLE person_obj_table OF person_typ
2 (PRIMARY KEY(idno),
3   UNIQUE(first_name,last_name),
4   UNIQUE (email),
5   phone not null);
```

Per quanto riguarda le operazioni di manipolazione come posso procedere? Posso tranquillamente fare un insert (ricordiamo che la tabella tipata ha le righe che sono oggetti del tipo su cui ho creato la tabella tipata).

Per leggere i valori in una tabella tipata devo usare la funzione VALUE(oggetto), nella SELECT.

In questo esempio dichiariamo una procedura e leggiamo una persona con un ID:

```
1 INSERT INTO person_obj_table VALUES
2   ( person_typ(101, 'John', 'Smith', 'jsmith@example.com',
3     '1-650-555-0135'));
4
5 SELECT VALUE(p)
6 FROM person_obj_table p
7 WHERE p.last_name = 'Smith';
8
9 SET SERVEROUTPUT ON
10 DECLARE person person_typ;
11 BEGIN -- PL/SQL block for selecting a person and displaying details
12 SELECT VALUE(p) INTO person
13 FROM person_obj_table p
14 WHERE p.idno = 101;
15     person.display_details();
16 END;
```

In questo esempio abbiamo una tabella relazione in cui abbiamo un attributo che è un oggetto, quando ci metto un dato devo usare il costruttore:

```
1 CREATE TABLE contacts
2 ( contact person_typ,
3   contact_date DATE );
4
5 INSERT INTO contacts VALUES (
6   person_typ (65, 'Verna', 'Mills', 'vmills@example.com',
7     '1-650-555-0125'),
8   to_date('24 Jun 2020', 'dd Mon YYYY'));
```

3.8.1 Accesso

La possibilità di andare a utilizzare diverse strategie per leggere le informazioni, questo è per dire che Oracle usa solo gli ALIAS:

```
1 #1 SELECT idno FROM person_obj_table;           --Correct
2 #2 SELECT contact.idno FROM contacts;           --Illegal
3 #3 SELECT contacts.contact.idno FROM contacts;  --Illegal
4 #4 SELECT p.contact.idno FROM contacts p;       --Correct
```

Chiaramente se non sto leggendo oggetti l'alias non è richiesto.

Questa è la sintassi degli altri comandi per manipolare le tabelle:

```
1 DROP TYPE typename;
2 DROP TABLE tablename;
3 ALTER TYPE typename REPLACE AS OBJECT (new type definition)
4 CREATE OR REPLACE TYPE BODY typename IS method {method};
```

3.9 Tipi riferimento

È simile rispetto a quello dello standard, non cambia molto la sintassi. Se devo creare una colonna all'interno di una relazionale o un attributo in un oggetto non cambia niente. Se questi valori devono essere dei riferimenti a righe di una tabella tipata devo dichiararli come REF. Con la clausola SCOPE IS che ci permette di restringere la lettura ad una certa tabella tipata.

Creo un tipo di dato manager_of_typ, che è un record con due attributi, nulla vieta di avere un attributo che è dello stesso tipo di quello che sto creando:

```
1 CREATE TYPE manager_of_typ AS OBJECT
2 (name VARCHAR2(30),
3  manager REF manager_of_typ );
```

Creo una tabella tipata manager_of costruita sopra questo tipo di dato, la clausola SCOPE la devo andare a specificare quando creo la tabella tipata:

```
1 CREATE TABLE manager_of OF manager_of_typ
2 (manager SCOPE IS manager_of);
```

Creo un INSERT in cui metto un oggetto di tipo NULL come riferimento:

```
1 INSERT INTO manager_of VALUES
2 (manager_of_typ ('John Smith', NULL));
```

Per poter creare un riferimento (dato che ho una SELECT con una riga sola), prendo la persona che ho appena inserito e metto un nuovo impiegato che ha come riferimento la persona che ho messo in precedenza. Se dovessi eliminare la prima persona inserita, il secondo riferimento sarebbe penzolante (va gestito):

```
1 INSERT INTO manager_of
2   SELECT manager_of_typ ('Bob Jones', REF(e))
3   FROM manager_of e
4  WHERE e.name = 'John Smith';
```

Per poter leggere degli oggetti puntati dovrei usare le funzioni REF per riferirsi ad un oggetto, Deref per andare a prendere l'oggetto riferito e la funzione VALUE per leggere il contenuto dell'oggetto.

Lo vediamo tramite un altro esempio: Creo il tipo t_movie, che ha vari attributi, la cosa in più è il remake_of che è un riferimento ad un altro oggetto t_movie:

```
1 CREATE TYPE t_movie AS OBJECT
2   (title      VARCHAR2(30),
3    director   VARCHAR2(20),
4    year       DECIMAL(4),
5    kind       CHAR(15),
6    val        NUMERIC(3,2),
7    remake_of  REF t_movie);
```

Poi faccio la tabella tipata movie che ha come scope la tabella stessa:

```
1 CREATE TABLE movie OF t_movie
2   (remake_of SCOPE IS movie,
3    PRIMARY KEY (title,director));
```

Ci facciamo un po' di INSERT per popolarla, senza riferimenti:

```
1 INSERT INTO movie
2   VALUES(t_movie('underground', 'emir
3     kusturica',1980,'drama',4,NULL));
4
5 INSERT INTO movie
6   VALUES(t_movie('big fish', 'tim burto',1990,'drama',2,NULL));
7
8 INSERT INTO movie
9   VALUES(t_movie('frankenstein', 'J. Searle
10     Dawley',1910,'horror',1,NULL));
11
12 INSERT INTO movie
13   VALUES(t_movie('frankenstein', 'James
14     Whales',1931,'horror',3,NULL));
15
16 INSERT INTO movie
17   VALUES(t_movie('frankenstein', 'Howard W.
18     Kock',1970,'horror',1,NULL));
```

Poi andiamo a creare dei riferimenti, sono articolati perchè non ho accesso diretto all'identificatore, voglio dire che il film Frankenstein ha un rifacimento, quindi inserisco un nuovo film e gli sto dicendo di andare a dire che è il rifacimento di un altro film che avevo già inserito. Il film di Whale è un rifacimento del film di Dawley. Faccio un UPDATE sulla tabella tipata, dell'oggetto nella clausola WHERE più esterna, prendo di questa riga la colonna remake_of e dato che è un puntatore devo andare a cercare cosa punta, per prendere il puntatore devo usare la funzione ref, che prende un oggetto e di questo oggetto ne prende il puntatore, l'oggetto che prendo è nella stessa tabella (con alias diverso). Quindi nella SELECT interna pesco un oggetto, di questo oggetto (m1) ne ho bisogno il puntatore, quindi uso la funzione ref. Che poi andrò ad utilizzare per fare l'update:

```

1 UPDATE movie m
2 SET m.remake_of = (SELECT ref(m1) FROM movie m1
3                     WHERE m1.title = 'frankenstein' AND
4                           m1.director = 'J. Searle Dawley')
5 WHERE m.title = 'frankenstein' AND m.director = 'James Whales';

```

Il film 'frankenstein' di 'James Whales' è un remake del film 'frankenstein' di 'J. Searle Dawley'

```

1 UPDATE movie m
2 SET m.remake_of = (SELECT ref(f1) FROM movie f1
3                     WHERE f1.title = 'frankenstein'
4                           AND f1.director = 'James Whales')
5 WHERE m.title = 'frankenstein' AND
6       m.director = 'Howard W. Kock';

```

Il film 'Frankenstein' di 'Howard W. Kock' è un remake del film 'frankenstein' di 'James Whales.

Per leggere i dati di una tabella tipata nelle SELECT devo usare la funzione value, a quel punto posso usare la dot notation per leggere gli attributi dell'oggetto:

```

1 SELECT value(f) FROM movie f

```

Invece se dovessi andare a leggere un riferimento, non potrei leggerlo con la dot notation perchè non ho l'accesso. Il modo di leggerlo è usare la funzione di Deref (da un puntatore recupero l'oggetto puntato). Dato un film voglio sapere se è un remake di un altro film:

```

1 SELECT deref(m.remake_of)
2 FROM movie m
3 WHERE m.title = 'frankenstein' AND
4       m.director = 'James Whales';

```

3.10 Collezione

La cosa particolare è l'uso della Nested table invece del MULTISSET, rende trasparente la logica di tabella figlia che viene associata al concetto di collezione.

La sintassi:

```

1 CREATE TYPE [name] AS TABLE/VARRAY OF ...

```

3.10.1 VARRAY

È meno utilizzato, ha un numero fisso di elementi che crea una collezione. Ha due vantaggi:

- Dimensione fissa
- Per leggerla devo creare io una procedura (funzione)

Vediamo degli esempi di questa struttura. È un po articolato perchè non posso in unica istanza creare una collezione di oggetti, devo creare un oggetto, creare il tipo collezione e poi usarlo in una tabella. Ovviamente se dovesse essere un VARRAY di tipi built in non ci sarebbe la creazione del tipo.

```
1 CREATE TYPE project_t AS OBJECT(  
2   id      INTEGER,  
3   title   VARCHAR2(25),  
4   cost    NUMBER(7,2));  
5  
6 CREATE TYPE projects_list AS VARRAY(50) OF project_t;  
7  
8 CREATE TYPE department_t AS OBJECT(  
9   id      INTEGER,  
10  name    VARCHAR2(15),  
11  budget  NUMBER(11,2),  
12  projects projects_list);  
13  
14 CREATE TABLE department OF department_t;
```

Inserisco dei valori, ho una collezione di progetti che vado ad istanziare chiamando il costruttore del VARRAY, all'interno devo creare tanti oggetti che sono i singoli progetti:

```
1 INSERT INTO department  
2   VALUES (department_t(30, 'R-D', 1000,  
3     projects_list(project_t(1, 'DBMS', 1000),  
4       project_t(3, 'C++', 2000))));  
5  
6 INSERT INTO department  
7   VALUES (32, 'Marketing', 1000,  
8     projects_list(project_t(1, 'Nuova Pubblicità', 1000),  
9       project_t(3, 'Incentivi Personale', 2000)));  
10  
11 INSERT INTO department  
12   VALUES (34, 'Marketing', 1000, NULL);  
13  
14 INSERT INTO department  
15   VALUES (38, 'Marketing', 1000, Projects_list());
```

3.10.2 Nested Table

È il corrispettivo del MULTISSET. Dichiaro il tipo di oggetto, poi creo il tipo collezione che è una nested table con la sintassi AS TABLE, non devo definire le dimensioni perchè non hanno una dimensione fissa.

```
1 CREATE TYPE course_t AS OBJECT(  
2   id      NUMBER(4),  
3   name    VARCHAR2(25),  
4   cfu     NUMBER(1));  
5  
6 CREATE TYPE courses_list AS TABLE OF course_t;
```

Abbiamo creato il tipo di dato nested table, a questo punto creo un tipo di dato faculty_t, quando creo una tabella che ha all'interno una nested table (sia relazionale sia

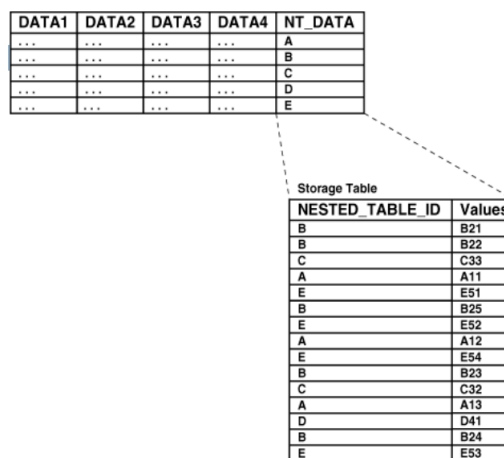
tipata), devo per forza dirgli che dentro la tabella c'è un attributo (colonna) che è una nested table (nel nostro caso courses) e gli devo dire come si chiamerà la tabella fisica su cui questa nested table si appoggia, genera questa tabella in cui mette i dati della collezione:

```

1 CREATE TYPE faculty_t AS OBJECT(
2     name          VARCHAR2(20),
3     director      VARCHAR2(20),
4     courses       courses_list );
5
6 CREATE TABLE faculty OF faculty_t
7 NESTED TABLE courses STORE AS courses_tab;

```

Sostanzialmente ho una child table in cui metto i dati della nested table.



L'INSERT è simile a quello che facevamo con il VARRAY, inserisco tanti oggetti chiamando il loro costruttore:

```

1 INSERT INTO faculty
2     VALUES('Informatica','Castano',
3         courses_list(course_t(1000,'Programmazione I',2),
4             course_t(1001,'Logica Matematica',1),
5             course_t(1002,'Basi di Dati',2),
6             course_t(1003,'Grafica',1)));

```

Per leggere i dati devo usare la funzione TABLE, per poter leggere la child table, non faccio altro che prendere faculty, dirgli che dentro c'è una nested table, la trasformo in tabella virtuale e poi usando la funzione value con la dot notation posso accedere ai dati.

```

1 SELECT value(t)
2 FROM faculty d, TABLE(d.courses) t;

```

La funzione TABLE la uso tutto le volte che devo trasformare la collezione in qualcosa che devo leggere nella proiezione. Potrei anche chiedere di avere tutti i corsi della facoltà di informatica, devo quindi usare la funzione TABLE:

```

1 SELECT *
2 FROM TABLE(SELECT courses FROM faculty
3             WHERE name = 'Informatica');

```

Un po' più articolate sono le manipolazioni delle nested table. Vediamo un esempio in cui voglio fare un UPDATE degli oggetti di una collezione. Dobbiamo usare la funzione TABLE:

```
1 UPDATE
2     TABLE(SELECT courses FROM faculty
3             WHERE name = 'Informatica')
4     SET cfu = cfu + 1
5     WHERE id IN (1000,1002);
```

Così come con la DELETE, prendo tutti i corsi di informatica e elimino tutti quelli con CFU = 1, questo posso farlo perchè ho usato la funzione TABLE:

```
1 DELETE FROM TABLE(SELECT courses FROM faculty WHERE name='Informatica')
2 p
3 WHERE p.cfu = 1;
```

Se devo leggere gli elementi di un VARRAY devo creare per forza una procedura, mettiamo di voler leggere tutti gli oggetti del dipartimento, devo creare una variabile, vado a prendere il progetto dal dipartimento 30, lo metto in myProject ed essendo una collezione devo fare qualche loop per leggere i progetti prelevati dal VARRAY:

```
1 DECLARE my_projects projects_list;
2
3 SELECT project INTO my_projects
4 FROM department
5 WHERE id = 30;
6
7 IF my_projects(i).title = 'DBMS' ...
```

3.11 Ereditarietà

Può esserci tra tipi e tabelle.

Vediamo come viene creata partendo dal tipo di dato cliente poi la creazione del tipo cliente vip e standard che hanno delle particolarità:

```
1 CREATE TYPE t_costumer AS OBJECT
2     (cod DECIMAL(4),
3     name VARCHAR2(20),
4     phone CHAR(15),
5     location
6     VARCHAR2(20))
7 NOT FINAL;
8
9 CREATE TYPE t_standard UNDER t_costumer (missingPoints INTEGER)
10 NOT FINAL;
11
12 CREATE TYPE t_VIP UNDER t_costumer (bonus INTEGER)
13 NOT FINAL;
```

Metto i NOT FINAL perchè voglio poter creare l'ereditarietà.

Mettiamo di creare una tabella che è fatta sul tipo padre:

```

1 CREATE TABLE Rental(
2     video DECIMAL(4) PRIMARY KEY,
3     costumer t_costumer);
4
5 INSERT INTO Rental VALUES
6     (1101,t_costumer(6601, 'Mario','021010','Milano'));
7
8 INSERT INTO Rental VALUES
9     (1110,t_standard(6801, 'Aldo','062020','Roma',30));
10
11 INSERT INTO Rental VALUES
12     (1125,t_vip(7020, 'Elena','0103030','Genova',5));

```

Non è una tabella tipata, se volessi farlo:

```

1 CREATE TABLE costumer OF t_costumer;
2
3 INSERT INTO costumer VALUES
4     (t_costumer(6601,'Mario Rossi','02101010','Milano'));
5
6 INSERT INTO costumer VALUES
7     (t_standard(6801,'Aldo Gialli','06202020','Roma',30));
8
9 INSERT INTO costumer VALUES
10     (t_vip(7020,'Elena Rossi','010303030','Genova',5));

```

Se creo una tabella tipata così posso inserire all'interno di questa tabella i figli, se voglio fare dei limiti dovrei dirgli che non posso inserire in questa colonna niente che non sia il padre, con la clausola NOT SUBSTITUTABLE AT ALL LEVELS:

```

1 CREATE TABLE Rental(
2     video DECIMAL(4) PRIMARY KEY,
3     costumer t_costumer)
4 COLUMN costumer NOT SUBSTITUTABLE AT ALL LEVELS;

```

La stessa cosa la faccio con la creazione della tabella tipata:

```

1 CREATE TABLE costumer OF t_costumer NOT SUBSTITUTABLE AT ALL
2 LEVELS;

```

Potrei anche avere delle clausole particolare, mettici tipo padre e tipo t_standard, specifico solo un figlio:

```

1 CREATE TABLE rental(
2     video DECIMAL(4) PRIMARY KEY,
3     costumer t_costumer)
4 COLUMN costumer IS OF (ONLY t_standard);

```

Potrei dirgli dammi tutti i noleggi il cui customer sia di tipo standard o vip, oppure solo del padre:

```

1 SELECT *
2 FROM rental r
3 WHERE r.costumer IS OF (t_standard,t_VIP)
4
5 SELECT *
6 FROM Customer c

```

```
7 WHERE VALUE(c) IS OF (t_standard,t_VIP)
```

3.12 Vista

È simile allo standard. Lo vediamo tramite degli esempi, creo una tabella emp_table, un tipo di dato che è employee_t. Poi voglio creare una vista emp_view1 su una serie di oggetti, voglio solo la vista che vada a vedere la proiezione di una SELECT, prendo dei dati di una tabella tale per cui il job sia developer.

La vista diventa una sorta di vista tipata, perchè va a leggere degli oggetti, che crea andando ad utilizzare dei valori che sono presenti in una tabella relazionale classica. Prendo le risposte della query su una tabella relazionale, la risposta (righe) vengono usate per istanziare degli oggetti di tipo che ho dichiarato, su cui creerò una vista tipata (è una collezione di oggetti). Una volta che ho creato la vista inserisco dei valori nella tabella e vedo che questo si riflette sulla vista.

```
1 CREATE TABLE emp_table
2 ( empnum NUMBER (5),
3   ename VARCHAR2 (20),
4   salary NUMBER (9,2),
5   job VARCHAR2 (20));
6
7 CREATE TYPE employee_t AS OBJECT
8 ( empno NUMBER (5),
9   ename VARCHAR2 (20),
10  salary NUMBER (9,2),
11  job VARCHAR2 (20));
12 /
13
14 CREATE VIEW emp_view1 OF employee_t WITH OBJECT IDENTIFIER (empno) AS
15   SELECT e.empnum, e.ename, e.salary, e.job
16   FROM emp_table e
17   WHERE job = 'Developer';
18
19 insert into emp_table values(1,'John',1000.00,'Architect');
20 insert into emp_table values(2,'Robert',900.00,'Developer');
21 insert into emp_table values(3,'James',2000.00,'Director');
22
23 select * from emp_view1;
```

Ne vediamo un altro esempio, creo la tabella dept, un tipo address, un tipo dept (al cui interno uso il tipo address), a questo punto creo una vista che è costruita sul tipo di dato dept, è costruita andando a leggere dalla tabella relazionale tutte le righe di questa tabella e metto questi valori nel tipo di dato. Poi faccio l'INSERT e la SELECT per vedere che funziona:

```
1 CREATE TABLE dept
2 ( d_no NUMBER PRIMARY KEY,
3   d_name VARCHAR2(20),
4   d_street VARCHAR2(20),
5   d_city VARCHAR2(10),
6   d_state CHAR(2),
7   d_zip VARCHAR2(10));
```

```

8
9 CREATE TYPE address_t AS OBJECT
10 ( street VARCHAR2(20),
11   city VARCHAR2(10),
12   state CHAR(2),
13   zip VARCHAR2(10));
14 /
15
16 CREATE TYPE dept_t AS OBJECT
17 ( deptno NUMBER,
18   deptname VARCHAR2(20),
19   address address_t );
20 /
21 CREATE VIEW dept_view OF dept_t WITH OBJECT IDENTIFIER (deptno) AS
22   SELECT d.d_no, d.d_name,
23          address_t(d.d_street,d.d_city,d.d_state,d.d_zip) AS deptaddr
24 FROM dept d;
25
26 insert into dept values(1,'Sales','500 Oracle pkwy','Redwood
27   S','CA','94065');
28
29 insert into dept values(2,'ST','400 Oracle Pkwy','Redwood
30   S','CA','94065');
31
32 insert into dept values(3,'Apps','300 Oracle pkwy','Redwood
33   S','CA','94065');
34
35 select * from dept_view;

```

3.13 Esempio

Facciamo un esempio di modellazione completa di uno scenario.

```

CINEMA(CodC, Nome, Indirizzo, Tel, NSale)
SALE(CodS, Nome, Nposti, CodCCINEMA)
FILM(CodF, Titolo, CodRegistaPERSONE, CodProtagonistaPERSONE, Durata, Video)
PERSONE(CodA, Nome, Cognome)
ATTORI(CodAPERSONE, CodFFILM)
PROIEZIONI(CodSSALE, OraInizio, CodFFILM)

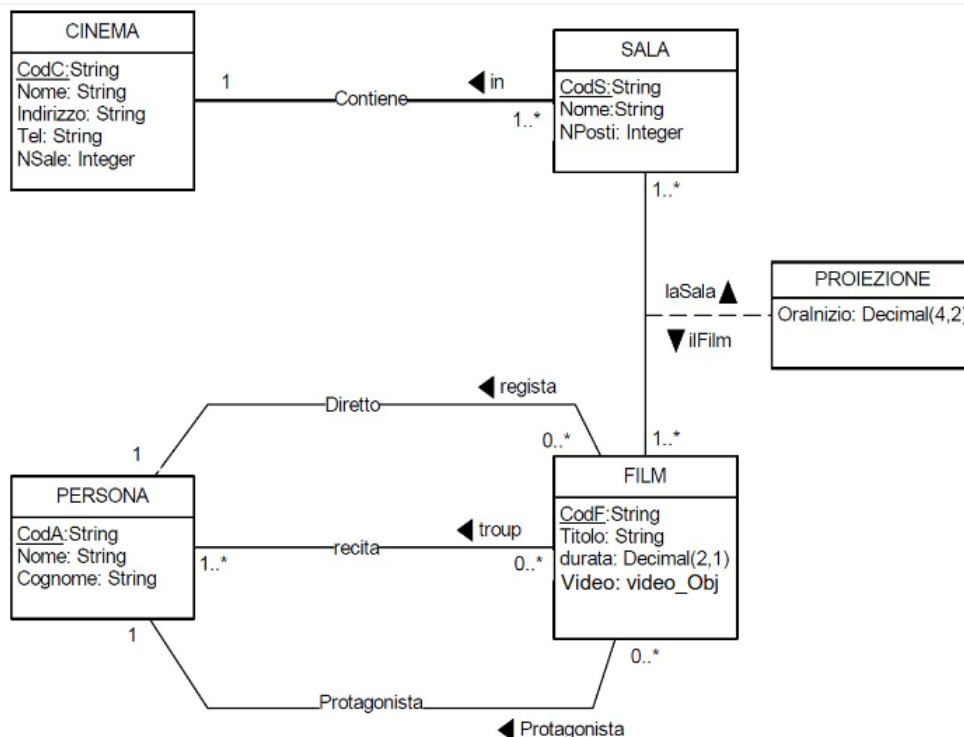
```

Partiamo da un modello relazionale con delle relazioni che rappresentano le tabelle. Queste tabelle essendo in un modello relazionale hanno chiavi primarie e chiavi esterne.

La richiesta nel punto 1 è di implementare uno schema relazionale ad oggetti che permetta di ottimizzare il carico di lavoro, l'ultima parte è perchè dobbiamo decidere il verso di lettura dei riferimenti. Dobbiamo capire quali sono le query che in questo dominio applicativo verranno maggiormente utilizzare e usare il verso che mi ottimizzi le query.

Poi ci sono una serie di query.

Ha ricreato il modello ER andando ad evidenziare il senso di lettura, in base a quello che ha letto nelle query.



Partiamo dalla prima tabella, CINEMA, dato che è puntata potrei decidere che è una tabella tipata (creare un tipo oggetto CINEMA e poi crearci una tabella sopra), in realtà questo è un caso in cui l'interrogazione è molto semplice, tanto che potrei fare una tabella relazionale. Noi la teniamo relazionale.

La tabella SALA dato che è riferita (avrò un riferimento a SALA), quindi creo un tipo tSALA, dopo di che devo pensare se mi basta l'oggetto o mi serve il riferimento. Per evitare ridondanza, se un film è proiettato in una sola sala mi basta avere l'oggetto sala nella tabella PROIEZIONE, altrimenti è meglio avere il riferimento (quindi creare una tabella tipata SALA) in modo da non replicare l'informazione. In questo caso posso fare la stessa proiezione in diverse sale, quindi facciamo l'ipotesi che la tabella SALA sia una tabella tipata (creo il tipo e la tabella).

Anche per la PROIEZIONE potrei sia modellarla come tabella relazionale, perchè non è mai puntata, o implementarla come una tabella tipata. È la stessa cosa. La cosa importante è che dentro avrò due puntatori, uno alla sala ed uno al film.

La PERSONA è puntata 3 volte, quindi sicuramente tabella tipata, quindi tipo persona e tabella tipata. perchè la stessa persona può essere attore in diversi film.

FILM è puntato quindi anche lui tabella tipata, le caratteristiche è che avrà un riferimento a regista, un riferimento a protagonista che sono unici. Inoltre avrà una collezione di attori, non creo più una tabella fittizia, questa collezione rappresenterà diversi riferimenti alla tabella persona. In Oracle avrà una nested table.

Il primo errore nella modellazione è identificare dei versi delle query non ottimali, questo non crea effettivamente uno schema sbagliato, ma ottimizzato male. Il secondo errore è quello di fare delle scelte sbagliate nelle identificazione delle tabelle tipate. Se

ho deciso un verso di lettura, la tabella riferita deve essere necessariamente una tabella tipata. Inoltre devo capire i riferimenti per evitare la ridondanza, l'oggetto lo uso una volta sola? mi basta il tipo. Lo uso più volte? mi serve la tabella tipata.

Siamo a cavallo, creiamo il modello:

```
1 create table cinema(
2     CodC varchar(5) PRIMARY KEY,
3     Nome varchar(10) NOT NULL,
4     Indirizzo varchar(20),
5     Tel varchar(10),
6     NSale integer
7 );
8
9 create type Tsala as OBJECT (
10     CodS varchar(5),
11     Nome varchar(10),
12     Nposti integer,
13     inCinema varchar(5)
14 );
15
16 create table sala of tsala (
17     PRIMARY KEY(CodS),
18     CONSTRAINT cinema_fk FOREIGN KEY (inCinema) REFERENCES cinema(CodC)
19 );
20
21 create type Tpersona AS OBJECT(
22     CodP varchar(5),
23     Nome varchar(10),
24     Cognome varchar(10)
25 );
26
27 create table persona of tpersona (PRIMARY KEY(CodP));
28
29 #Nested Table per collezione
30
31 CREATE TYPE Tattori as TABLE OF REF Tpersona;
32
33 create type Tfilm AS OBJECT(
34     CodF varchar(5),
35     Titolo varchar(10),
36     Durata integer,
37     troup Tattori,
38     protagonista REF TPersona,
39     regista REF TPersona,
40     video BLOB
41 );
42
43 #Devo dirgli esplicitamente il nome della nested table, e quella in cui
    metto la collezione
44 create table Film of Tfilm(PRIMARY KEY(CodF)
45 ) NESTED TABLE troup STORE AS troup_tab;
46
47 create type Tproiezioni AS OBJECT (
48     inSala REF Tsala,
49     ilFilm REF Tfilm,
50     oraInizio integer
```

```

51 );
52
53 create table Proiezioni of Tproiezioni;

```

Ora vediamo le interrogazioni SQL:

Trovare il nome delle sale del cinema Augustus:

```

1 SELECT s.Nome
2 FROM Sala s, Cinema c
3 WHERE s.inCinema=c.CodC AND c.Nome = 'Augustus';

```

Questo deriva dalla scelta effettuata in fase di passaggio dal modello concettuale a quello logico, perchè abbiamo una tabella relazionale.

Determinare le proiezioni (ore inizio e titolo film) delle sale con almeno 100 posti:

```

1 SELECT p.OraInizio, p.ilFilm.Titolo
2 FROM Proiezioni p
3 WHERE p.inSala.Nposti >= 100;

```

Questa SELECT sarebbe stata lunghissima se avessimo usato il modello relazionale e non quello ad oggetti. Tramite la navigazione per riferimento posso recuperare l'attributo Nposti che è dentro l'oggetto inSala nella nostra tabella tipata.

Vogliamo tutti gli attori del film troy:

```

1 SELECT VALUE(t.cognome)
2 FROM film f, TABLE(f.toup) t
3 WHERE f.titolo ='troy';

```

Vediamo un'altra query che ci obbliga a prendere dei dati da una collezione, determinare il titolo dei film il cui attore è Brad Pitt, questa è una query che va nella direzione opposta a quella che abbiamo stabilito, quindi è poco ottimizzata:

```

1 SELECT DISTINCT f.Titolo
2 FROM Film f, TABLE(f.Troup) t
3 WHERE VALUE(t).Cognome = 'Pitt' and value(t).Nome = 'Brad';

```

come prima devo usare la funzione TABLE perchè ho una collezione, la differenza è che nella WHERE devo usare la funzione VALUE perchè devo trovare gli oggetti riferiti. Mi obbliga a pescare tutte le collezioni e cercare in tutte le collezioni quelle in cui compare un certo attore. Le query non ottimizzate sono quelle in cui c'è la lettura di una child table nel filtro (WHERE).

Determinare il titolo dei film il cui protagonista NON è Brad Pitt:

```

1 SELECT f.Titolo
2 FROM Film f
3 WHERE f.protagonista.Cognome != 'Pitt' and f.protagonista.Nome != 'Brad'
4 and EXISTS (select * from TABLE(f.Troup) p where
5 value(p).Cognome='Pitt' AND value(p).Nome='Brad');

```

Nella WHERE devo dire che il protagonista non è Pitt come nome e Brad come nome, EXIST ci dice che deve essere un film in cui partecipa Brad Pitt, devo prendere tutti i film in cui partecipa, leggo tutte le collezioni e vedo quelle in cui c'è. Quindi nella SELECT interna faccio una lettura sulle collezioni. Di nuovo non è ottimizzata, perchè la lettura della child table è fatta nella clausola WHERE.

Per ogni film restituire il suo titolo e il numero di attori che hanno lavorato in quel film:

```
1 SELECT f.Titolo, COUNT(*)
2 FROM Film f, TABLE(f.troup)
3 GROUP BY f.Titolo;
```

Oppure sfruttando la logica object relational:

```
1 SELECT f.Titolo, CARDINALITY(f.troup)
2 FROM Film f
```

Restituire tutti i registri dei film che superano le due ore:

```
1 SELECT deref(f.regista)
2 FROM Film f
3 WHERE f.durata > 2;
```

Devo usare la funzione Deref, che dato il puntatore mi recupera l'oggetto.

Le domande d'esame potrebbero essere: se ho questo modello la query è ottimizzata? Dato questo modello relazionale, questa tabella come andrebbe implementata? (nel nostro esempio la tabella FILM come andrebbe implementata). Quale sarebbe la SELECT più performante? Come modello la collezione? Quale tipo di SELECT faccio per ottimizzare la lettura?

Chapter 4

Text Retrieval

Perchè ci serve una struttura più complessa per i testi rispetto alle stringhe? devo trovare una strategia per interrogare un dato che non è strutturato. I dati strutturati sono ad esempio il Booleano, la Stringa, ma anche un Ciclista modellato nell'object relational è un dato strutturato su cui posso creare le interrogazioni.

Tutta l'informazione strutturata è deterministica, se faccio diverse interrogazioni sugli stessi dati mi danno sempre lo stesso risultato. Non c'è neanche il concetto di probabilità.

Quello che interessa andare ad indagare è quello di avere un processo di information retrieval non così deterministico. I testi fanno parte di un concetto di big data, ma non in senso di tanti dati, hanno un volume sul singolo oggetto corposo (ad esempio un testo di 1GB). Il cui contenuto non è strutturato, addirittura c'è una semantica che devo catturare da questo dato, la semantica è intuitiva quando parliamo di immagini perchè è cosa rappresenta l'immagine. In un testo la semantica è di cosa tratta questo documento.

Quindi devo andare ad estrarre informazioni non strutturate da un oggetto voluminoso di cui devo capirne la semantica per poi andare a rispondere a delle query che fa l'utente.

A livello architetturale ho sempre il concetto di:

- formulazione della richiesta
- un recupero di dati rilevanti per la richiesta dell'utente
- l'analisi dei dati da parte dell'utente
- eventualmente la riformulazione della richiesta da parte dell'utente

Per poter rispondere ad una query in linguaggio naturale dell'utente devo prima avere memorizzato i documenti e modellato in modo da poter rispondere a questo tipo di interrogazioni non strutturate. Ovviamente l'obiettivo è ottenere una risposta precisa, che abbia una recall, che all'utente soddisfi.

Quindi data una singola interrogazione ottengo più risposte, che potrebbero anche cambiare nel tempo, ogni risposta ha il concetto di rilevanza e grazie alla rilevanza ho il concetto di ordinamento delle risposte.

L'enfasi iniziale dev'essere come l'utente formula la sua query, da quella vincolata del sistema booleano a quella libera usando linguaggio naturale.

Problemi da risolvere:

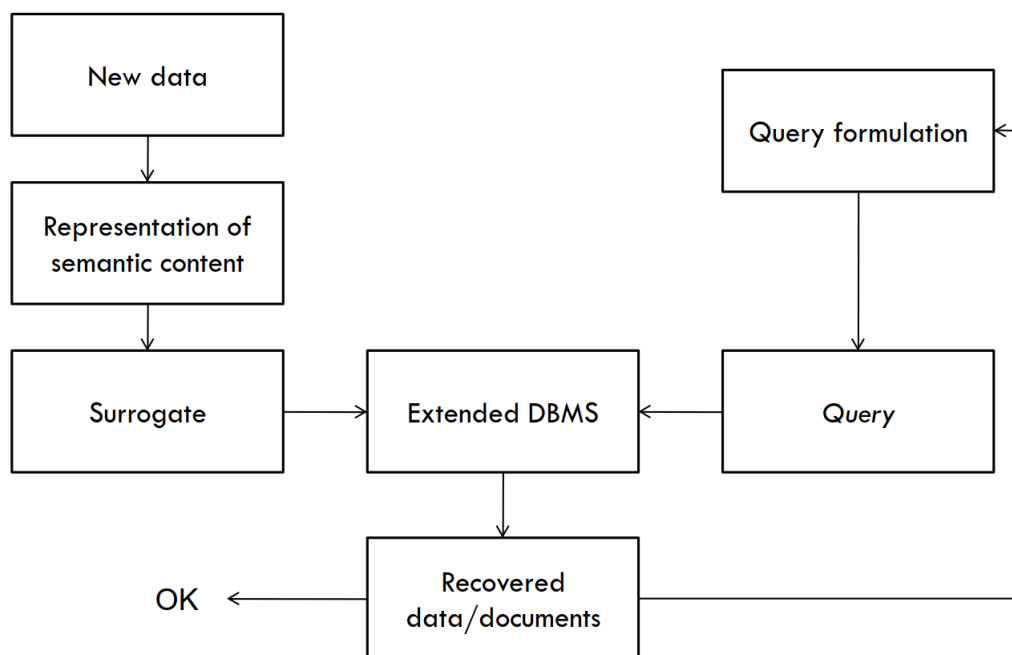
- Strategia rappresentazione dato
- Operatori che permettano di unire più richieste
- Strategie che possono andare ad estendere l'interrogazione. Estendere una query perchè potrei avere documenti in cui non c'è il termine usato dall'utente ma potrebbero esserci sinonimi, quindi bisogna esplodere la query

Quindi il ciclo di interazione è:

- Interrogazione
- Retrieval
- Browsing da parte dell'utente

Recap:

	IRS	DBMS
Data	Unstructured	Structured
Queries	Free	With schema
Browsing	Few	No
Collection	Big	Big
Recovery	Probabilistic	Deterministic
Interactivity	Fair	Low



4.1 Creazione di un archivio di documenti testuali

Sono strategie che faccio offline, prima di creare un surrogato per il testo devo prima lavorarlo.

C'è una fase di comprensione della strutturazione del testo e una fase di indicizzazione del testo.

- Location: inserire i documenti nella collezione

- Format decoding: riduzione dei documenti in stringhe
- Indexing: rappresentazione dell'informazione contenuta nei documenti
- Generazione di una struttura dati adeguata

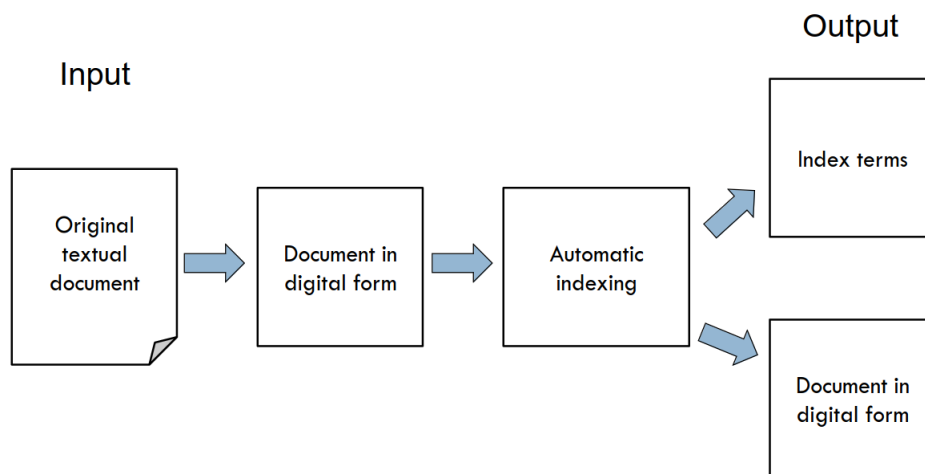
4.1.1 Rappresentare semanticamente un documento

Capiamo come catturare quello di cui parla il documento, per poter capire di cosa parla devo estrarre da questo documento dei termini, devo capire quali sono i termini più importanti all'interno, per poterne dare una rappresentazione sintetica. Un documento è un insieme di termini, ogni termine ha delle radici, ogni termine forma delle frasi, queste parole possono appartenere ad un dizionario controllato (se sto creando l'archivio in ambito giuridico avrò dei termini particolari che sono specifici per il dominio) e possono contenere dei metadati e appartenere ad una rete semantica.

Un indice è la rappresentazione dei termini significativi che vengono usati nel documento.

4.1.2 Indicizzazione

Il goal è creare un surrogato del documento tramite i suoi descrittori.



Deve avvenire in maniera automatica:

- Full text: prende i termini del documento
- Tematica: usa i temi per indicizzare il documento

La rappresentazione del documento è un set di parole che rappresentano il concetto del documento. Quando devo definire i termini caratterizzanti del documento parto dall'analisi lessicale, parola per parola vedo com'è composto, non tutte le parole hanno la stessa rilevanza per capire il significato del documento, le congiunzioni non sono parole discriminanti all'interno del documento. Sono parole dette stop word che non ha senso considerare, perchè non sono rappresentative per il documento (non mi dicono niente su

quello che vuole dire il documento). A volte non mi serve neanche tutta la parola, mi basta la radice per usarla come rappresentante della semantica di un documento.

Una volta che ho trovato le parole significative creo una struttura ad indice ed eventualmente la peso.

Fasi:

- Analisi lessicale
- Rimozione delle stop-word
- Estrazione degli stems (stemming)
- Creazione degli indici
- Pesì

4.1.3 Analisi lessicale

Prendo tutte le parole, dette token, che sono le parole che caratterizzano un documento. Attenzione che non basta prendere tutte le parole perchè alcune sono combinazioni, ad esempio aren't è una combinazione di parole, inoltre dobbiamo essere tolleranti rispetto ad errori scritti nel documento, o di maiuscole.

Possono esserci parole che sono sequenze, perchè sequenze di parole hanno un concetto solo, ad esempio un indirizzo mail, un URL, un IP address.

Quindi estraiamo tutti i token distintivi presenti all'interno del documento.

Durante questa fase di tokenizzazione devo fare strategie:

- di conversione (maiuscolo/minuscolo)
- di normalizzazione (date in un unico formato)
- normalizzazione anche di nomi

4.1.4 Rimozione delle stop word

Una volta che ho trovato tutti i token normalizzati devo chiedermi se tutti sono rilevanti. Devo andarne a vedere la frequenza, se è un termine troppo frequente all'interno dello stesso documento avrà un'incidenza sul peso (rilevante). All'interno di documenti diversi significa che questo termine non è discriminante. Ci sono circa 250 parole che sono troppo comuni e ci sono sempre, dobbiamo toglierle con la fase di rimozione delle stop word.

L'eliminazione delle stop word riduce il numero di token del 30/40%. Attenzione che questo ha senso se abbiamo testi voluminosi, si eliminano se il contesto dell'applicazione lo permette. Tant'è che i motori di ricerca non eliminano a priori le stop word.

4.1.5 Stemming

Diciamo che è inutile considerare il token così com'è, basta considerare la sua radice, ad esempio nei verbi non interessa la forma verbale, basta l'infinito.

Si assimilano le parole con la stessa radice ad una keyword. Ad esempio:

- computer, computational, computing e computed → compute
- paint painting painted painter → paint
- colour colours → color
- different difference → differ
- Computers are painted different colours → comput ar paint differ color

Porter's algorithm

C'è un algoritmo molto famoso per fare Stemming, ci sono tutte le regole ma sono molto meccaniche. Se trovi i suffissi eliminiali e così via.

Pro:

- Ne incrementano l'efficacia. Perché unendo varie parole insieme riusciamo poi ad espandere la query, perché nonostante chiediamo una parola ci restituisce tutti i documenti che hanno parole che collassano sulla stessa della query.

Contro:

- A volte riduco troppo rischiando di perdere completamente la semantica (fabricate and fabric)
- Alcune riduzioni (stems) non sono parole: saw → s

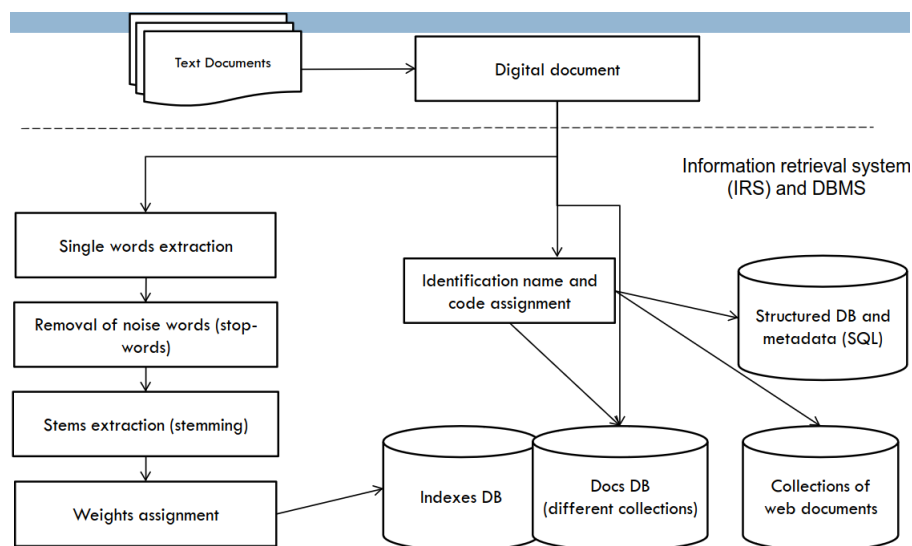
4.1.6 Indice inverso

L'Indicizzazione è associata ai termini, prendo tutti i termini che ho trovato in tutti i documenti che ho e vado a dire ogni termine a che documento appartiene.

shipment	D1		D3
gold	D1		D3
damag	D1		
fir	D1		
deliveri		D2	
silver		D2 - D2	
arriv		D2	D3
truck		D2	D3

Con eventualmente dei pesi, che per il momento è la frequenza dei termini:
Questo ci porta ad una prima rappresentazione completa del text retrieval.

Stems \ Docs	D1	D2	D3
arriv	0	1	1
damag	1	0	0
deliveri	0	1	0
fir	1	0	0
gold	1	0	1
shipment	1	0	1
silver	0	2	0
truck	0	1	1



4.2 Recovery

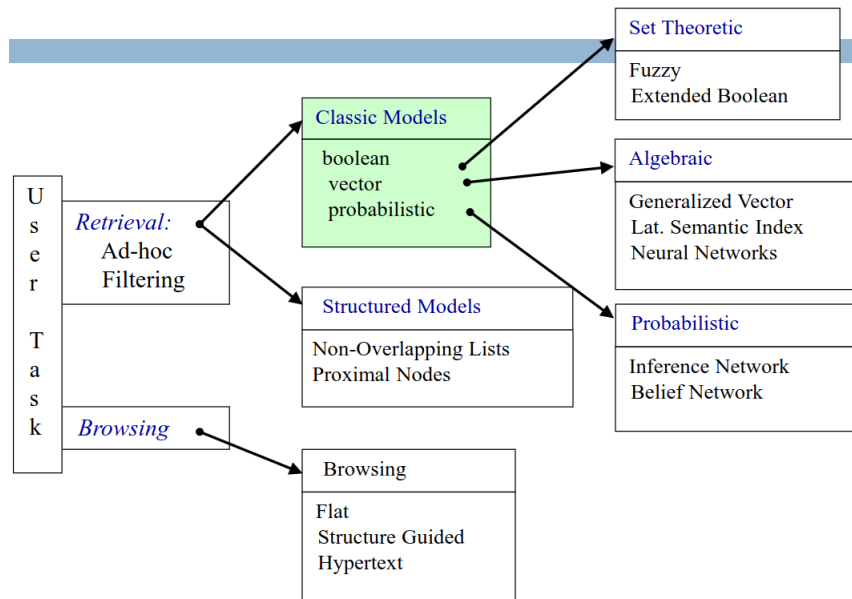
Un' altro concetto rappresenta il concetto di retrieval, una volta che ho preparato la tabella di indici inversi sono interessato a recuperare i documenti che rispondono esattamente alla query oppure rispondono al meglio.

Avrò una rappresentazione dell'interrogazione, anche la frase che l'utente dice sarà elaborata per rappresentarla da un punto di vista della sua semantica. Quindi deve passare in tutte le fasi di normalizzazione che abbiamo visto. Perché anche l'interrogazione dell'utente deve avere un suo surrogato. Nel caso nella query ci siano più richieste avrò degli operatori booleani che mi restituiscono cose.

4.3 Modelling

Una volta che ho fatto passare le query e i documenti in queste fasi devo definire un modello di rappresentazione che possa essere implementato e che ci permetta di capire se un documento fa parte di una query o no.

Nel caso di modello booleano ci darà un exact match nel caso di modello vettoriale ci dà un insieme di documenti da ordinare in base a quanto sono pertinenti alla domanda dell'utente.



4.4 Classici

Tutti i modelli classici hanno questi concetti basilari:

- Ciascun documento deve essere rappresentato da un insieme di termini indici
- L'indice è una parola che rappresenta il documento
- Posso prendere tutti i termini o parte dei termini di un documento
- Non tutti i termini hanno lo stesso rilievo

Da un punto di vista formale tutti i modelli classici si definiscono così

- t_i è l'indice i-esimo
- d_j è il documento j-esimo
- w_{ij} è il peso associato a t_i nel documento d_j

La w rappresenta quanto è importante il termine indice per rappresentare la semantica del documento.

In tutti i modelli classici possiamo rappresentare il documento come un vettore, pesato le cui componenti sono tutti i pesi dal termine 1 a n :

$$\text{vec}(d_j) = (w_1, w_2, \dots, w_{n_j})$$

C'è anche la funzione che permette di recuperare il peso del termine :

$$g_i(\text{vec}(d_j)) = w_{ij}$$

Una semplificazione è quella della presenza (o peso) del termine indice indipendente. L'apparire o meno di un termine indice con l'apparire o meno di un altro termine indice sono avvenimenti indipendenti. Questa cosa permette di avere un modello computazionalmente più efficace, ma rende la ricerca meno precisa.

La funzione di recupero è una funzione che dati i termini indici che rappresentano una query (anche più di una richiesta con operatori booleani AND, OR e NOT), ci permette di recuperare tutti i documenti che si avvicinano alla query.

4.4.1 Booleano

È il più semplice. La rilevanza di un termine indice è binaria, o c'è o non c'è, 0/1. Il vettore è solo un vettore di 0 e 1.

La query è sempre espressione di un'operazione booleana in AND, OR e NOT. Supporta l'exact matching, il documento matcha perfettamente alla query oppure no.

La query dell'utente è rappresentata come un'operazione booleana, l'utente cerca delle keywords mettendole in combinazione booleana con gli operatori booleani.

Il matching funziona come operazioni insiemistiche.

Con il modello booleano sappiamo che per quanto sia complessa la query possiamo rappresentarla nella sua forma normale disgiunta. Ci interessa perchè ci dà i documenti che fanno parte della query. Ad esempio data la query:

$$q = t_a \wedge (t_b \vee \neg t_c)$$

La forma disgiuntiva normale è:

$$qdnf = (t_a \wedge t_b \wedge t_c) \vee (t_a \wedge t_b \wedge \neg t_c) \vee (t_a \wedge \neg t_b \wedge \neg t_c)$$

Da un punto di vista computazionale è la definizione di un albero binario.

Decidere cosa va nelle foglie si decide in base alle regole di precedenza degli operatori booleani (prima l'and e poi gli or).

È importante l'ordine di valutazione, perchè in base a come decido di risolvere le operazioni booleane il risultato cambia.

Ma se ho lo stesso operatore cosa faccio? cosa faccio prima o dopo? da un punto di vista logico non cambia niente. Ma dal punto di vista computazionale dipende dalla cardinalità dell'insieme che contiene la keyword. Solitamente si prendono le cardinalità con più bassa frequenza per poi lasciare quelle con più alta frequenza dopo, faccio prima gli AND veloci e poi quelle lente.

Il problema del modello booleano è che non ci permette di fare un ordinamento dei risultati. C'è anche una difficoltà di espressione della query, non sempre le query sono esprimibili in espressioni booleane.

4.4.2 Modello vettoriale

Si appoggia sull'algebra lineare, non tratta i documenti come insiemi, ma un documento rappresenta un vettore n-dimensionale, con n numero di indici.

Quindi un documento è un vettore di termini pesati, il peso dei termini è un numero tra 0 e 1:

$$w_{kj} : R(\vec{d}) := d_j = (w_{1j}, w_{2j}, \dots, w_{Nj})$$

La query anche lei è un vettore di termini:

$$R(\vec{q}) := q = (w_{1q}, w_{2q}, \dots, w_{Nq})$$

La rilevanza è una funzione di similarità calcolata tra due vettore, sostanzialmente la distanza tra due vettore. Qui torna in gioco la reciproca indipendenza di due termini.

$$Relevance(d) \approx sim(\vec{d}, \vec{q})$$

Un termine indice è un vettore in cui tutti i termini sono 0 tranne se stesso.

Tutti i vettori di termini indici sono linearmente indipendenti uno dall'altro in uno spazio n dimensionale. Sono perpendicolari nello spazio. Mi permette anche di risolvere il problema della similarità. Il documento possiamo scriverlo come la sommatoria dei pesi moltiplicati per il termine indice:

$$\vec{d}_r = \sum w_{ir} \vec{t}_i$$

Come faccio a calcolare la distanza tra due termini indici? immaginiamo di essere in uno spazio bidimensionale, calcolo il coseno dell'angolo che si forma tra due vettori: faccio il prodotto vettoriale degli indici pesati fratto la moltiplicazione dei moduli dei vettori. Quando alfa è zero i due documenti sono identici, quando alfa è 90 gradi i documenti sono diversi.

$$\cos \alpha = \frac{(w_{1s} w_{1r} \vec{t}_1 \cdot \vec{t}_1 + w_{1s} w_{2r} \vec{t}_1 \cdot \vec{t}_2 + w_{2s} w_{1r} \vec{t}_2 \cdot \vec{t}_1 + w_{2s} w_{2r} \vec{t}_2 \cdot \vec{t}_2)}{|\vec{d}_s| |\vec{d}_r|}$$

$\alpha = 0^\circ \rightarrow \vec{d}_s \equiv \vec{d}_r$

La distanza tra una query ed un documento è il prodotto vettoriale tra i pesi:

$$\vec{d}_r = \sum_{i=1}^N w_{ir} \vec{t}_i \qquad \vec{q} = \sum_{j=1}^L q_{ij} \vec{t}_j$$

$$\vec{d}_r \cdot \vec{q}_s = \sum_{i,j=1}^N w_{ir} q_{ij} \vec{t}_i \cdot \vec{t}_j$$

Però il prodotto vettoriale tra gli indici è banale, perchè stiamo moltiplicando vettori perpendicolari, quindi sarà 0 se i due indici sono diversi e 1 se sono uguali: Possiamo quindi risolvere il prodotto vettoriale come una sommatoria dei pesi del documento per i pesi della query:

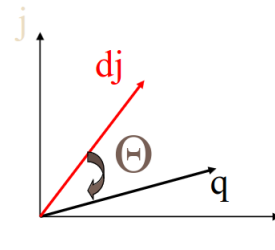
$$\vec{d}_r \cdot \vec{q} = \sum_{i=1}^N w_{ir} X q_i$$

$$\vec{d}_r \cdot \vec{d}_s = \sum_{i=1}^N w_{ir} X w_{is}$$

Quindi la similarità viene risolta da un'operazione matematica:

$$\square \text{sim}(\vec{d}_i, \vec{q}) = \cos(\Theta) = \frac{[\vec{d}_i \bullet \vec{q}]}{|\vec{d}_i| \times |\vec{q}|} =$$

$$\frac{\left(\sum_{i=1}^N w_{ij} \times w_{iq} \right)}{\sqrt{\sum_{i=1}^N w_{ij}^2} \times \sqrt{\sum_{i=1}^N w_{iq}^2}}$$



La similarità che si ottiene è tra 0 e 1. Ottengo una distanza che viene presentata con un numero tra 0 e 1, quindi posso fare un ordinamento alla fine.

Spazio virtuale del documento

Original state	After the assignment of terms having a good discriminating factor	After the assignment of terms having low discriminating factor

Essendo vettori, i termini indici posso rappresentarli in uno spazio vettoriale, quando i vettori nello spazio sono simili sono vicini (densità spazio vettoriale compatta), quando sono differenti sono distanti (densità spazio vettoriale grossa). Decidere se aggiungere o meno un termine indice porta ad una conseguenza, perchè entra in gioco nella sommatoria un nuovo termine, che può portare a rendere più distanti o più vicini i documenti.

Aggiungendo un termine nuovo quando questo è un buon discriminante? quando faccio la query ho una risposta più vicina? quando questo termine allontana i vettori nello spazio vettoriale. Quindi potrebbe essere bene non considerare i termini non discriminanti nella creazione dell'archivio, anche se stiamo facendo l'archivio di un dominio specifico ci saranno dei concetti base ripetuti, ma comunque non posso buttare via quel concetto (ad esempio giudizio in documenti giuridici).

Si dice che un buon separatore di documenti è il termine indice che ci permette di aumentare la distanza tra i vettori nello spazio vettoriale, quindi avere uno spazio meno denso.

Dal punto di vista matematico si definisce così, la densità di uno spazio vettoriale è il calcolo della similarità di tutti i documenti in esso contenuti:

$$Den = 1 / (N(N-1)) \sum_{i=1}^N \sum_{k=1, i \neq k}^N sim(d_i, d_k)$$

Per calcolare se un nuovo termine è un buon discriminante calcolo la densità prima e dopo.

Pesi

É vero che la frequenza di termine nel documento ne dà il peso, ma c'è un modello TFIDF che permette di calcolare il peso di un termine in base alla sua frequenza nel documento (ne rappresenta la rilevanza) e un secondo termine ne rappresenta la frequenza del termine in tutto il dominio dei documenti. Ovviamente più il termine è presente da per tutto meno è rilevante, perchè è meno specifico e non permette di discriminare bene quel documento.

$$w_{ij} = tf(i, j) \cdot idf(i)$$

Frequenza normalizzata fratto l'inverso della frequenza globale. La frequenza normalizzata significa normalizzare la frequenza del termine che stiamo considerando fratto la frequenza del termine che appare più frequentemente:

$$tf(i, j) = freq(i, j) / max(freq(I, j))$$

Poi mi interessa l'inverso di quanto è frequente in tutto il dominio, log di N (numero di documenti) fratto il numero di documenti che contengono quel termine.

$$idf(i) = \log(N/n_i)$$

Vantaggi:

- Concetto di peso del termine
- Comparare query e documenti, quindi avere un ordinamento delle similarità tra query e documenti

Svantaggi:

- Vettori linearmente indipendenti

- I termini che non sono presenti in una query non hanno rilevanza
- A volte si parte da una query che non sempre è espressiva, è sempre una combinazione di termini

4.5 ORACLE Text

Oracle dice che il testo è ovunque, ovviamente testo non strutturato ad esempio la mail. Vedremo che in ORACLE c'è la possibilità di fare delle query specifiche se il testo ha una struttura.

Vedremo anche a livello teorico i tesauri.

Gestisce fino a 150 tipi di formati. La potenza linguistica di ORACLE è molto avanzata, noi faremo esempi in inglese, si può però implementare il dizionario italiano. Il concetto di indice inverso è esteso.

Questa è l'architettura del modello ORACLE, notiamo che ci sono dei moduli comuni al modello classico.

Il data store è il posto in cui mettiamo i documenti, potremmo anche prendere dei documenti in rete o ad esempio avere delle procedure che chiamano API che raccolgono testi. Sostanzialmente dobbiamo andare a personalizzare dei moduli. Il filter è un modulo automatico, che dizionario stiamo usando, da un documento ne estrae il flat text. Il sectionar è il modulo in cui definiamo se ci sono delle sezioni predefinite nel testo. Il Lexter è il modulo che mi crea i tokens, si occupa di normalizzare il testo i token che crea sono tematici, chiaramente dipende molto dalla lingua, quindi devo definire quale dizionario userò. Possiamo rimuovere le stop list. Al che otteniamo l'indice inverso su cui possiamo fare le interrogazioni.

Nella creazione degli indici inversi non usa come pesi TF-IDF ma usa una cosa più facile a livello matematico.

La personalizzazione dei moduli viene implementata tramite una logica di preferences. Devo istanziare un oggetto di preferenze, in questo oggetto vado a definire le mie preferenze, poi questo oggetto posso passarlo al modulo.

4.5.1 Main Features

Tipi di indice gestiti:

- CONTEXT: classico di full text retrieval
- CTXCAT: catalog Index usata per gestire cataloghi di e-business
- Altri indici per il mondo XML che non vedremo

Per fare delle query contenitivi si usa la keyword CONTAINS. Prima di fare queste query devo però prima creare gli indici, per creare un indice full text base, non elimina quindi prende tutti i token del documento. Deve avere un nome, book_title, devo dirgli dove sono i data store:

```
1 create index books_title on books ( title )
2 indextype is ctxsys.context;
```

Con questa query andiamo a vedere gli indici che abbiamo creato:

```
1 select
2     index_name          name ,
3     index_type          type ,
4     ityp_owner          "type owner",
5     ityp_name           "type name",
6     domidx_opstatus     d_status
7 from user_indexes
8 where index_name like 'BOOKS%';
```

Vediamo meglio le query contenutistiche. Lo score è il calcolo della distanza tra il vettore query ed il vettore documento:

```
1 select title, score(42) s from books
2 where contains ( title, 'dog', 42 ) >= 0
3 order by s desc;
```

In teoria la distanza dovrebbe essere un numero tra 0 e 1 (la distanza tra i vettori, il coseno) ma ottengo valori maggiori di 1, questo perchè la normalizzazione di oracle va da 0 a 100. Score è un indice numerico posizionale per dire nella parte di select voglio uno score, ma lo voglio della contains che è stata etichettata con un particolare numero, questo perchè nella where potrei avere tante contains ognuna deve avere un indice (tag posizionale), questo indice numerico è usato per distinguere la contains. Deve essere un numero ma non è necessario partire da uno.

Posso fare query esatte sulla keyword, sulle parole e le inexact match non esattamente la parola che ti ho dato ma una parola simile. Fino ad arrivare all'intelligence match.

Questa è la creazione dell'indice più astratta:

```
1 Create index (myindex) on DOC (htmlfile) INDEXTYPE IS
2 ctxsys.context
3 PARAMETERS (
4     'DATASTORE ctx.sys.default_datastore
5     FILTER ctxsys.null_filter
6     SECTION GROUP stxsys.html_section_group')
```

Qualora il datastore fosse nel file system devo dirglielo, ho un oggetto di tipo file_datastore che creo e poi ne popolo gli attributi:

```
1 ctx_ddl.create_preference('mypref',
2     'FILE_DATASTORE');
3
4 ctx_ddl.set_attribute('mypref',
5     'PATH', '/docs');
6
7 create index doc_index on docs(text)
8     indextype is ctxsys.context
9     parameters ('datastore mypref');
```

La pesatura viene calcolata come 3 volte la frequenza del termine mentre l'idf è simile, cambia dal punto di vista matematico. Lo score viene calcolato in base a come è fatta la query. In ORACLE posso usare gli operatori logici, quando uso questi operatori come viene calcolato lo score?

- AND: entrambi devono essere true è come se avessi due query, ho due calcoli di distanza quindi in questo caso recupero il valore minore.

- OR: almeno uno deve essere vero ed il valore che viene ritornato è il massimo

L'indice che andiamo a creare non posso usarlo come il CONTEXT di default, devo creare un oggetto preference da passargli:

```

1 begin
2   Ctx_Ddl.Create_Index_Set (
3     set_name => 'my_index_set' );
4   Ctx_Ddl.Add_Index (
5     set_name => 'my_index_set',
6     column_list => 'price' );
7 end;
8 /
9
10 create index books_title_cat on books ( title )
11   indextype is ctxsys.ctxcat
12   parameters ( 'index set my_index_set' );

```

ctxcat è un indice creato da ORACLE che permette di unire query relazionali e contenutistiche, ad esempio i cataloghi sicuramente avranno degli ID e delle descrizioni. I prodotti avranno una parte testuale e dei metadati. Devo però indicare che metadato voglio ottimizzare, dichiaro un indice catalogo e aggiungo il prezzo.

Facciamo delle query. Con questa query sto ancora usando l'indice CONTEXT:

```

1 select price, title from books
2   where contains ( title, 'dog | cat' ) > 0
3   and price < 15
4   order by price desc;

```

Se invece voglio usare il ctxcat la sintassi cambia, catsearch cerca un indice ctxcat, la query è sempre una combinazione di due cose, contenuto e query strutturata legata all'indice catalogo:

```

1 select price, title from books
2   where catsearch (
3     title,
4     'dog | cat',
5     'price < 15 order by price desc' ) > 0;

```

Non è che cambia niente in questo caso, ma questa query è ottimizzata per fare la seconda query, quindi il tempo di esecuzione diminuirebbe.

4.5.2 Query particolari

Proviamo a vedere qualche query particolare, possono essere di tipo:

- Logical
- Di prossimità
- Fuzzy
- Stemming
- Thesaurus

- Wildcards
- Ricerca sulle sezioni

Accumulate

É un operatore, ho documenti molto simili ma con frequenze diverse:

```
1 create table accum_tbl(id number primary key, text clob);
2 insert into accum_tbl values(1,'cat dog cow');
3 insert into accum_tbl values(2,'    dog cow');
4 insert into accum_tbl values(3,'cat      cow');
5 insert into accum_tbl values(4,'cat dog    ');
6 insert into accum_tbl values(5,'cat cat cat cat cat cat cat cat cat
   cat');
7 insert into accum_tbl values(6,'dog dog dog dog dog dog dog dog dog
   dog');
8 insert into accum_tbl values(7,'cow cow cow cow cow cow cow cow cow
   cow');
9 commit;
```

Creiamo l'indice:

```
1 create index accum_tbl_text on accum_tbl( text )
2 indextype is ctxsys.context
```

L'accumulate è simili all'or, l'or andrebbe a prendere quello con peso maggiore, la frequenza conta in nel calcolo del peso in ORACLE. Se invece non mi interessa la frequenza ma non voglio che ci sia l'AND per forza, cioè è più interessante che ci siano tutti e tre con meno frequenza rispetto a due con più frequenza. In ORACLE si usa l'accumulate:

```
1 select score(1) s, text from accum_tbl
2 where contains ( text, 'cat, dog, cow', 1 ) > 0
3 order by s desc;
```

L'accumulate somma i pesi.

Near

Creo le tabelle:

```
1 create table near_tbl(id number primary key, docs clob);
2 insert into near_tbl values(117,'My dog will chase and chase the big
   cat around the house until she falls with exhaustion. ');
3 insert into near_tbl values( 118,'I have to work Monday Tuesday
   Wednesday Thursday and Friday weekly and am assigned weekends based
   on schedule. ');
4 insert into near_tbl values( 119,'It is hard to raise a dog and cat in
   the same house with a mouse. ');
5 insert into near_tbl values( 120,'Tuesday Wednesday Monday and Friday I
   only work until 4:00. ');
6 commit;
```

Creo l'indice:

```
1 create index near_idx on near_tbl( docs )
2 indextype is ctxsys.context;
```

dammi i documenti che contengono queste parole ma entro 20 caratteri:

```
1 select id,docs,score(1) from near_tbl
2   where contains ( docs, 'near((monday,tuesday,wednesday),20,TRUE)',1 )
3     > 0
3 order by score(1) desc;
```

Minus

Sottrae gli score:

```
1 select id,docs,score(1) from minus_tbl
2   where contains ( docs, 'cars - ford',1 ) > 0
3 order by score(1) desc;
```

Equivalence

É abbastanza semplice, ci permette di andare a sfruttare un concetto di equivalenza tra due termini che per me hanno lo stesso significato. In questi insert abbiamo due insert di big dogs, quando faccio una query sui big dogs vorrei che restituisse entrambi:

```
1 drop table equiv_tbl;
2
3 create table equiv_tbl(id number primary key, docs clob);
4 insert into equiv_tbl values(117,'Labradors are big dogs. ');
5 insert into equiv_tbl values( 118,'Alsatians are big dogs. ');
6 commit;
```

L'indice è quello di base:

```
1 drop index equiv_idx;
2 create index equiv_idx on equiv_tbl( docs )
3   indextype is ctxsys.context;
```

La usiamo tramite l'operatore "=":

```
1 -- Equivalence Example...
2 select id,docs,score(1) from equiv_tbl
3   where contains ( docs, 'labradors=alsatians are big dogs',1 ) > 0
4 order by score(1) desc;
```

Soundex

Funziona solo con la lingua inglese.

```
1 drop table soundex_tbl;
2
3 create table soundex_tbl(id number primary key, text clob);
4 insert into soundex_tbl values(111,'Jon Smith');
5 commit;
```

Creo l'indice di base:

```
1 drop index soundex_idx;
2 create index soundex_idx on soundex_tbl( text )
3   indextype is ctxsys.context;
```

Funziona con il "!", recupera tutti i documenti che hanno un suono fonico che è simile a quello che ti do in input:

```
1 select text from soundex_tbl
2   where contains ( text, '!John',1 ) > 0 /* finds "Jon" */;
3
4 select text from soundex_tbl
5   where contains ( text, '!Smythe',1 ) > 0 /* finds "Smith" */;
```

Sque

Voglio creare una base documentale in cui si parla di disastri ambientali, mi piacerebbe che il documento che contiene un istanza legata ad un disastro naturale (non importa quale è) venga recuperato.

Creo le tabelle con due documenti:

```
1 drop table sque_tbl;
2
3 create table sque_tbl(id number primary key, docs clob);
4 insert into sque_tbl values(112,'Florida was hit by hurricane Fran 2
5   years ago');
6 insert into sque_tbl values(113,'this row should not be returned');
7 commit;
```

La sequenza è un oggetto, dentro lo schema object relational, che contiene tutti i termini che so essere associati.

```
1 drop index sque_idx;
2
3 begin
4   Ctx_Query.Remove_Sqe ( 'disasters' );
5 end;
6 /
7 begin
8   Ctx_Query.Store_Sqe ( 'disasters', 'hurricane, earthquake, blizzard'
9   );
10 end;
11 /
12 create index sque_idx on sque_tbl( docs )
13   indextype is ctxsys.context;
```

Voglio tutti i documenti che contengono una qualunque parola che fa parte della sequenza (il nome della sequenza è il primo termine della sequenza stessa)

```
1 select id,docs,score(1) from sque_tbl
2 where contains ( docs, 'sque(disasters) and florida',1 ) > 0
3 order by score(1) desc;
```

È ancora una ricerca puntuale, ma ci stiamo avvicinando alle ricerche tematiche.

Stemming

È il modulo che mi consente di andare a considerare la radice del token (non tutto il token), Oracle non utilizza lo stem in fase di indicizzazione (non indicizza usando le radici

delle parole, ma gli indici inversi sono tutti i termini che compaiono). Utilizza invece lo stemming in fase di espansione della query, usando lo stem posso dire "dammi tutti i documenti che sono indicizzati ad un indice che può essere ricondotto a questa radice che ti ho dato come input della mia interrogazione".

Useremo un modulo di oracle, Stemmer. Creiamo la tabella:

```
1 drop table stemmer_tbl;  
2  
3 create table stemmer_tbl(id number primary key,docs clob);  
4 insert into stemmer_tbl values(111,'We are testing the Stemmer  
   attribute option');  
5 insert into stemmer_tbl values(112,'The girl sang like a frog');  
6 insert into stemmer_tbl values(113,'My import is committing too often');  
7 commit;
```

La creazione dell'indice è più articolata, perchè implica che nella creazione del Context gli vada a passare un oggetto di preferenze, che è di tipo word list a cui specifico un po' di parametri. Questo oggetto dice al creatore dell'indice inverso quale dizionario dati utilizzare nella creazione degli stem. Come primo parametro di questo oggetto gli dico che dizionario linguistico userò per fare le interrogazioni basate su stemming.

```
1 --  
2 -- Basic WordList Stemmer attribute  
3 -- This example uses English for Stemming.  
4 -- Language choices are:  
   NULL,ENGLISH,DERIVATIONAL,DUTCH,FRENCH,GERMAN,ITALIAN  
5 -- and SPANISH. By default we use the language of the database. If the  
6 -- database is not one of the listed languages then we choose NULL for  
   stemmer  
7 -- and default for fuzzy  
8 --  
9  
10 begin  
11   Ctx_Ddl.Drop_Preference ( 'stemmer_pref' );  
12 end;  
13 /  
14  
15 drop index stemmer_idx;  
16  
17 begin  
18   Ctx_Ddl.Create_Preference('STEMMER_PREF', 'BASIC_WORDLIST');  
19   ctx_ddl.set_attribute('STEMMER_PREF','STEMMER','ENGLISH');  
20 end;  
21 /  
22  
23 create index stemmer_idx on stemmer_tbl ( docs )  
24   indextype is ctxsys.context  
25   parameters ('Wordlist STEMMER_PREF');
```

Una volta che ho specificato in maniera esplicita il dizionario linguistico posso utilizzare la keyword \$, che mi cerca tutti i documenti che hanno come radice la radice della parola che metto dopo, nel nostro caso test:

```
1 select id,docs,score(1) from stemmer_tbl  
2   where contains ( docs, '$test',1 ) > 0;
```

Così facendo verrebbe recuperato il documento che contiene "testing".

Fuzzy

Come potrei rimediare ad un errore che commette l'utente quando fa una query. La logica fuzzy implica la ricerca del termine che ho dato e di una sua derivata in base a delle condizioni.

Creiamo la base documentale:

```
1 drop table fuzzy_tbl;
2
3 create table fuzzy_tbl(id number primary key,docs clob);
4 insert into fuzzy_tbl values(111,'We are testing the Fuzzy attribute
   option in the baisic wordlist feature. ');
5 insert into fuzzy_tbl values(112,'Basic is a programming language');
6 insert into fuzzy_tbl values(114,'cat');
7 insert into fuzzy_tbl values(113,'cats');
8 insert into fuzzy_tbl values(115,'calc');
9 insert into fuzzy_tbl values(116,'case');
10
11
12 commit;
```

Quando creo l'indice devo specificare delle preferenze, non devo specificare il dizionario linguistico, ma dobbiamo specificare:

- SCORE: ci dice la distanza che intercorre tra il termine scritto ed eventuali ricombinazioni dello stesso termine: Ad esempio se ho computer, voglio tutte le parole che hanno una distanza di 50 dalla parola computer (computers, bomputers...)
- NUMRESULTS: partendo da quelle più vicine me ne trovi altre 1000/5000. Non usa lo score, è l'alternativo dello SCORE

```
1 --
2 -- Basic WordList Fuzzy attribute
3 -- This example uses English for Fuzzy searching.
4 -- Language choices are:
   GENERIC,JAPANESE_VGRAM,KOREAN,CHINESE_VGRAM,ENGLISH,
5 -- DUTCH,FRENCH,GERMAN,ITALIAN,SPANISH,and OCR
6 -- and SPANISH. By default we use the GENERIC for fuzzy_match.
7 --
8
9 begin
10   Ctx_Ddl.Drop_Preference ( 'fuzzy_pref' );
11 end;
12 /
13
14 drop index fuzzy_idx;
15
16 begin
17   Ctx_Ddl.Create_Preference('FUZZY_PREF', 'BASIC_WORDLIST');
18   ctx_ddl.set_attribute('FUZZY_PREF','FUZZY_MATCH','ENGLISH');
19 --   ctx_ddl.set_attribute('FUZZY_PREF','FUZZY_SCORE','0');
20   ctx_ddl.set_attribute('FUZZY_PREF','FUZZY_NUMRESULTS','5000');
```

```

21 -- ctx_ddl.set_attribute('FUZZY_PREF','STEMMER','NULL');
22 end;
23 /
24
25 create index fuzzy_idx on fuzzy_tbl ( docs )
26     indextype is ctxsys.context
27     parameters ('Wordlist FUZZY_PREF');

```

Per usarlo devo usare il "?", prendi la parola basic, dato che abbiamo usato il numresult mi generi 5000 parole simili di basic e queste saranno un'espansione della query in input. Quindi è come se da una query ne avessi fatte 5000.

```

1 select id,docs,score(1) from fuzzy_tbl
2 where contains ( docs, '?basic',1 ) > 0;
3
4 select * from fuzzy_tbl
5 where contains (docs, '?cat')>0;

```

Lexer

La ricerca tematica usa un altro oggetto per quanto riguarda la personalizzazione dell'indice. Il Lexer ci permette di andare a lavorare sugli indici.

Quello basic ci permette di definire come vogliamo tokenizzare il documento, possiamo specificare ad esempio la spaziatura, la punteggiatura, il simbolo di interlinea. Potrei andare a personalizzare alcune logiche ortografiche del dizionario che sto caricando. Specifico diverse lingue su cui andremo a lavorare. Solitamente non si vanno a personalizzare le caratteristiche della sintassi ortografica del dizionario. La cosa più interessante è il concetto di indice tematico.

```

1 drop table index_theme_tbl;
2
3 create table index_theme_tbl (id number primary key, docs clob);
4
5 insert into index_theme_tbl values(113,'Your Mailbox
6 Standard mailboxes hold 25 total messages at a maximum of 3 minutes
7 each.');
```

Vediamo come creare l'indice. per creare query tematiche se utilizziamo come base il dizionario inglese anche il contesto del canvas deve essere di lingua inglese altrimenti c'è una discrepanza tra la lingua usata per fare query e la nostra sessione. Dobbiamo quindi usare l'alter session per modificare la lingua usata nel nostro contesto. Creo la Preference, sto utilizzando un BASIC LEXER, l'unica cosa che devo fare è decidere se fare indicizzazione full text o tematica (uno false e l'altro true).

```

1 ALTER SESSION SET NLS_LANGUAGE=English;
2
3 drop index index_theme_idx;
4 begin
5     Ctx_Ddl.Drop_Preference ('index_theme_lexer_pref');
6 end;
7 /
8

```

```

9 begin
10   Ctx_Ddl.Create_Preference ( 'index_theme_lexer_pref', 'BASIC_LEXER' );
11   Ctx_Ddl.Set_Attribute ( 'index_theme_lexer_pref', 'index_themes',
12     'TRUE' );
13   Ctx_Ddl.Set_Attribute ( 'index_theme_lexer_pref', 'index_text',
14     'FALSE' );
15 end;
16 /
17
18 create index index_theme_idx on index_theme_tbl ( docs )
19   indextype is ctxsys.context
20   parameters ( 'LEXER INDEX_THEME_LEXER_PREF' );

```

La sintassi cambia leggermente, se abbiamo un indice tematico dobbiamo usare la parola chiave ABOUT, dammi tutti i documenti che parlano di mailbox. Possiamo anche avere la query che ci da tutti i token creati a partire dall'indice:

```

1 -- Selects the indexed words from the Token Table
2 select token_text from dr$index_theme_idx$i;
3
4 select id, docs, score(1) from index_theme_tbl
5   where contains ( docs, 'about(mailbox)', 1 ) > 0;

```

Themes

Creiamo il documento:

```

1 drop table words;
2
3 create table words
4   (
5     id          number primary key,
6     text        varchar2(80)
7   );
8
9 insert into words ( id, text )
10   values ( 1, 'I cat dog bird botany zoology car.' );
11 commit;

```

Creiamo l'indice, che vogliamo sempre essere un indice inverso tematico:

```

1 begin
2   Ctx_Ddl.Drop_Preference ( 'themes_only' );
3 end;
4 /
5 begin
6   Ctx_Ddl.Create_Preference ( 'themes_only', 'basic_lexer' );
7   Ctx_Ddl.Set_Attribute ( 'themes_only', 'index_text', 'false' );
8   Ctx_Ddl.Set_Attribute ( 'themes_only', 'index_themes', 'true' );
9 end;
10 /
11
12 create index words_text on words ( text )
13   indextype is ctxsys.context
14   parameters ( 'lexer themes_only stoplist ctxsys.empty_stoplist' );
15 select token_text from dr$words_text$i

```

```
16 order by token_text;
```

Ci piacerebbe estrarre da un documento il temi e il peso dei temi, li voglio travasare in una tabella:

```
1 drop table ctx_themes;
2 create table ctx_themes (
3     query_id    number,
4     theme       varchar2(2000),
5     weight      number );
6
7 delete from ctx_themes
8     where query_id = 1;
9 begin
10     Ctx_Doc.Set_Key_Type ( Ctx_Doc.Type_Primary_Key );
11     Ctx_Doc.Themes (
12         index_name => 'words_text',
13         textkey     => 1,
14         restab      => 'ctx_themes',
15         query_id    => 1,
16         full_themes => true );
17 end;
18 /
19 select weight, theme from ctx_themes
20     where query_id = 1
21     order by weight desc;
```

In Themes dobbiamo specificare l'indice tematico che abbiamo creato, poi specifichiamo l'indice univoco utilizzato per identificare il documento da cui estrarre il tema (in questo caso la primary key è di valore 1). Poi diciamo dove travasare i temi specificando il nome della tabella e la primary key. Inoltre diciamo che vogliamo tutta l'alberatura dei temi (full_themes = true).

Esempio

Possiamo caricare un documento usando delle API che ci mette a disposizione Oracle.

Gist

Posso tramite il gist creare il riassunto di un documento. Il doc gist mi chiede quel'è l'indice tematico da usare per estrarre il riassunto, quel'è la keyword per identificare il documento, dove devo andare a metterlo e con quale indice. Il POV mi dice se voglio fare un riassunto generico o tematico:

- Generico: dato un testo fai il riassunto
- Tematico: fammi un riassunto ma con particolare attenzione su un particolare tema

```
1 drop table ctx_gists;
2 create table ctx_gists (
3     query_id    number,
4     pov         varchar2(80),
5     gist        clob );
```

```

6
7 delete from ctx_gists
8   where query_id = 2;
9 begin
10   Ctx_Doc.Gist (
11     index_name      => 'realistic_docs_text',
12     textkey         => 2,
13     restab          => 'ctx_gists',
14     query_id        => 2,
15     pov             => 'GENERIC',
16     glevel          => 'paragraph',
17     numparagraphs   => 2,
18     maxpercent       => 100 );
19 end;

```

L'indice:

```

1 begin
2   Ctx_Ddl.Drop_Preference ('themes_only');
3 end;
4 /
5 begin
6   Ctx_Ddl.Create_Preference ( 'themes_only', 'basic_lexer');
7   Ctx_Ddl.Set_Attribute     ( 'themes_only', 'index_text',   'false' );
8   Ctx_Ddl.Set_Attribute     ( 'themes_only', 'index_themes', 'true'  );
9 end;
10 /
11
12 create index realistic_docs_text on realistic_docs ( text )
13   indextype is ctxsys.context
14   parameters ( 'lexer themes_only stoplist ctxsys.empty_stoplist' );
15 select token_text from dr$realistic_docs_text$i;

```

Attenzione che il riassunto è molto meglio farlo con un LLM, sono molto più intelligenti.

Stoplist

Quando generiamo un indice possiamo dire di non considerare tutti i token ma di considerare solo alcuni token specifici perchè altri sono poco discriminanti.

```

1 drop table stopwords;
2
3 create table stopwords
4   (
5     stopwords_id          number
6     constraint stopwords_pk primary key,
7     text                  varchar2(200)
8   );
9
10 insert into stopwords ( stopwords_id, text ) values (
11   1,
12   'An and envelope is on its journey to you. Not a catastrophe. 1234
13     5678' );
14
15 insert into stopwords ( stopwords_id, text ) values (

```

```

15 2,
16 'And by the fascinating postmark I understand it was him. 98.76
17 54.32' );
18 commit;

```

Se creo un indice così mi usa tutti i token, come vediamo dalla query:

```

1 begin
2   Ctx_Ddl.Drop_Preference ('my_lexer');
3 end;
4 /
5 begin
6   Ctx_Ddl.Create_Preference ( 'my_lexer', 'basic_lexer');
7   Ctx_Ddl.Set_Attribute      ( 'my_lexer', 'index_text',   'true');
8   Ctx_Ddl.Set_Attribute      ( 'my_lexer', 'index_themes', 'false');
9 end;
10 /
11
12 drop index stopwords_text;
13 create index stopwords_text on stopwords ( text )
14   indextype is ctxsys.context
15   parameters ( 'lexer my_lexer stoplist ctxsys.empty_stoplist' );
16
17 select token_text from dr$stopwords_text$i
18   order by length(token_text), token_text;
19
20 -- TOKEN_TEXT
21 -- -----
22 -- A
23 -- I
24 -- AN
25 -- BY
26 -- IS
27 -- IT
28 -- ON
29 -- TO
30 -- AND
31 -- HIM
32 -- ITS
33 -- NOT
34 -- THE
35 -- WAS
36 -- YOU
37 -- 1234
38 -- 5678
39 -- 54.32
40 -- 98.76
41 -- JOURNEY
42 -- ENVELOPE
43 -- POSTMARK
44 -- UNDERSTAND
45 -- CATASTROPHE
46 -- FASCINATING

```

Devo quindi creare una stoplist, faccio un alter index per andare a fare il replace della stoplist:

```

1 begin
2   Ctx_Ddl.Drop_Stoplist ( 'my_stoplist' );
3 end;
4 /
5 begin
6   Ctx_Ddl.Create_Stoplist ( 'my_stoplist' );
7   Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'a' );
8   Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'i' );
9   Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'an' );
10  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'by' );
11  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'is' );
12  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'it' );
13  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'on' );
14  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'to' );
15  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'and' );
16  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'him' );
17  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'its' );
18  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'not' );
19  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'the' );
20  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'was' );
21  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'you' );
22  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'both' );
23  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'that' );
24  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'with' );
25  Ctx_Ddl.Add_Stopword ( 'my_stoplist', 'notwithstanding' );
26
27  Ctx_Ddl.Add_Stopclass ( 'my_stoplist', 'numbers' );
28 end;
29 /
30
31 alter index stopwords_text rebuild /* can't use "online" */
32   parameters ( 'replace stoplist my_stoplist' );
33
34 select token_text from dr$stopwords_text$i
35   order by length(token_text), token_text;
36
37 -- TOKEN_TEXT
38 -- -----
39 -- JOURNEY
40 -- ENVELOPE
41 -- POSTMARK
42 -- UNDERSTAND
43 -- CATASTROPHE
44 -- FASCINATING
45
46 -- The 'replace stoplist my_stoplist' command has BOTH updated the
47 -- index metadata with the new stopwords AND deleted all rows for
48 -- these tokens from the $I table

```

Thesaurus

Un TESAURO rappresenta una ontologia/tassonomia che va a descrivere un dominio applicativo o tutta una lingua. In Oracle dobbiamo pensare ad una struttura di termini (dizionario) che ha delle associazioni che legano le parole appartenenti a questo dizionario.

In Oracle si usano i tesauri tematici, non generici perchè vorrebbe dire creare un tesoro di una intera lingua (dovrei considerare tutto un dizionario e i collegamenti tra tutte le parole). I tematici sono più semplici da realizzare perchè sono per un dominio specifico. Quando utilizzo un tesoro viene usato per estrarre i token che rappresentano una struttura ad indice, ci saranno dei token che rappresenteranno un particolare dominio applicativo e verranno usati per generare una struttura ad indice di una base documentale.

Facciamo un esempio, pensiamo un tesoro che è un dizionario di termini che potrebbe avere rilevanza in un dominio specifico. Ricordiamo che un tesoro arricchisce un dizionario linguistico.

I link più semplici sono quelli legati alla sinonimia, chiamati PREFERENTIAL LINK (PT), posso quindi descrivere nel dizionario che ci sono dei sinonimi (Student PT Pupil). Esistono anche i Broader Term (BT) e i Narrow Term (NT) ad esempio Vehicle BT Car e Car NT Vehicle. Lo faccio per andare ad espandere la query.

Possiamo anche specificare dei Relation Term (RT) senza specificare il tipo di relazione che hanno.

Aspetti negativi:

- Abbiamo bisogno di esperti del dominio per creare le connessioni
- Va mantenuto nel tempo

I vantaggi sono ovviamente quelli di espandere le query.

Vediamo un esempio. Creo inizialmente l'indice inverso specificando la preference ed il lexer da utilizzare, poi nella query recupero i token che ha estratto dal documento:

```
1 begin
2   Ctx_Ddl.Drop_Preference ('animals_pref');
3 end;
4 /
5
6 begin
7   Ctx_Ddl.Create_Preference ( 'animals_pref', 'basic_lexer' );
8   Ctx_Ddl.Set_Attribute   ( 'animals_pref', 'index_text', 'true' );
9   Ctx_Ddl.Set_Attribute   ( 'animals_pref', 'index_themes', 'false' );
10 end;
11 /
12
13 drop index thes_animals_text;
14 create index thes_animals_text on animals_txt ( text )
15   indextype is ctxsys.context
16   parameters ( 'lexer animals_pref' );
17
18
19
20 select token_text from dr$thes_animals_text$i
21   order by token_text;
```

Invece per estrarre i temi creo la tabella in cui metterò i temi, poi con il for ci metto dentro i temi:

```
1 drop table kb_ext_themes;
2
3 create table kb_ext_themes
```

```

4  (
5      query_id    number,
6      theme       varchar2(2000),
7      weight      number
8  );
9
10 begin
11     for r_articles in
12     (
13         select article_id from animals_txt
14     )
15     loop
16         ctx_doc.themes
17         (
18             index_name => 'thes_animals_text',
19             textkey     => to_char(r_articles.article_id),
20             restab      => 'kb_ext_themes',
21             query_id    => to_char(r_articles.article_id),
22             full_themes => true
23         );
24     end loop;
25 end;
26 /
27
28 select theme from kb_ext_themes
29 order by weight desc;

```

Adesso invece proviamo ad estendere la knowledge base, possiamo estendere il dizionario dei termini, non dei temi. Per creare un tesauro in Oracle posso farlo o tramite riga di comando o usando le API di Oracle. Devo creare una knowledge base:

```

1 cat
2   SYN feline
3   NT domestic cat
4   NT wild cat
5   BT mammal
6 mammal
7   BT animal
8 domestic cat
9   NT Persian cat
10  NT Siamese cat
11 wild cat
12  NT tiger
13 tiger
14  NT Bengal tiger
15 dog
16  BT mammal
17  NT domestic dog
18  NT wild dog
19  NT fufi
20  SYN canine
21 domestic dog
22  NT German Shepard
23 wild dog
24  NT Dingo

```

Una volta che abbiamo creato i tesauro dobbiamo utilizzare un particolare comando di Oracle:

```
1 ctxload -user <nomeUtente> -name thes_animal -file animali/animals.thes  
  -thes -thescase y
```

Non basta fare questo, bisogna anche compilarlo:

```
1 ctxkbtc -user <nomeUtente> -name THES_ANIMAL -verbose
```

Posso crearlo anche tramite SQL, FALSE è per il case sensitive:

```
1 begin  
2 ctx_thes.create_thesaurus('thes_animal', FALSE);  
3 end;
```

La cosa scomoda è che in SQL per inserire devo inserire tutti i termini da mettere nel dizionario e poi esprimere la relazione tra di essi:

```
1 ctx_thes.create_phrase('thes_animal','animal');  
2 ctx_thes.create_phrase('thes_animal','mammal');  
3 ctx_thes.create_relation('thes_animal','animal','NT','mammal');  
4 ctx_thes.create_relation('thes_animal','mammal','NT','cat');
```

Vediamo delle query, voglio prendere a partire dal tesauro voglio vedere qual'è l'espansione della parola dog:

```
1 set serveroutput on  
2 declare  
3 terms varchar2(2000);  
4 begin  
5 terms := ctx_thes.nt('dog', 1, 'thes_animal');  
6 dbms_output.put_line('The narrow expansion for DOG is: '||terms);  
7 end;
```

Invece se voglio vedere i sinonimi posso fare:

```
1 declare  
2 synonyms_cat varchar2(2000);  
3 synonyms_dog varchar2(2000);  
4 begin  
5 synonyms_cat := ctx_thes.trsyn('cat','ALL','thes_animal');  
6 synonyms_dog := ctx_thes.trsyn('dog','ALL','thes_animal');  
7 dbms_output.put_line('all synonyms for CAT are: '||synonyms_cat);  
8 dbms_output.put_line('all synonyms for DOG are: '||synonyms_dog);  
9 end;
```

Se voglio sfruttare il tesauro per fare delle interrogazioni tematiche di dominio, voglio prendere tutti i termini che contengono dog e anche i termini Broader Term di Dog. Tutti i documenti in cui compare la parola dog e i suoi fili:

```
1 select text from animals_txt where contains ( text, 'BT ( dog, 1,  
  thes_animal )' ) > 0;
```

In questo modo possiamo creare le query legate al tesauro specifiche di dominio, quindi sono query di dominio.

4.5.3 Valutazione

Come faccio a giudicare se il motore di text mining è buono? tramite due indici:

- Precisione
- Recall

Partiamo da una collezione, da delle query e da delle risposte per verificare quanto è buono il sistema di recupero informazioni. Per andare a valutare il processo di recupero devo vedere quanto il motore recupera cosa giuste, quante cose non rilevanti recupera (il rumore che crea), piuttosto che non mi da cose che mi interessano (perdita di informazioni).

Abbiamo i:

- Recuperati
- Non recuperati
- Rilevanti
- Non rilevanti

La combinazione corretta la ho quando recupero ciò che è rilevanti. Ho errori quando recupero quello non rilevante o non recupero quelli rilevanti.

La query evaluation va a studiare quanto è negativo l'impatto dei fasi positivi e falsi negativi.

La recall va a studiare di tutti i rilevanti quanto ne è recuperati:

$$Recall = \frac{RecoveredRelevant}{TotalRelevant}$$

Ovviamente il sistema perfetto sarebbe quello in cui ho preso tutti i rilevanti.

La precisione dice di quelli che ho recuperato quanti sono effettivamente rilevanti?

$$Precision = \frac{RecoveredRelevant}{TotalRecovered}$$

Anche qua il rapporto a 1 sarebbe il migliore.

Chapter 5

Modello Probabilistico

Fa un mix tra modello booleano, vettoriale e probabilistico.

Il modello probabilistico anziché valutare la distanza tra una query ed un documento andando a definire l'angolo che li distanzia, cambia la prospettiva e studia una distribuzione di probabilità. Ovvero quanto un documento è probabile che sia rilevante secondo un'interrogazione.

Nel modello probabilistico cerchiamo di rappresentare l'incertezza e il costo della decisione (prendere o meno un documento) dove l'IRS deve decidere se recuperare il documento o meno.

5.1 Eventi ed errori

Abbiamo un sistema di information retrieval con un insieme di documenti:

- Rilevanti
- Non rilevanti

Fare degli errori, rilevanti un non rilevante (e viceversa), porta a degli errori:

- Rumore: quando recupero un documento non rilevante
- Perdita di informazione: non ho recuperato un documento rilevante

Questo modello fa delle assunzioni:

- Tutti gli eventi che possono succedere sono randomici, non sono dipendenti uno dall'altro
- Siamo interessati all'evento che porta al recupero di un documento

Una volta che ho recuperato il documento devo scegliere se è rilevante o meno. Anche questi sono due eventi:

- Eventi A : rilevante
- Evento $\neg A$: non rilevante

Attenzione che fondamentalmente è binaria come scelta, perchè o è rilevante o meno. Abbiamo ancora l'assunzione di mutua esclusione che è una semplificazione, ma rende la rappresentazione del modello meno generica.

Questo porta a due tipi di errore, si possono esprimere con delle probabilità.

- Errore1: rumore, ho recuperato qualcosa di non rilevante
- Errore2: non ho recuperato un documento rilevante

5.2 Rischio

Combinato a questo concetto di errore abbiamo il concetto di rischio, i rischi sono legati a probabilità:

- Rischio in favore di A, rischio che ho considerando il documento rilevante: $R(A|B)$
- Rischio in sfavore di A: $R(\neg A|B)$

La decisione è vista come un problema di minimizzazione del rischio.

Il rischio è legato al costo, lo devo andare a definire come costo di incertezza o di perdita informativa.

- Il costo di incertezza: costo relativo al rumore. È la mia scelta in favore di A ma in realtà A non era rilevante. Mi dice il costo di aver fatto la scelta sbagliata in favore di A: $c(\neg A, A)$
- Costo di perdita di informazione: Il costo di perdita dell'informazione quando consideriamo un documento irrilevante mentre in realtà era rilevante: $c(A, \neg A)$

Il costo di aver fatto la scelta giusta in favore o sfavore di A sarà sempre minore di aver fatto la scelta sbagliata. Solitamente si considera la scelta corretta a costo 0.

$$c(\neg A, \neg A) < c(A, \neg A) \quad c(A, A) < c(\neg A, A)$$

Dobbiamo definire una probabilità in favore di A accettabile, quando supera il costo che avrei nel caso in cui supera il costo che avrei facendo una scelta sbagliata. Il rapporto tra il costo del rumore fratto il costo totale è il costo che devo superare. La probabilità di fare una scelta in favore di A deve essere superiore al costo che ho avendo fatto quella scelta:

$$Pr(A|B) > c \rightarrow c = c(\neg A, A) / (c(\neg A, A) + c(A, \neg A))$$

Possiamo dire che il rischio in favore di A è il costo di aver fatto la scelta giusta dato la probabilità che sia effettivamente rilevante più il costo che ho in favore dell'incertezza per il costo che sia effettivamente rilevante:

$$R(A|B) = c(A, A)Pr(A|B) + c(\neg A, A)Pr(\neg A|B)$$

Possiamo anche modellare il rischio in favore di non A (ovvero non scegliere A):

$$R(\neg A|B) = c(A, \neg A)Pr(A|B) + c(\neg A, \neg A)Pr(\neg A|B)$$

Dato il teorema di Bayes possiamo riscrivere la probabilità in questi termini:

$$Pr(A|B) = \frac{Pr(B|A)pr(A)}{Pr(B)}$$

$$Pr(\neg A|B) = \frac{Pr(B|\neg A)pr(\neg A)}{Pr(B)}$$

Visto che sono mutuamente esclusivi possiamo riscrivere i due rischi in questo modo:

$$R(A|B) = c(A, A)Pr(A|B) + c(\neg A, A)(1 - Pr(A|B))$$

$$R(\neg A|B) = c(A, \neg A)Pr(A|B) + c(\neg A, \neg A)(1 - Pr(A|B))$$

Questo mi permette di dire che il rischio in favore di A è minore del rischio in favore di non A per la probabilità di A dato B:

$$R(A|B) < R(\neg A|B)Pr(A|B)$$

se e solo se la probabilità di A dato B supera il costo espresso:

$$Pr(A|B) > c$$

e il costo espresso è:

$$c = \frac{(c(\neg A, A) - c(\neg A, \neg A))}{(c(\neg A, A) - c(\neg A, \neg A)) + (c(A, \neg A) - c(A, A))}$$

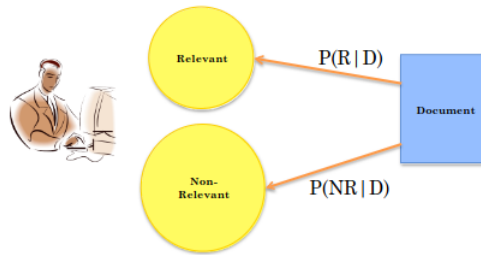
Sostanzialmente se la probabilità in favore di A supera il costo allora il rischio è minore (il gioco vale la candela).

Fondamentalmente devo procedere ad una stima di A dato B per determinare se è maggiore o minore del costo in termini di information lost e imprecision.

Come facciamo a stimare questa probabilità? dipende anche dal tipo di query, quindi devo considerare quanto è rilevante il documento secondo una query.

5.3 Probabilistic Retrieval Model

Il processo di recupero di un documento è considerato un processo di ordinamento dei documenti, per ogni query dobbiamo considerare tutto i documenti come risposta, potenzialmente l'intera collezione. Ma facendo un processo di ordinamento possiamo prendere solo quelli più rilevanti.



$$\text{Bayes' Rule } P(R | D) = \frac{P(D|R)P(R)}{P(D)}$$

$$P(R|D) = \frac{P(D|R)P(R)}{P(D)} P(NR|D) = \frac{P(D|NR)P(NR)}{P(D)}$$

La rilevanza dipende, allora D viene classificato come rilevante:

$$P(D|R)P(R) > P(D|NR)P(NR)$$

5.3.1 Stima rilevanza documento

Per stimare la rilevanza di un documento dobbiamo andare a vedere i termini. Quindi cosa vuol dire che devo andare a definire la probabilità di D dato R, vuol dire considerare la produttoria di tutti le probabilità di tutti i termini (d_n) nel documento dato R (la rilevanza) o dato non R (la non rilevanza).

$$P(D|R) = \prod_{i=1}^t P(d_i|R)$$
$$P(D|NR) = \prod_{i=1}^t P(d_i|NR)$$

La stima di questa probabilità viene fatta tramite il likelihood ratio, che prendiamo come vero:

- s_i è la probabilità che il termine i compaia nei documenti non rilevanti
- p_i è la probabilità che il termine i compaia nei documenti rilevanti
- N è il numero totale dei documenti non rilevanti
- n_i è il numero dei documenti non rilevanti che contengono il termine
- r_i è il numero dei documenti non rilevanti che contengono il termine R : il numero totale di documenti rilevanti

$$\frac{P(D|R)}{P(D|NR)} = \prod_{i:d_i=0} \frac{p_i}{s_i} \cdot \prod_{i:d_i=1} \frac{1-p_i}{1-s_i} = \sum_{i:d_i=1} \log \frac{p_i(1-s_i)}{s_i(1-p_i)}$$
$$\sum_{i:d_i=q_i=1} \log \frac{(r_i + 0,5)/(R - r_i + 0,5)}{(n_i - r_i + 0,5)/(N - n_i - R + r_i + 0,5)}$$

Questa stima di probabilità dipende dalla tabella di contingenza, vado a vedere un campione di documenti, preso questo campione vado a vedere il termine i . Dato il fatto che il termine compare o no, posso all'interno del campione posso definire:

- r_i : tutti i documenti che contengono il termine i
- n_i : tutti i documenti sia rilevanti che non che contengono il termine i
- N : conosco il numero totale di documenti
- R : Conosco il numero totale di documenti rilevanti

	A	$\neg A$	
$x_i = 1$	r_i	$n_i - r_i$	n_i
$x_i = 0$	$R - r_i$	$N - n_i - R + r_i$	$N - n_i$
	R	$N - R$	N

5.3.2 BM25

Su questa stima si appoggia il fattore BM25, che è il fattore di retrieval utilizzato per fare lo scoring, per calcolare la probabilità che un documento sia rilevante rispetto alla query. Ha la prima parte che è la stima di probabilità e la seconda parte è una pesatura:

- $k_1 k_2 b$: sono i parametri di tuning
- dl : la lunghezza del documento
- $avgdl$: la lunghezza media dei documenti nel dataset
- qf : la frequenza del termine nei termini della query

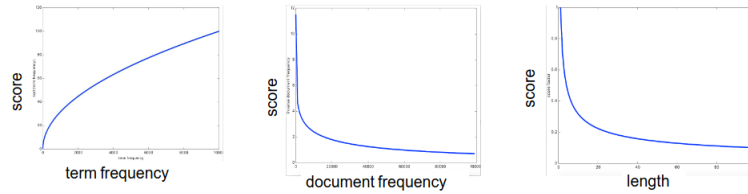
$$R(q, D) = \sum_{i \in Q} \log \frac{(r_i + 0.5)/(R - r_i + 0.5)}{(n_i - r_i + 0.5)/(N - n_i - R + r_i + 0.5)} \cdot \frac{(k_i + 1)f_i}{K + f_i} \cdot \frac{(k_2 + 1)qf_i}{k_2 + qf_i}$$

$$K = k_1((1 - b) + b \cdot \frac{dl}{avgdl})$$

Di base va a stimare la probabilità che un documento sia rilevante per una query a partire da un likelihood ratio:

$$bm25(d) = \sum_{t \in q, f_t, d > 0} \log(1 + \frac{N - df_t + 0.5}{df_t + 0.5}) \cdot \frac{f_{t,d}}{f_{t,d} + k \cdot (1 - b + b \frac{1(d)}{avgdl})}$$

Da oltre un particolare focus sulla frequenza dei termini, perchè dobbiamo cercare usando queste frequenze di pesare il termine nel documento o nella query, così come facevamo con il tfidf. Ovviamente la rilevanza del documento deve aumentare se il termine della query è presente tanto nel documento. Mentre daltra parte se il termine è tanto presente negli altri documenti la rilevanza va diminuita.



5.3.3 Cord-factor

Non abbiamo considerato la lunghezza del documento. Un altro fattore considerato in questo modello è quello del cord-factor, dice che è importante la frequenza ma anche la concomitanza dei termini cercati. $\text{coord}(q,d)$ è un fattore di punteggio basato sul numero di termini della query nel documento specificato. In genere, un documento che contiene un numero maggiore di termini della query riceverà un punteggio più alto rispetto a un altro documento con un numero minore di termini della query.

5.3.4 TF/IDF

Come funziona il TF/IDF all'interno del modello probabilistico? La rilevanza è binaria:

- Il documento è rilevante rispetto alla query
- Il documento non è rilevante

Come fa invece la valutazione dei pesi? Dobbiamo pesare i termini di un documento. Per la stima dei pesi considera la sommatoria di tutte le probabilità dei termini nel documento rispetto al fatto che sia rilevante o no. Riscrivo il teorema considerando la frequenza dei termini della query contenuti nel documento:

Possiamo stimare anche la probabilità di un indice in un documento:

$$P(R = 1|d, q) = \frac{P(d|R = 1, q)P(R = 1|q)}{p(d|q)}$$

dove:

- $P(A|B)$ = probabilità di A dato B
- R è la rilevanza (1 o 0)
- d è il documento
- q è la query

Se non ho tutti i documenti posso fare una stima, possiamo stimare qual'è la probabilità di frequenza uno dato che non sia rilevante, dato che sia rilevante. Non consideriamo la frequenza maggiore di 1:

$P(A|B)$ = probability of A given B
 R = relevancy (1/0)
 d = document
 q = query

$$P(R = 1|d, q) =$$

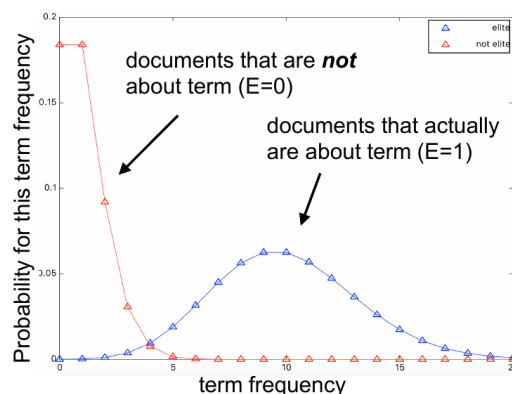
	R=1	R=0
d1	0.1	0.9
d2	0.2	0.8
d3	0.7	0.3
...	...	...

for each query q!

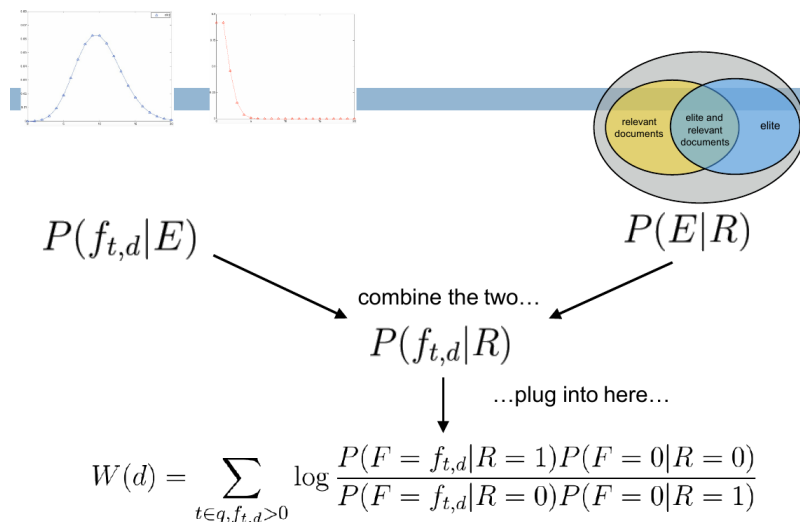
Eliteness

Oltre a considerare la frequenza BM25 considera un altro fattore, l'eliteness. È qualcosa in più della frequenza. Si intende che un termine anche se appare tante volte potrebbe non essere il focus del documento.

La stima di eliteness dipende dal fatto che il termine centri o meno con il documento. Mettiamo il caso che il documento parli di altro, allora mi sta bene che ci sia una bassa frequenza, se però quella parola c'è spesso dobbiamo abbattere l'eliteness. Consideriamo invece il caso in cui il documento centri, se la frequenza è bassa non posso farci niente, ma appena cresce la frequenza l'eliteness deve aumentare in maniera esponenziale, fino ad un certo punto, perché l'esagerata frequenza non la posso considerare.



Questa stima di elliteness viene legata alla stima di frequenza. Va a quindi a fondersi con la stima di pesatura legata alla frequenza Questo ci porta a definire una pesatura che è così fatta:

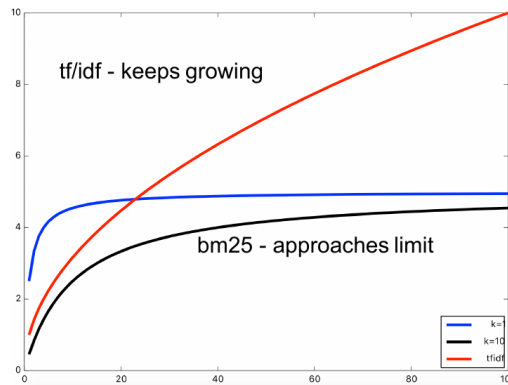


Tf saturation curve

Il fattore di eliteness è dato dal fattore k chiamato fattore di saturazione, dipende dai casi ma solitamente ci viene dato, il fattore di saturazione serve per saturare il TF. Quello

classico aumenta sempre più (più aumenta la frequenza più aumenta), se cerco di saturare la frequenza creo una curva che raggiunge un asintoto prima:

$$w(t) = \frac{t_{t,d}}{f_{t,d} + k}$$



IDF comparison

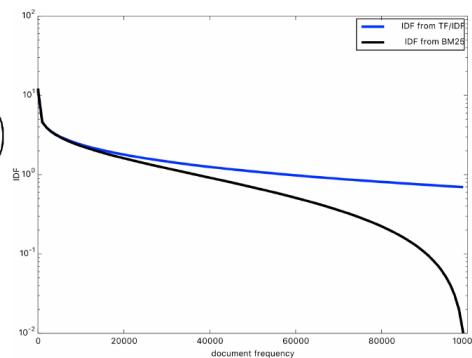
IDF anche in BM25 viene legato anche lui al concetto del numero di documenti e al numero di documenti che contengono il termine. In BM25 l'IDF è più pesante, ad un certo punto se il termine è dappertutto non lo devo proprio considerare, abbasso lo score legato ad un termine troppo frequente:

BM25

$$w^{IDF}(t) = \log \left(\frac{N - df_t + 0.5}{df_t + 0.5} + 1 \right)$$

TF/IDF

$$w^{IDF}(t) = \log \left(\frac{N + 1}{df_t + 1} + 1 \right)$$



Poisson distribution

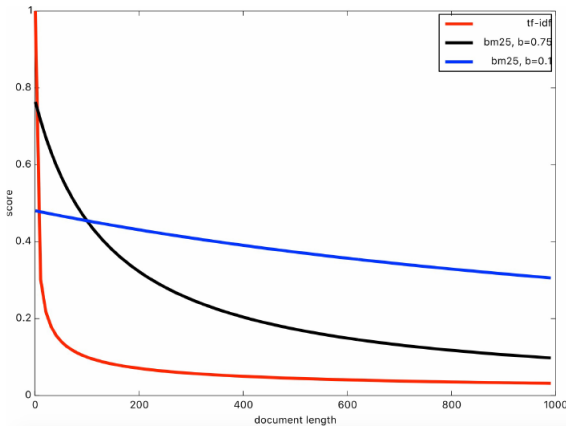
Non abbiamo ancora visto la distribuzione sulla lunghezza del documento. É sempre legato alla frequenza.

- b è il parametro di lunghezza del documento (lungo, corto, medio...)
- $f_{t,d}$ è la frequenza del termine nel documento
- k è il parametro di saturazione
- $l(d)$ è il numero di token nel documento

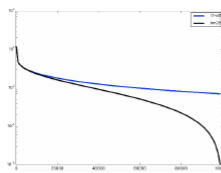
- avgdl è la lunghezza media dei documenti

Più aumenta la lunghezza del documento più vorrei che l'incidenza del documento si abbassasse. Ci permette di andare ad ammorbidire la crescita del TF nel modello BM25:

$$norm(t) = \frac{f_{t,d}}{f_{t,d} + k \cdot \left(1 - b + b \cdot \frac{l(d)}{avgdl}\right)}$$

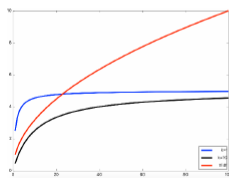


Quindi abbiamo TF, IDF ed il peso modulato secondo la lunghezza del documento:



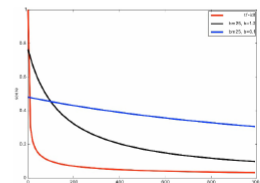
idf - how popular
is the term in the
corpus?

$$bm25(d) = \sum_{t \in q, f_{t,d} > 0} \log \left(1 + \frac{N - df_t + 0.5}{df_t + 0.5} \right) \cdot \frac{f_{t,d}}{f_{t,d} + k \cdot \left(1 - b + b \cdot \frac{l(d)}{avgdl}\right)}$$



saturation
curve - limit
influence of tf
on the score

length weighing -
tweak influence of
document length



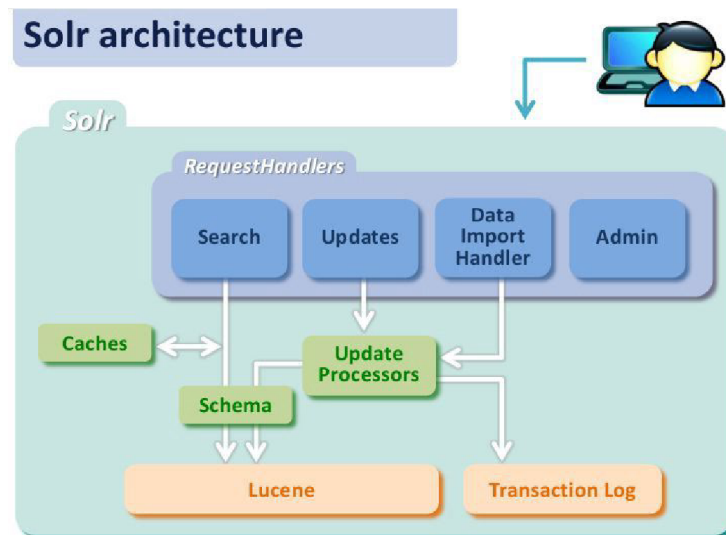
Chapter 6

Sorl

Anche solr è basato sul concetto di indice inverso. Viene valutato con Recall e Precision e si basa sul modello BM25.

Lavora su qualunque tipo di documento testuale, noi lo vedremo tramite linea di comando e pagina web. La vera potenza è usarlo tramite API, perchè ci permette di inglobare il concetto di text mining in applicazione web. I risultati posso esportarlo in qualsiasi formato.

Architetture:



La logica del modello: parte da query espresse con logica booleana e l'aspetto booleano viene fatto per filtrare le query dell'utente poi c'è un modello vettoriale esteso al probabilistico che implementa BM25. Lo score poi è il calcolo della distanza, il modello probabilistico mi serve per calcolare i pesi degli indici.

6.1 Score calculation

Si appoggia sempre ai concetti:

- TF: frequenza del termine nel documento
- $IDF(t)^2$
- Boost: se ho dato un fattore di spinta a un termine nella mia ricerca
- Coord: numero dei termini che appaiono nei documenti e che sono nella query / numero di termini nella query
- Query normalization: calcolato con i fattori di boost legati ai termini nella query

Cos'è un fattore di spinta? Innanzitutto solr è molto basato sul concetto di schema, ad esempio dobbiamo avere nella base documentale per ogni documento dei field (titolo e description). Nella query possiamo mettere dei boos (moltiplicare per n) ad un termine o un documento.

Il calcolo dello score prende in considerazione alcune cose più articolate rispetto al calcolo di score del modello vettoriale. Viene calcolato con questa funzione matematica più complicata in cui però vengono presi in considerazione dei parametri diversi:

- **Coord:** va a mediare la frequenza con la presenza di tutti i termini cercati, è più rilevante che ci siano tutti i termini cercati rispetto ad un solo termine più frequente. Questo fattore va a mitigare il problema di considerare solo la frequenza
- **Boost:** quanto lato utente voglio dare maggior rilievo ad un termine preciso, do un fattore moltiplicativo ad un termine che viene poi considerato nel calcolo dei vari pesi

Architetture:

$$\text{Score}(q,d) = \sum_{t \text{ in } q} \left(\text{tf}(t \text{ in } d) \cdot \text{idf}(t)^2 \cdot t.\text{getBoost}() \cdot \text{norm}(t,d) \right) \cdot \text{coord}(q,d) \cdot \text{queryNorm}(q)$$

In realtà vedremo che ci sono diverse modalità per lavorare con Sorl, in alcune modalità di ricerca possiamo usare il bm25 con i fattori di saturazione per il tf e per la lunghezza del documento.

6.2 Schema

Fondamentalmente sorl non ha uno schema definito a cui devo conformarmi quando devo creare un repository di documenti, i documenti possono avere il proprio schema con la loro struttura però non è obbligatorio che gli altri documenti rispettino lo stesso schema. Nella definizione dello schema posso definire i campi dello schema:

- Fields
- Data types
- Analysers

Posso definire anche due tipi di campi:

- Dinamic fields
- Copy fields

6.2.1 Field Types

Se devo definire dei campi li definisco con i classici tipi int, float, long, double. Per il text si usa text con il tipo di lingua. Non è potente quanto Oracle nella trattazione dei documenti in lingua asiatica.

Attributi field comuni:

- Indexed

- Stored
- Type
- Multivalued

Esempio:

```
1 <field name="id" type="string" indexed="true"
2 stored="true" required="true" multiValued="
3 false" />
```

6.2.2 Dynamic Fields

Quando definisco un campo posso farlo con delle wild cards. Con un pattern, voglio che nel documento ci siano dei campi ma non ti dico il nome ma ti dico il pattern che devono rispettare:

```
1 <dynamicField name="*_i" type="tint" indexed="true" stored="true"/>
```

Non vado a rappresentare un unico campo all'interno dello schema ma ne rappresento diversi con delle wild card.

6.3 Indexing

Si può fare da linea di comando o da interfaccia grafica tramite dashboard web.

Vediamo un esempio di uso delle API in un applicazione web:

```
1 curl http://localhost:8983/solr/my_collection/update
2 -H "Content-Type: text/xml" --data-binary '
3 <add>
4 <doc>
5 <field name="id">012ab1</field>
6 <field name="authors">Patrick Eagar</field>
7 <field name="subject">Sports</field>
8 <field name="dd">796.35</field>
9 <field name="isbn">0002166313</field>
10 <field name="yearpub">1982</field>
11 <field name="publisher">Collins</field>
12 </doc>
13 </add>'
```

6.4 Query Parsers

La query testuale che andiamo a vedere utilizza determinate strategie di interrogazione che si basano su un motore chiamato Lucene.

6.5 Hello World!

Seguiamo il tutorial ufficiale [sorl](#).

Facendolo in cloud possiamo impostare delle repliche di [sorl](#) su vari nodi, ovviamente dobbiamo specificare le porte a cui rispondono.

```
1 bin/solr start -e cloud
```

Per creare una nuova collection, specificando che abbiamo due nodi:

```
1 bin/solr create -c <yourCollection> -s 2 -rf 2
```

Attualmente è una collection vuota, come carico dei documenti, è molto semplice a livello di riga di comando:

```
1 bin/solr post -c techproducts example/exampledocs/*
```

Prende tutti i documenti dalla cartella `example/exampledocs`. È il comando `post` che mette dentro i documenti.

con una collection posso:

- Fare analisi
- Caricare documenti (facciamolo da riga di comando)
- Vedere lo schem: di base lui è andato a popolare lo schema rispetto a quello che ha trovato, se c'erano degli xml o csv ha usato i tag per creare la struttura

6.5.1 Query

Le più facili le vediamo in dashboard e le più complicate tramite il comando `CURL`.

Nella prima query recuperiamo tutti i documenti con `"*:"` (select *), possiamo anche scegliere il formato in cui vengono restituiti i documenti:

```
Request-Handler (qt)
/select
--- common ---
q
*:*
fq
sort
start, rows
0 10
fl
df
Raw Query Parameters
key1=val1&key2=val2
wt
json
[indent off]
[debugQuery]
[dismax]

http://localhost:8983/solr/techproducts/select?q=*:*
{
  "responseHeader": {
    "zkConnected": true,
    "status": 0,
    "QTime": 25,
    "params": {
      "q": "*:*",
      "_": "1501179300840"
    }
  },
  "response": {
    "numFound": 52, "start": 0, "maxScore": 1.0, "docs": [
      {
        "id": "0812521390",
        "cat": ["book"],
        "name": "The Black Company",
        "price": 6.99,
        "price_c": "6.99,USD",
        "inStock": false,
        "author": "Glen Cook",
        "author_s": "Glen Cook",
        "series_t": "The Chronicles of The Black Company",
        "sequence_i": 1,
        "genre_s": "fantasy",
        "_version_": 1574100232462925824,
        "price_c__l_ns": 699,
      },
      {
        "id": "0441385532",
        "cat": ["book"],
        "name": "Jhereg",
        "price": 7.95,
        "price_c": "7.95,USD",
        "inStock": false,
      }
    ]
  }
}
```

Con `CURL` sarebbe:

```
1 curl "http://localhost:8983/solr/techproducts/select?indent=on&q=**"
```

Proviamo a cercare tutti i documenti che contengono il termine foundation, se lo facciamo solo mettendo foundation non trova niente, perchè se non ho trovato copy field lui va a fare una ricerca specifica, dovrei dirgli io in che campi andare a cercare ad esempio "cat:electronics". Chiaramente non è la cosa più comoda del mondo quindi conviene creare i copy field. Vediamo un esempio di chiamata CURL, indicando l'IP di risposta di solr e la collection:

```
1 curl "http://localhost:8983/solr/techproducts/select?q=foundation&fl=id"
```

6.5.2 Copy Field

Li posso creare a livello di chiamata, crea un copy field che si chiama asterisco e ha come destinazione un testo:

```
1 curl -X POST -H 'Content-type:application/json' --data-binary
   '{ "add-copy-field" :
2  { "source": "*", "dest": "_text_" } }' http://localhost:8983/solr/films/schema
```

Server per dire in che campi fare la query, devo dirgli qual'è la sorgente da cui vado a prelevare i documenti durante l'interrogazione. Devo comunque reindicizzare tutti i documenti rifacendo il post.

6.5.3 Parametri

Possiamo definire qual'è il parser (il motore da usare per fare l'interrogazione). La q è la query. La sq è per fare della cache per delle sotto query.

fl ci dice qual'è la proiezione. df è il parametro di recupero, i testi. L'output lo definiamo con wt.

Parameter	Description	Default
defType	This selects the query parser, with the three most common being 1) lucene, 2) dismax and 3) edismax.	lucene
q	This is where you enter search terms	*.*
fq	This is used to set a filter query which creates a cache of potential sub-queries. So if your user will look at more granular information it helps with speed to identify the fields ahead of time so the results are ready in cache.	none
sort	Here you enter how you would like results stored with common options being asc or desc for ascending and descending.	desc
start,rows	The start,rows parameters are used like a search in Google that provides the top 10 results by default and allows you to resubmit the query to find the next 10 results. So think of it as a way to paginate query results. The 0 starts at the first record and 10 shows 10 records, or rows.	0,10

Parameter	Description	Default
fl	The field list parameter limits the results to a specific list of fields. In order for them to show up the schema must have one of two settings for the field: 1) stored="true" or 2) docValues="true". Multiple fields can be selected and are separated by commas or spaces. You can also return the score given as a measure of the relevance of the search results. A * shows all stored fields.	*
df	In the default field parameter you could enter a specific field you would like Solr to search. In our case, the default search field is the Copy Field we created called _text_ .	_text_
Raw Query Parameters	The Raw Query Parameters section is for advanced use, like for query debugging.	none
wt	The wt parameter selects one of 6 popular Response Writers with output customized for 1) json, 2) xml, 3) python, 4) ruby, 5) php, 6) csv. There are another 16 that can be input from the command line.	json

6.5.4 And/Or

Solr cerca di implementare anche delle query di tipo Booleano, quindi semplicemente possiamo usare la logica booleana per fare le query. Se voglio i documenti che hanno entrambi i documenti uso il +, mentre con curl devo usare un carattere speciale per il + che è %2B:

```
1 curl "http://localhost:8983/solr/techproducts/select?q=%2Belectronics%20%2Bmusic"
```

Se invece voglio documenti che contengono electronic ma non contengono music dovrei usare electronics -music nella UI, mentre con curl:

```
1 curl "http://localhost:8983/solr/techproducts/select?q=%2Belectronics+-music"
```

6.5.5 Faceting

Creo una nuova collezione:

```
1 bin/solr create -c films -s 2 -rf 2
```

Creo il copy field, lo abbiamo fatto da interfaccia grafica.

Serve ad esempio recuperare i film divisi per genere, quanti sono i documenti di questo genere?

```
1 curl "http://localhost:8983/solr/films/select?q=*:*&rows=0&facet=true&facet.field=genre"
```

Specificando rows a zero gli sto dicendo di non restituirmi i documenti ma solo la statistica.

Si possono fare dei raggruppamenti e poi all'interno fare anche dei range di raggruppamento, raggruppa i film in base al genere ma raggruppa da adesso a 25 anni fa. Mi restituisce il numero di film per anno:

```
1 curl "http://localhost:8983/solr/films/select?q=*:*&rows=0\
2 &facet=true\
3 &facet.range=initial_release_date\
4 &facet.range.start=NOW/YEAR-25YEAR\
```

```

5 &facet.range.end=NOW\
6 &facet.range.gap=%2B1YEAR"

```

6.5.6 Pivot

Creo il gruppo e poi il sottogruppo, prima raggruppa per genere e poi per regista:

```

1 curl "http://localhost:8983/solr/films/select?q=\*:*&rows=0&facet=on
    facet.pivot=genre_str,directed_by_str"

```

6.6 Using a Solr Analyzer for Text Analysis

Come recupero i documenti dipende dal tipo di analyzer, ci sono diversi tipi di analyzer in solr: In base ai diversi tipi di analyzer cambiano le strategie di tokenizzazione.

Analyzer Name	Tokenizer, Filter(s)
Classic Analyzer	•Classic Tokenizer •Classic Filter •Lowercase Filter •Stop Filter
Keyword Analyzer	•Creates a single token from the entire stream of text •Well suited for id fields and structured data
Simple Analyzer	•Letter Tokenizer •Lowercase Filter
Standard Analyzer	•Standard Tokenizer •Standard Filter •Lowercase Filter •Stop Filter
Stop Analyzer	•Letter Tokenizer •Lowercase Filter •Stop Filter
Whitespace Analyzer	•Whitespace Tokenizer

6.6.1 Classic Analyzer

Usa il classic tokenizer, come filtro usa il Classic Filter, lowercase filter e lo stop filter.

Se mettiamo come input: "I've gotten 5 E-mails from joe@example.com." otteniamo come output: "i", "ve", "gotten", "5", "e", "mails", "from" "joe@example.com".

6.6.2 Keyword Analyzer

Mi permette di considerare delle keyword insieme anche se sono di due termine, come ad esempio "Washington DC".

6.6.3 Simple Analyzer

Indicizzazione full text ma con tutte le parole scritte in minuscolo. Ad esempio se mettiamo come input: "The MLB is a long-lived monopoly. The most recent challenge went to the U.S. Supreme Court in 1972", otteniamo come input: ""The", "mlb", "is", "a", "long", "lived", "monopoly", "the", "most", "recent", "challenge", "went", "to", "the", "u", "s", "supreme", "court", "in"".

6.6.4 Standard Analyzer

Ci permette di non considerare delle stop word.

Se mettiamo in input: "They call it N.Y.C., NY, New York, Number-1 City or the Big Apple."

Otteniamo in output: ""call", "n.", "y.", "c", "ny", "new", "york", "number", "1", "city", "big", "apple"".

6.6.5 Stop Analyzer

Ci permette di specificare noi le stop word da evitare durante la tokenizzazione. Per esempio se mettiamo nel file stopwords.txt le parole "the", "a", dato l'input: "The green fox jumped over a log. Wait, a green fox?" otteniamo in input: ""green", "fox", "jumped", "over", "log", "wait", "green", "fox"".

6.6.6 Space Analyzer

Ci permette di specificare noi la spaziatura.

6.7 Tokenizzatore

Nella parte di analisi di possiamo andare a vedere come il motore preformerebbe rispetto a tipi di interrogazioni che vogliamo fare. Vediamo che cambiando il tipo del testo cambiano i tipi di tokenizzatori che possiamo usare, ci fa vedere evidenziati i token che matcherebbero con la query.

Quindi la cosa potente di sorl è che quando facciamo la query in base al tipo di dato che abbiamo definito possiamo personalizzare la tokenizzazione, ad esempio se vogliamo usare lo stemming dobbiamo usare una tokenizzazione rispetto alla lingua.

Chapter 7

Question Answering

L'evoluzione naturale della ricerca che stiamo facendo fino ad adesso è il question-answering. Quando facciamo un'interrogazione con oracle o sorl è che lui ci recupera un documento. L'evoluzione quindi è che il sistema mi dia una risposta puntuale alla mia richiesta, non che trovi il documento.

Sono tutti i chatbot (anche vocali in realtà), che interrogati danno una risposta. Si basano sul concetto di information retrieval ma hanno anche qualcosa di diverso, perchè oltre a trovare il documento deve dare una risposta. Questo lo vedremo a livello teorico e poi a livello più pratico. Il primo step è quello che abbiamo visto adesso, dobbiamo sempre avere una base documentale. Poi per costruire la risposta posso farlo tramite regole o tecniche di ML. Lo vedremo con due sistemi:

- Dialog Flow di Google, che è la piattaforma usata per implementare google assistant/home
- ChatGPT, anche ChatGPT è un motore che è legato al concetto di documento vettoriale, chiaramente lì il patrimonio documentale è molto ampio e le logiche con cui genero la risposta sono diverse

Anche in questo caso abbiamo problemi di risposte non veritiere. Modelliamo questo con dei costi:

- Rumore: mi da risposte sbagliate
- Perdita informativa

Prende spunto da due mondi:

- Information retrieval
- Information extraction: dovrà essere la base per generare la risposta

L'utente fa delle domande e vuole una risposta precisa, non una lista di documenti. La differenza tra QA e IF sono:

- Come formula la domanda: faccio delle interrogazioni in linguaggio naturale che deve essere capita
- La risposta è generalmente estratta dal testo

L'information extraction invece è volto ad essere la base di estrazione cumulativa su cui creare la risposta finale. Il principale problema del QA è che mi da una risposta secca, quindi se me la da sbagliata non posso andare a vedere i documenti da cui ha trovato la risposta.

Un sistema di QA in breve si distingue per queste caratteristiche:

- In input prende una query espressa in linguaggio naturale
- L'output deve essere una risposta in linguaggio naturale

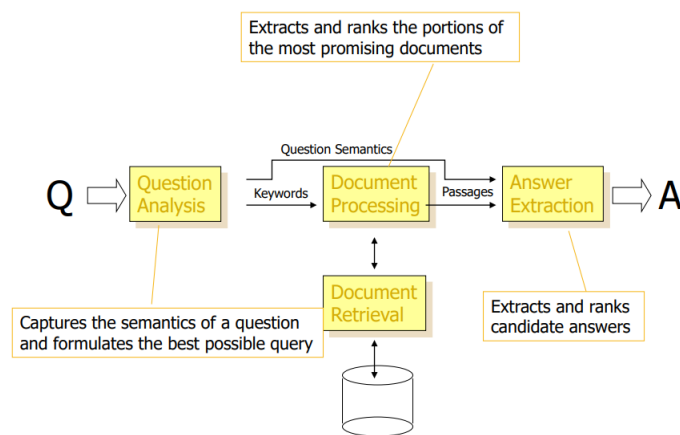
La prima cosa che deve capire è che risposta deve dare, ad esempio un numero, una data, una persona, una lista etc. Già la domanda in se aiuta il sistema di QA a formulare la risposta.

Il problema principale è quanto è buona la risposta che mi da, misurare l'incertezza. Il concetto di buono e cattivo dipende, sicuramente è legato alla precisione ma è anche legato a come è formulata la risposta perchè potrebbe essere formulata male.

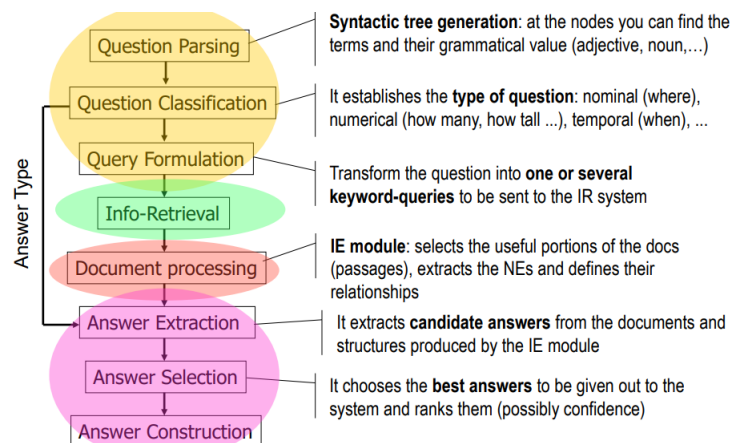
7.1 Architettura di un QA

Di base tutte hanno questi moduli, alcuni sono legati a sistemi di regole altri a ML:

- Analisi della domanda: prova a capire la natura della domanda
- Retrieval dei documenti: riprende il discorso che abbiamo fatto fino ad adesso, questo è un problema molto discusso, in quanto al momento chatGPT fa retrieval su tutta la base documentale, non sempre è necessario. Potremmo voler creare dei bot con una knowledge base specifica di dominio. Prende i documenti più rilevanti.
- Document processing: per analizzare il documento che ho preso, estrarre dal documento la parte rilevante
- Estrazione della risposta: estrarre la risposta dal documento

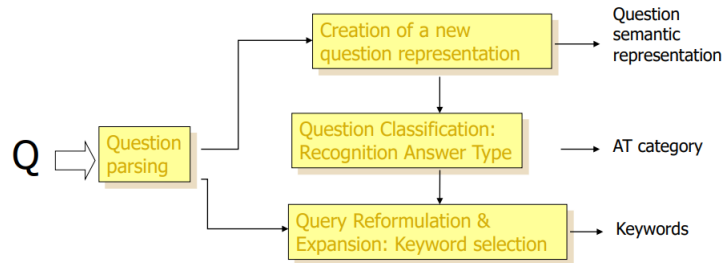


Devo avere una generazione dell'albero sintattico di quello che ha detto l'utente, devo quindi fare una classificazione della domanda per capire di che tipo sarà la risposta. Non è detto che la query dell'utente sia sufficiente, non è detto che data la query io sia in grado di trovare i documenti di cui ha bisogno, potrei espandere la query. Devo estrarre dai documenti solo la porzione interessante. Poi selezionare la risposta migliore tra le candidate. Poi ricostruisco una risposta che sia in linguaggio naturale.



7.1.1 Question analysis

Il primo modulo si focalizza sulla domanda, è un insieme di moduli che vanno a caratterizzare la domanda che l'utente ha fatto.



Ci sono diverse tecniche per farlo, l'obiettivo è capire che tipo di domanda è stata fatta:

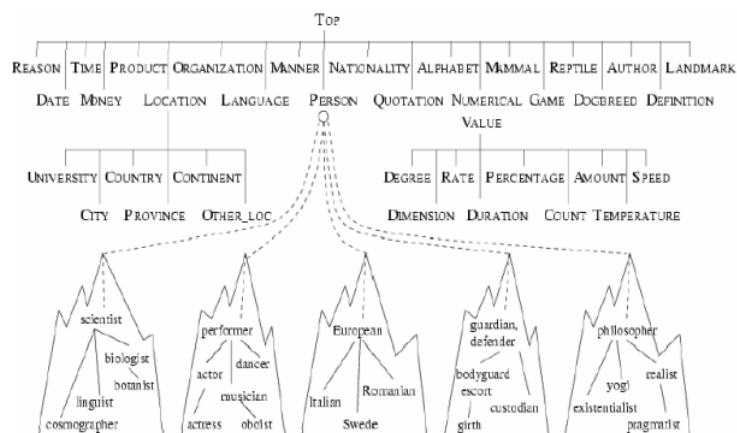
- Knowledge engineering: AT detection
- Machine Learning

AT Detection

Intuitivamente devo capire la radice della domanda. Quindi a seconda dello stem della domanda:

- WHY: ragione
- When: data
- Who: persona

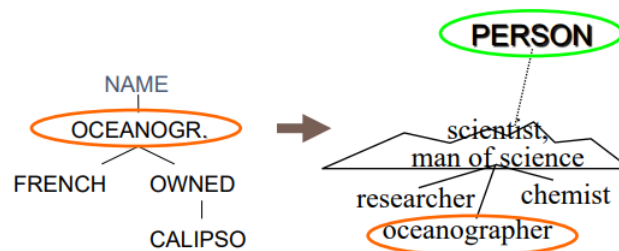
Potrebbe portare delle ambiguità, perchè potremmo chiedere di una persona esprimendo la domanda come un cosa "What was the name of Titanic's captain?". Per disambiguare la possibile risposta, una delle tecniche più utilizzate è quella che sfrutta un'ontologia di concetti con i quali posso andare a capire l'answer-type. Sono strutture informative in cui ho dei concetti dizionario e delle relazioni padre-figlio:



Questa tassonomia di concetti posso usarla per disambiguare lo stem. La detecion dell'anwere type funziona così:

- Parsing della domanda: rimuovere le stop word
- Delle parti rimanenti dovrei trovare i termini più significativi per rappresentare la domanda
- Uso la tassonomia per trovare l'answer type

Creo un albero sintattico della domanda che mi permette di individuare dei termini della tassonomia di riferimento, dopo aver mappato sulla tassonomia posso risalirla per capire il contesto. Ad esempio "What is the name of the French oceanographer who owned Calypso?":



Statisticamente non è necessario usare tutta la tassonomia, ci bastano meno concetti.

7.1.2 Query formulation

Ci si appoggia sulle named entity, sono parole che specificano una particolare entità, ad esempio "Alessandro Manzoni" specifica una persona. La domanda può essere approssimata con un insieme di parole che sono rilevanti. Le keywords sono le non stop words quotes, le named entities e i termini meno frequenti. Poi ad esempio prendo tutte i modificatori delle entità che ho trovato, prendo tutte le altre entità, i verbi e tutto quello che mi rimane se non è sufficiente. Ad esempio: What researcher discovered the vaccine against Hepatitis-B? Hepatitis-B, vaccine, discover, researcher What is the name of the French oceanographer who owned Calypso? Calypso, French, own, oceanographer What U.S. government agency registers trademarks? U.S., government, trademarks, register, agency What is the capital of Kosovo? Kosovo, capital

Dopo di che se non ho abbastanza keyword per tirare fuori delle risposte devo espandere la query:

- Prendo i sinonimi
- Prendo un tesaurus
- Strategie di stemming

Alla fine di tutte le keywords che ho tirato fuori ne creo una composizione logica concatenandole in AND o OR. Chiaramente non è quello che ha detto l'utente, ma è quello che mi serve. (Hepatitis-B OR Hepatitis) AND (vaccine OR vaccines OR cure OR cures) AND (discover OR discovered OR discovers) AND (researcher OR scientist)

È utile riformulare la domanda per formulare la risposta, anche per usare lo stesso linguaggio dell'utente per rispondere così da usare termini a lui comuni.

L'ultimo step riguarda un aggiustamento delle keywords, perchè se alla fine dell'information retrieval non sono soddisfatto dei documenti che ho recuperato posso modificare le keywords per fare una query più o meno selettiva. Ci sono della fasi di allenamento del motore per allenarlo a domande tipiche da aspettarsi dall'utente. Per aiutare l'agente a dare delle risposte coerenti.

7.1.3 IR And Document Processing

Alla fine dell'identificazione dell'intento dell'utente utilizziamo la query per individuare i documenti che hanno più similarità con la query espressa. Tutti i documenti avranno una confidenza, ma prima dobbiamo capire qual'è la parte del documenti più interessante, la parte chiamata passage retrieval.

Passage Ranking

É interessante prendere le keyword e vedere in che posizione sono del documento:

- La stessa sequenza della query
- Stessa frase
- Distanza diverse

Infine faccio il ranking del passaggio basato su alcune metriche:

- Numero di parole della domanda trovate nello stesso ordine
- Numero di keywords mancanti nella finestra
- Numero di parole tra le keywords più distanti

Answer Extractoion

Una volta che ho trovato questi snippets dai documenti devo usarli per formulare la risposta stessa. Quindi in input in questo modulo ho:

- Gli snippets dei documenti
- Che tipo di risposta

Devo fare:

- Trovare le named entities
- Cerco le risposte candidate
- Ranking delle risposte candidate per creare la risposta finale

Solitamente si usano tecniche di ML che si appoggiano a questi parametri:

- Numero di termini nella frase
- Numero di termini nello stesso ordine nella frase

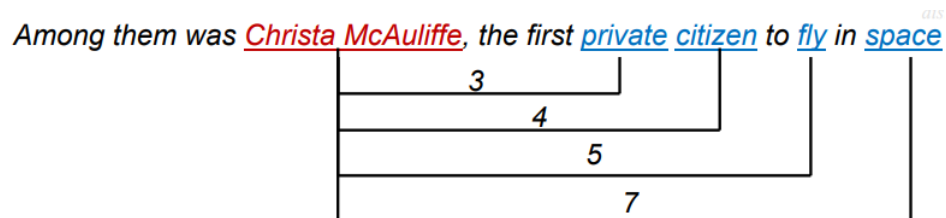
- numero di termini fuori dalla frase ma vicini
- numero di termini in ordine
- media tra il candidato e i termini della domanda

In fase di training posso addestrare il sistema dandogli delle regole empiriche, sono legate ad answer type precise, i pattern possono essere sfruttati perchè grazie ai pattern trovo le risposte giuste.

Altrimenti posso usare degli algoritmi sulla distanza, calcolare le distanza che ci sono data la query e le keywords trovo tutte le distanza tra le keywords della domanda e delle keywords del passaggio:

$$dist(w_k, Q, P_i) = \sqrt{\sum_{t=0}^{terms \in Q} dist(w_k, w_{Q_t}, P_i)^2}$$

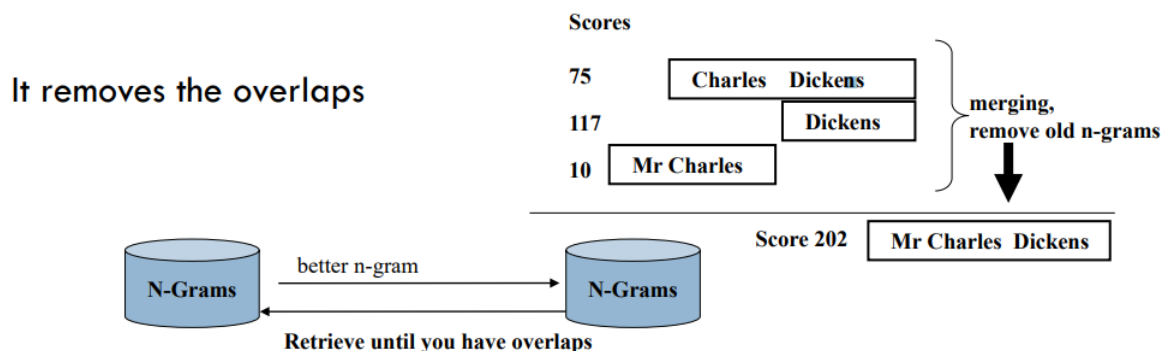
dove $dist(w_k, w_{Q_t}, P_i)$ è il numero minimo di parole che separa w_k da w_{Q_t} :



Se ho un ranking delle candidate risposte tendenzialmente prendo la prima, ma in realtà potrei fare delle tecniche migliori per fondere le risposte che si sovrappongono, facendo così mergio anche lo score, rendendo la candidata ancora più forte.

Ad esempio: "Who created the character of Scrooge?":

- Dickens: 117, Charles Dickens: 75, Disney: 72,
- Carl Banks: 54, Uncle: 31, Mr Charles: 10



7.2 Dialog Flow

L'obiettivo principale è quello di creare dispositivi chat bot o voice bot. Di base dobbiamo implementare la logica vista dal punto di vista teorico, abbiamo una domanda dell'utente, il sistema deve tirare fuori le keywords.

Dialog flow ES è una piattaforma che ci permette di creare in modo semplice il chatbot. Tutti devono capire cosa vuole l'utente, questo si chiama capirne l'intento, una critica fatta agli attuali sistemi di AI è che si occupano più di capire l'intento rispetto che a dare la risposta, in realtà perchè è fatto così il modello, è un calcolo della probabilità che data una domanda un documento possa aderire alla domanda. L'intento deve ricadere in certe caratteristiche predette a monte, quando l'utente parlerà con me devo catturare il suo intento.

- Welcome intent: c'è sempre un intent di saluto quando parlo con l'agente

Per addestrare il bot a capire gli intenti devo dargli le training phrases, quali sono le frasi che innescheranno il bot. Una volta catturato l'intento devo dirgli cosa rispondere, è un set di risposte a cui il bot attinge per dare l'idea di essere intelligente.

Sicuramente di default tra gli utenti troveremo il fallback intent, c'è tutto uno studio a livello di ricerca perchè il problema del bot è come gestire gli errori e le incomprensioni, vanno fatte quindi strategie di ricattura dell'utente. Non è facile sempre gestire il fallback, perchè l'utente può dire quello che vuole, la cosa più complicata è quando l'utente nella stessa frase fa più di una query.

Le cose che devo prevedere sono gli utenti per cui questo chatbot è stato progettato, a seconda dell'agente devo strutturare il flusso di dialogo, attualmente ci sono delle aziende in cui viene assunta gente solo per fare il dialog designer, disegnano il dialogo, che è un flusso di botta e risposta tra utente e agente che deve essere predefinito. Questo concetto di flusso è ancora più chiaro in dialog flow cx.

7.2.1 Parametro

Abbiamo definito un agente (set-language) che si attivava se l'utente diceva qualcosa riguardante un linguaggio. Ogni volta che nella training phrases cattura un valore testuale che associa a qualche parametro interno lo colora, ad esempio German french e english li cattura come sys.language. I parametri interno sono parecchi (url, genere musicale, country, name, name...). Chiaramente non tutti i parametri sono previsti, ad esempio nel nostro caso abbiamo nelle training phrases alcuni linguaggi di programmazione, possiamo definire noi il parametro (per noi language-programming) addestrandolo dicendogli quali sono i valori che deve avere e i loro sinonimi.

Nella risposta mettiamo la variabile che abbiamo catturato, risponde a seconda se catturo una training phrases che riguarda una lingua o un linguaggio di programmazione.

7.2.2 Fullfillment

La risposta posso crearla andando ad esempio ad interrogare un API esterna o un DB interno, quindi a seconda della domanda che viene fatta si genera una risposta con un linguaggio di programmazione. Tendenzialmente si usa JavaScript e FireBase come DB.

Dobbiamo crearlo, di base è disabilitato perchè per abilitarlo dobbiamo attivare l'indirizzo di fatturazione. Possiamo:

- Collegarci con un URL di una funzione fatta su Google cloud, tramite web hook
- Usare l'inline editor, bisogna scrivere in JavaScript

Attiviamo all'interno dell'intento la possibilità di legare la risposta ad un fulfillment. Di base funziona sempre con le funzioni di firebase per il deployment di funzioni JS, come catturiamo l'intento e creiamo la risposta? Ci sono delle funzioni che permettono di catturare degli intenti. Creiamo una cattura a livello di codice di un intento personalizzato che catturi la gestione del nome (l'utente chiede come si chiama l'agente e da una risposta cablandola nella funzione):

```
1 function getAgentNameHandler(agent) {  
2   agent.add('From fulfillment: My name is Dialogflow!');  
3 }
```

Questa funzione viene chiamata quando c'è un intento che si chiama "getagentname" il cui handler viene gestito dalla funzione che abbiamo appena scritto (quando catturi questo intento attiva questo handler):

```
1 intentMap.set('get-agent-name', getAgentNameHandler);
```

Proviamo a fare un'altra funzione di handler, che prende un parametro sul linguaggio, sul language programming e risponde a seconda se cattura un linguaggio o un linguaggio di programmazione risponde in maniera differente:

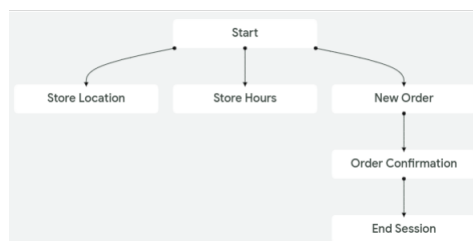
```
1 function languageHandler(agent) {  
2   const language = agent.parameters.language;  
3   const programmingLanguage =  
4     agent.parameters['language-programming'];  
5   if (language) {  
6     agent.add('From fulfillment: Wow! I did not know you knew  
7       ${language}');  
8   } else if (programmingLanguage) {  
9     agent.add('From fulfillment: ${programmingLanguage} is cool');  
10  } else {  
11    agent.add('From fulfillment: What language do you know?');  
12  }  
13 }
```

Attiviamo questo handler quando catturiamo questo intento ("setlanguage"):

```
1 intentMap.set('set-language', languageHandler);
```

É interessante la possibilità di integrazione, lo si può mettere in numerose applicazioni:

- Telegram
- Slack
- Pagine web



7.2.3 Dialog Flow CX

Bisogna creare un progetto in google cloud e all'interno del progetto mettere l'agente. Ci sono dei bot di default che sono già stati implementati che potremmo usare.

Cambia la logica, perchè è più vicino il flusso di dialogo, abbiamo visto un esempio con 3 flussi di dialogo, sostanzialmente sono 3 intenti. La cosa importante sono i:

- Build: creo i flussi di dialogo
- Mange:
 - intenti: la logica è sempre la stessa, metto delle training phrases per catturare l'intento degli utenti. Non da già risposte, catture le richieste espresse in linguaggio naturale
 - Parametri: anche in questo caso nelle training phrases vengono identificati dei parametri, possiamo crearne ad hoc
 - Web hooks: definisco le funzioni

Quello che cambia è come creo i flussi di dialogo, partiamo sempre da una start page, è l'innescò iniziale, dice che quando parto catturo il welcome intent e poi posso andare verso 3 possibili intenti. Quando creo un instradamento devo specificare che intento catturare per andare in quella direzione, posso mettere delle regole di condizioni, quando mi aspetto che ho bisogno di tot parametri per fare qualcosa (colore e dimensione per fare l'ordine) fino a che non ho questi parametri non vado avanti. Le risposte vengono generate nelle page, come se fossero lo step successivo, entro in uno stato in cui do una risposta o vado avanti nel percorso, la page è una descrizione di cosa deve succedere. Abbiamo visto che si possono mettere delle condizioni negli intent, ad esempio se ci sono tutti i parametri (che ho definito come required) mi da una risposta e mi porta ad una page specifica, se ne manca uno dei due posso definire una risposta specifica e non vado avanti.

Anche in CX posso fare fulfillment, lo chiama webhooks, in cui metto l'URL in cui ho creato la funzione ad esempio per fare delle query su un DB firebase.

7.2.4 Ton of voice

C'è tutta una problematica legata alle ton of voice, se mi accorgo che l'utente è molto formale la risposta deve essere formale, se invece vedo che è meno formale posso non esserlo. Questo perchè aderire al ton of voice dell'utente è una cosa apprezzata. Questo si lega alla sentiment analysis, devo capire qual'è il sentiment dell'utente ed adeguarmi.

Non devo più creare una struttura solo tecnica, ma anche una struttura che si adegui all'utente. La risposta "non ho capito per favore ripeti" deve essere sicuramente l'ultima spiaggia, perchè non sto rispondendo niente.

7.3 Generative AI

Con questa logica è difficile che il bot dica "non ho capito", indipendentemente dalla domanda che gli faccio mi da sempre una risposta. Perchè la logica di base è legata alla generazione di una risposta, la generazione della risposta prende spunto da una base di conoscenza così alta che una risposta la trova sempre. I contesti sono un po' diversi, creare un agente come quello per la vendita di magliette con ChatGPT non ha senso, perchè non conosce il dominio. Quindi più il bot è legato ad un contesto specifico più è difficile che dia delle risposte specifiche.

7.3.1 ChatGPT

Parte da InstructionGPT. Sin dalle prime fasi di questo strumento la logica non era mai non rispondere, ma chiedere input in più per convergere ad una risposta.

Il problema del training di ChatGPT è che ci è voluto parecchio. Il modello fu addestrato in maniera supervisionata, successivamente si è fatto reinforcement learning facendo un ranking delle risposte date dal modello. Logica a tre passi:

- Coinvolgere il trainer umano con domande fatte agli umani che dovevano rispondere
- Ranking di possibili risposte, ho diversi modelli con diverse risposte e l'uomo ha fatto un ranking per dire quale fosse la migliore
- Tuning finale con ottimizzazione delle politiche

Il training è stato fatto con un insieme di informazioni fino al 2022.

Limitazioni

- Può dare delle risposte plausibili ma scorrette (allucinazioni).
- Se cambio leggermente la domanda mi risponde
- Eccessiva verbosità della risposta e ripetizioni di frasi
- Limitata capacità di capire query ambigue
- Rischio di dare risposte inappropriate

7.3.2 Embeddings Models

Sono dei vettori numerici che rappresentano un testo, pensiamo ai pesi dei termini. Vengono definiti per poter rappresentare semanticamente un determinato testo. Abbiamo una query e dei numeri che rappresentano dei vettori associabili a quanto ha detto l'utente, vettori che poi usiamo per calcolare le distanze.

7.3.3 Text similarity models

I modelli di somiglianza testuale forniscono incorporazioni che catturano la somiglianza semantica di pezzi di testo. Questi modelli sono utili per attività di:

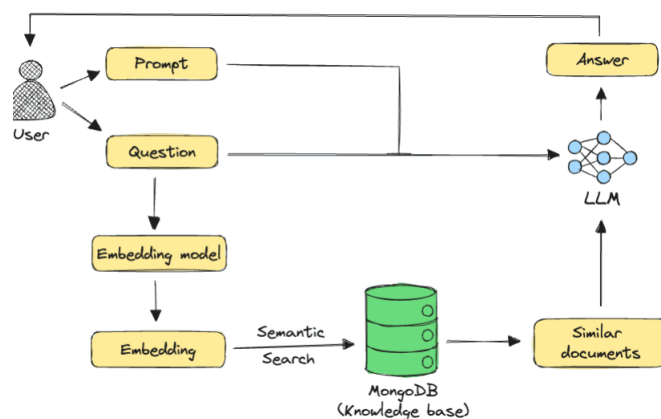
- data visualization
- clustering
- classification

in maniera molto simile rispetto a quello che abbiamo fatto fino ad adesso.

7.3.4 Modelli ricerca testuali

Su questo possiamo fare modelli di ricerca testuale su larga scala per trovare i documenti. Le distanze si calcolano sempre sulla similitudine del coseno dell'angolo che separa il vettore della query e del vettore.

7.4 RAG- Retrieval Augmented Generation



Cerca di ridurre le allucinazioni aumentando l'accuratezza, rimanere aggiornato e pescando da informazioni proprie. A differenza dei classici LLM che cercano di recuperare le informazioni da una mole di risorse generalistica cercano di integrare i due mondi, ha un motore che cerca di recuperare le informazioni da un modello molto ampio di conoscenza integrato da tutte le logiche di question and answering tipiche di dialog flow. Data una query trovare tutti i documenti (sempre nella rappresentazione vettoriale) che siano vicini (coefficienti di similarità) alla query dell'utente.

Quello che ci interessa è trovare un modello per la similarità vettoriale, ad esempio classifichiamo con il text similarity.

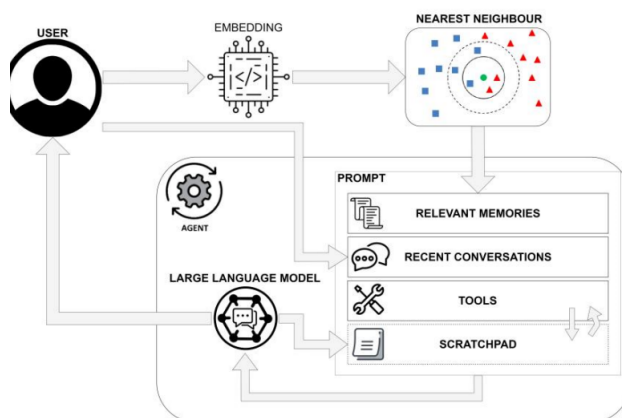
7.4.1 Cheshire Cat

Non è un LMM model, ma integra un LMM model esistente con la long term memory che possiamo aggiornare noi. Possiamo anche specificare delle funzioni in python per rispondere con delle API. È un framework per creare dei chatbot che integri la ricerca su un dominio molto ampio e una ricerca più localizzata. Si estende il concetto di LLM con il concetto di memoria, ci sono diverse memorie da mettere in gioco per dare risposta all'utente:

- Precedenti conversazioni con l'utente, per dare contesto
- Miei documenti
- Memorie che si basano sulle chiamate a specifiche API interessanti per un determinato contesto

Come funziona:

- Chat per interagire con l'utente
- Rabbit Hole: modulo che si occupa dell'ingestion dei documenti
- LLM: possiamo usare il LLM che vogliamo
- Embedder: codifica il testo in rappresentazioni numeriche, facilitando il recupero di frasi simili dalla memoria
- Vector Memory: memorizza i ricordi come vettori in un database, compresi i ricordi episodici di conversazioni passate e i ricordi dichiarativi di documenti caricati
- Agent: orchestrare le chiamate all'LLM, consentendo al Gatto di decidere le azioni in base agli input dell'utente, dalle conversazioni all'esecuzione di compiti
- Plugins: estendono il chatbot per personalizzarlo e renderlo aderente alle aspettative dell'utente



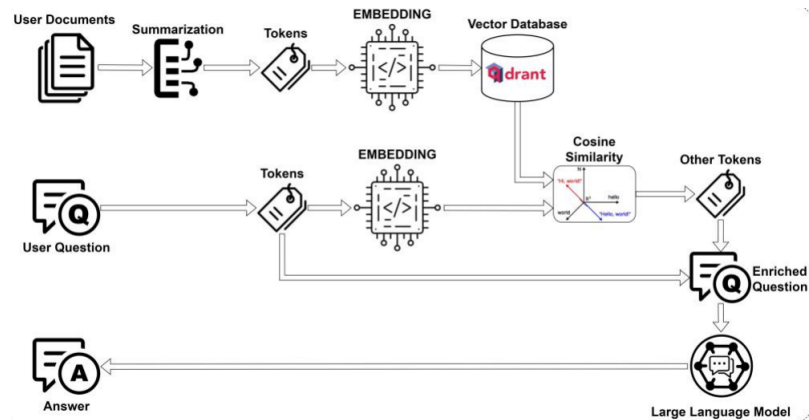
Vector Memory

La memoria all'interno si basa sui vettori n-dimensionali, di cui abbiamo già parlato, e il confronto di similarità avviene tramite un algoritmo che si basa sulla cosine similarity.

Come avviene l'integrazione di queste memorie nell'architettura:

- Domande dell'utente, tokenizzate
- Miei documenti, tokenizzazione

Con queste strutture a indici inversi si va a cercare la cosine similarity per generare le varie risposte:



Long Term Memory

Tipi di memorie che entrano in gioco:

- **Episodic Memory:** la mia domanda non è secca, può esserci un dialogo, è necessario che il dialogo entri in gioco per le risposte future. Se ti ho dato degli input tu tienili a mente (se durante la conversazione ti ho detto che sono di Milano e poi ti chiedo il tempo, tu devi ricordarti che sono di Milano). In dialog flow viene gestito con dei parametri
- **Declarative Memory:** se ho dei miei documenti l'estrazione dell'informazione deve partire da lì. Se sto facendo il chatbot per un negozio che deve vendere degli oggetti la prima informazione deve arrivare da lì
- **Procedural Memory:** sono informazioni che vengono estratte creando degli script (Python) che accedono a dei servizi esterni, ad esempio se voglio Wikipedia posso usarla come informazione principale

Da qui integro tutta l'interrogazione LLM per arricchire la risposta, generare una risposta articolata ed eventualmente colmare delle lacune.

C'è la possibilità nella dashboard di andare a vedere quanto dista la query dalle risposte.

Installazione

Per installarlo dobbiamo scaricare un docker nel quale attivare l'engine del motore.

Nella generazione devo caricare della memoria dichiarativa, facendo così parte la risposta localizzandola sulla memoria localizzata.

È interessante anche che spiega perchè ha dato quella risposta, ci dice anche la confidenza.

Possiamo creare dei plugin, in cui creo un Json in cui dico le specifiche del plugin, ci sono altri settings per personalizzare il ton of voice, nel codice python, ci sono due caratteristiche:

- Il hook serve per creare la personalità dell'agente, sostanzialmente prende il messaggio che ha generato l'agente e poi lo modifica tramite un prompt che manda all'LLM
- I tools: se ci sono delle domande specifiche possiamo generare le risposte a partire da pezzi di codice che abbiamo definito

```
1 # hooks are an event system to get finegrained control over your
  assistant
2 @hook
3 def agent_prompt_prefix(prefix, cat):
4     prefix = """You are Marvin the socks seller, a poetic vendor of
      socks.
5 You are an expert in socks, and you reply with exactly one rhyme.
6 """
7     return prefix
```

```
1 # langchain inspired tools (function calling)
2 @tool(return_direct=True)
3 def socks_prices(color, cat):
4     """How much do socks cost? Input is the sock color."""
5     prices = {
6         "black": 5,
7         "white": 10,
8         "pink": 50,
9     }
10
11     price = prices.get(color, 0)
12     return f"{price} bucks, meeeow!"
```

Chapter 8

Digital Images

Un'immagine digitale è un'area rettangolare che rappresenta delle informazioni visive. Un'informazione visiva è una matrice bidimensionale di pixel con delle informazioni che riguardano la luce e il colore.

Quindi abbiamo:

- Spatial sampling (image resolution)
- Colour sampling (colour depth)

8.1 Risoluzione

La risoluzione indica di quanti pixel misura l'immagine. È misurata in dpi (dots per inch) o pixel.

Ci sono dei limiti tecnici con cui possiamo visualizzare le cose, ad esempio nelle applicazioni web non possiamo superare determinati limiti (640x480 o 1024x768).

8.2 Profondità di colore

Lo stesso discorso vale per la profondità di colore, definisce da quanti colori è rappresentata l'immagine:

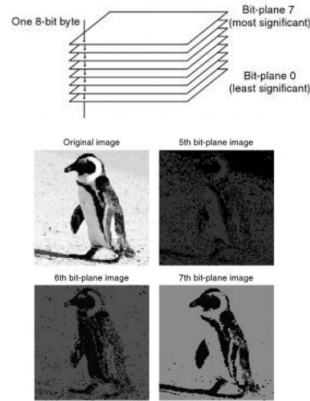
- Monocromatica: 1 bit
- Scala di grigi: 8 bit
- A colori: tendenzialmente 256 colori in 8 bits

Le immagini a 1 bit sono immagini in bianco e nero.

8.2.1 Scala di grigi

Chiaramente le immagini ad un bit non si prevede più, male che vada si usa un'immagine in scala di grigi a 8 bit con un valore che va da 0 a 256. Viene rappresentata tramite la bit planes, abbiamo 8 livelli che rappresentano il livello del bit nell'immagine, per ogni livello viene indicata la sequenza di bit (acceso/spento) che rappresenta la scala di grigi per quel particolare valore).

Quindi se devo calcolare a livello matematico quanto occupa un'immagine se è un'immagine 640x480 in scala di grigio mi occupa 300kB ($640 \times 480 = 307200$).



8.3 Dithering

Per passare da un'immagine digitale ad analogica devo dire se questo pixel è rappresentato in questa codifica ho una rappresentazione in 8 bit, ha quindi un numero, se questo pixel ha questa codifica colori viene usata questa strategia, ogni punto che la stampante andrà a stampare va a stampare il punto con una matrice 2x2 e posso decidere se mettere l'inchiostro su tutta la matrice o su una parte. Decido in base al valore del pixel relativo, prendo il valore del pixel lo divido per 5, creando una scala tra 0 e 256, se il valore cade nel primo quinto non coloro niente, nell'ultimo quinto coloro tutto e in mezzo va in ordine.

Input:	0	1	2	3	4
Output:	()	(•)	(• •)	(• • •)	(• • • •)



8 bit



Dithered version



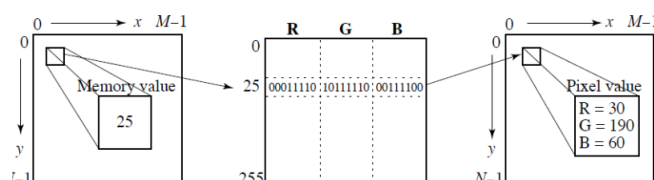
Detail of the dithered version

8.4 Colori

La profondità di colore più banale è la 8 bit color o la 24. Senza considerare la compressione, se utilizzo un'immagine a 24 bit color la rappresentazione avviene tramite una rappresentazione RGB, per ogni canale ho una rappresentazione a 256bit. Quindi possiamo rappresentare $256 \times 256 \times 256 = 16777216$. E mi peserebbe per un'immagine 640×480 in 24 bit 900kb senza compressione.

8.4.1 8 bit color information

Molti sistemi usano la 8 bit color information, che permette di risparmiare informazione. Ad ogni pixel non associo la combinazione RGB, ma associo un valore in 8bit che viene utilizzato per puntare ad una look up table, una tabella RGB, è una tabella in cui abbiamo tutti i 16 milioni di colori ed ogni puntatore mi dice qual'è la combinazione corretta. Questo fa risparmiare spazio, un immagine 640x480 occupa 300kB di storage rispetto ai 900 kB per una immagine colore senza compressione.



8.5 GIF

Sono immagini a 8 bit. Supportano l'interlacciamento.

È un'immagine codificata secondo una particolare codifica, i primi byte ci danno un tipo di informazione, gli altri un altro e così via. I primi sono gli header, danno una signature e la versione di GIF che utilizziamo. I secondi ci danno una descrizione dello screen finale (quanto deve essere grande la canvas che visualizzerà l'immagine). Altri che rappresentano la color table. Altri che rappresentano l'estensione di alcune specifiche grafiche (trasparenza o no) Altri che descrivono l'immagine, dove parte a sinistra, destra, alto basso, quanto è alta, larga. Image data, informazione che riguarda l'immagine in se con un algoritmo di compressione LZW. Testi descrittivi.

Fondamentalmente i byte che rappresentano la gif sono una sequenza di informazioni codificate secondo un particolare standard.

8.6 JPEG

È un algoritmo che ci dice quanto deve essere compressa una determinata immagine.

8.7 PNG

È una versione moderna della GIF in cui rappresento più byte per l'informazione. Sostanzialmente nella sua struttura informativa è più articolata, ci sono delle informazioni ancora aggiuntive.

8.8 TIFF

È un formato più flessibile, può essere usato per la scala di grigi sia a colore. Fa uso di un algoritmo di compressione.

8.9 EXIF

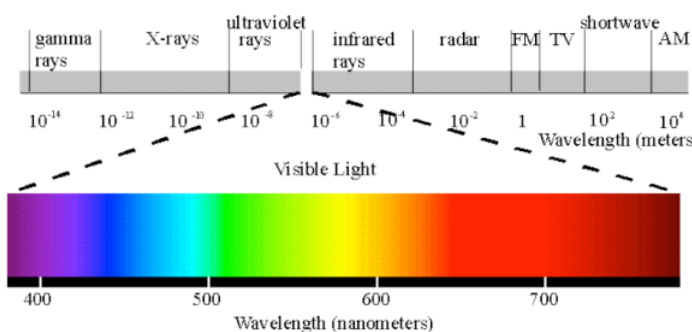
Integra anche tanti metadati (data, luogo), viene tanto usato nelle macchine fotografiche.

8.10 PS

Postscript, rappresenta una rappresentazione vettoriale, è molto più simile alla stampa. Così come il PDF.

8.11 Human perception of color

Un'immagine è un insieme di caratteristiche tecniche, ma abbiamo sicuramente altri tipi di informazione. A livello di percezione umana quando vediamo un'immagine percepiamo prima il colore, poi le informazioni spaziali che ci permettono di associare una semantica a quello che stiamo vedendo. Come codifichiamo il colore. Bisogna capire esattamente dal punto di vista biologico cos'è il colore, perché le query più interessanti dal punto di vista visuale riguardano il colore. Non è altro che un'onda elettromagnetica che viene percepita secondo un particolare spettro dall'occhio umano.



Sono stati fatti degli studi che dimostrano che le persone di sesso maschile e quelle femminile percepiscono colori diversi, c'è una leggera discriminante, l'occhio maschile è meno discriminante su alcune sfumature rispetto a quello femminile. Comunque sicuramente non vediamo tutti i colori dello spettro, vediamo solo una porzione della luce visibile che racchiude una piccola parte dello spettro di luce.

Dal punto di vista biologico quando parliamo di visione umana entrano in gioco coni e bastoncelli. I bastoncelli entrano in gioco quando dobbiamo vedere delle scale di grigio e i coni quando dobbiamo fare degli screening sui colori. Ci sono coni più o meno sensibili a Rosso Verde e Blu, è il motivo per cui parliamo di RGB. Con varie combinazioni RGB riusciamo a ricreare la gamma colori riconoscibile dall'occhio umano.

8.11.1 Codifica del colore

Dobbiamo creare una codifica di colore che ci permetta di sfruttare le caratteristiche fisiche dell'occhio umano. Avrà diversi scopi:

- archiviare le immagini

- proiettare le immagini
- stampare le immagini
- processare le immagini
- trasmettere le immagini (video)
- misurare la colour similairty della percezione umana
- creare uno standard

In realtà non esiste solo un modello di codifica colori, sono in realtà molto riconducibili uno con l'altro, sono tutti soggetti a delle logiche di trasformazioni (prendere un modello e trasformarlo in un altro con delle trasformate).

I modelli più conosciuti sono:

- Modello additivi: RGB, il colore è una combinazione di colori primari rispetto ad una base di partenza che è il nero. Viene usato in tutti gli strumenti di sorgente luminosa (video, proiettore... tutti i device che emettono colore)
- Modello sottrattivo: si parte da una luce bianca e si rimuovono dei colori per definire il range di colori. Sono legati a strumenti quali le stampanti

RGB

Ogni pixel viene associato ad una combinazione con un fattore moltiplicativo rispetto a tre canali:

- Red
- Green
- Blue

Se ad ogni canale associo 8bit ho 16 milioni di colori che posso combinare.

CMY

Si parte da una base di colore, bianco e si sottraggono:

- Ciano
- Magenta
- Giallo

Che sono i colori complementari dell'RGB. Non cambia niente per il numero di colori rappresentati.

Dal punto di vista matematico ha dei difetti nella rappresentazione del nero, perchè è deficitato da questa logica di sottrazione. In realtà si è cercato di creare un quarto componente nel modello da sfruttare per la stampa del nero.

RGB vs CMY

Sono due modelli complementari e linearmente indipendenti, si può quindi passare da un modello all'altro senza problemi.

Ci sono delle trasformate che si possono adottare per passare da un modello all'altro:

From CMYK to RGB:

$$R = 255 \times (1-C) \times (1-K)$$

$$G = 255 \times (1-M) \times (1-K)$$

$$B = 255 \times (1-Y) \times (1-K)$$

From CMY to CMYK:

$$K = \min (C,M,Y)$$

$$C = (C-K) / (1-K)$$

$$M = (M-K) / (1-K)$$

$$Y = (Y-K) / (1-K)$$

From CMYK to CMY:

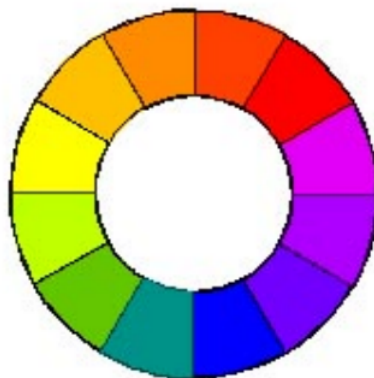
$$C = \min (1,C(1-K)+K)$$

$$M = \min (1,M(1-K)+K)$$

$$Y = \min (1,Y(1-K)+K)$$

8.11.2 Tinta

É vero che il colore è una tinta. Però a volte abbiamo lo stesso colore che può essere più o meno verde. Vuol dire che la tinta in se non è sufficiente per dare una rappresentazione del colore.

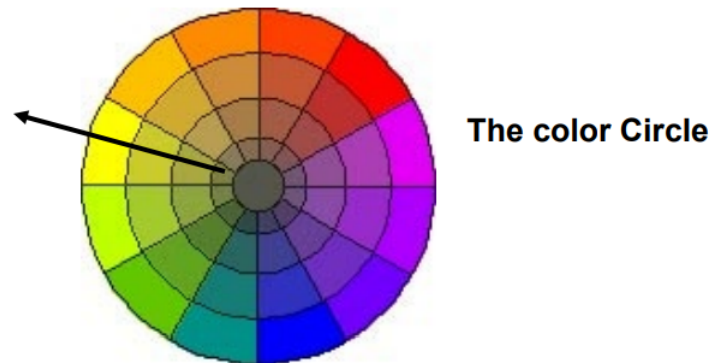


Colour wheel

In realtà nel modello RGB questa logica di più o meno colorato non è evidente, cosa vogliono dire queste sfumature?

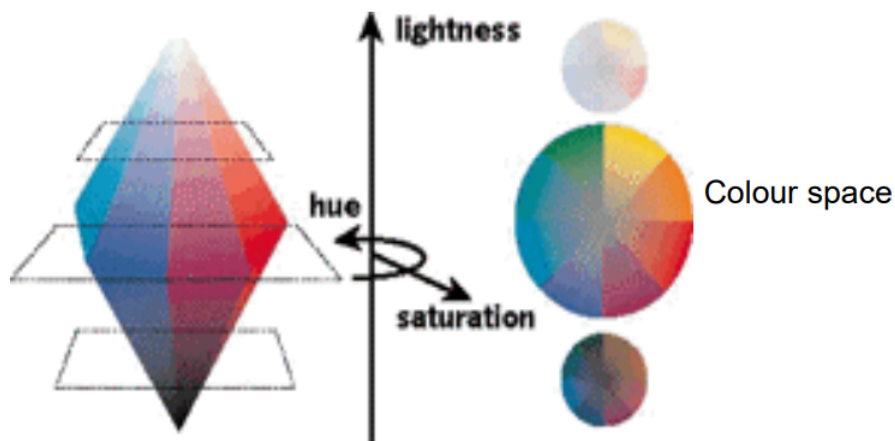
8.11.3 Saturazione

Quindi andiamo a parlare di saturazione, dato un colore puro mischiandolo con il colore bianco come possiamo modificarlo. Quindi passiamo ad un cerchio di rappresentazione del colore, in cui identifichiamo che possiamo rendere più o meno puro il colore in base a quanto lo mischiamo con il colore bianco.



8.11.4 Luminosità

La saturazione non è ancora sufficiente, c'è anche il concetto di più o meno chiaro. Dal cerchio del colore arriviamo alla piramide del colore, aggiungiamo una dimensione per creare un colore più o meno brillante:



Quindi quando percepisco un colore percepisco:

- Tinta
- Saturazione
- Luminosità

8.11.5 HSV

Sono modelli che ci permettono di dare una rappresentazione numerica che danno una discriminante su 3 dimensioni, che non sono i tre canali, ma le tre dinamiche di colore:

- Tinta
- Saturazione
- Luminosità

Posso trasformare un modello HSV in un modello RGB o CMY? sì perchè sono linearmente indipendenti, possiamo usare queste formule matematiche:

$$R' = R/255 \quad G' = G/255 \quad B' = B/255$$

$$Cmax = \max(R', G', B')$$

RGB to HSV

$$Cmin = \min(R', G', B')$$

$$\Delta = Cmax - Cmin$$

$$\text{Hue calculation: } H = \begin{cases} 60^\circ \times \left(\frac{G' - B'}{\Delta} \bmod 6 \right) & , Cmax = R' \\ 60^\circ \times \left(\frac{B' - R'}{\Delta} + 2 \right) & , Cmax = G' \\ 60^\circ \times \left(\frac{R' - G'}{\Delta} + 4 \right) & , Cmax = B' \end{cases}$$

$$\text{Saturation calculation: } S = \begin{cases} 0 & , \Delta = 0 \\ \frac{\Delta}{Cmax} & , \Delta <> 0 \end{cases}$$

$$\text{Value calculation: } V = Cmax$$

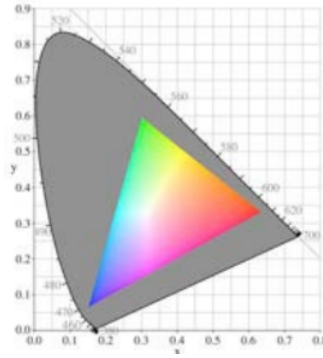
Questo modello ci permette di creare strumenti che ragionino anche sulla semantica delle immagini, quando vedo un colore non è il colore in se, ma è una combinazione di qualcosa.

8.11.6 CIE

C'è uno standard internazionale che ci dice che la visione umana non può essere legata all'RGB. Il CIE ci dà uno standard per la definizione del colore, che va al di là di quelle che sono attualmente le possibilità di rappresentazione del colore. La percezione dell'occhio umano è molto più ampia, copre una gamma di colori che supera i 16 milioni dell'RGB.

Data questo range di colori percepibili dall'occhio umano, ogni device è un sottoinsieme di questo insieme, viene chiamato il GAMUT, è il quantitativo di informazione che il device è in grado di rappresentare. È la porzione di tutti i colori possibili che sono rappresentabili da quel device.

Quello che interessa a noi è che ci sono diversi modelli di rappresentazione colore che però non coprono tutto il range di colori che l'occhio umano è in grado di catturare.



8.12 Image Retrieval

Date le caratteristiche che abbiamo appena visto come le usiamo per creare degli strumenti di image retrieval. Innanzitutto non ci bastano le caratteristiche tecniche dell'immagine, perchè a noi interessa principalmente quello che c'è dentro l'immagine (quello che rappresentano).

Cosa contiene l'immagine è legato a tre componenti:

- Forme
- Texture
- Colore

Ovviamente entrano in gioco tecniche di:

- Compressione
- Segmentazione
- Indicizzazione

E facendo query secondo due approcci:

- Metrico: usato da tutti
- Transazionale

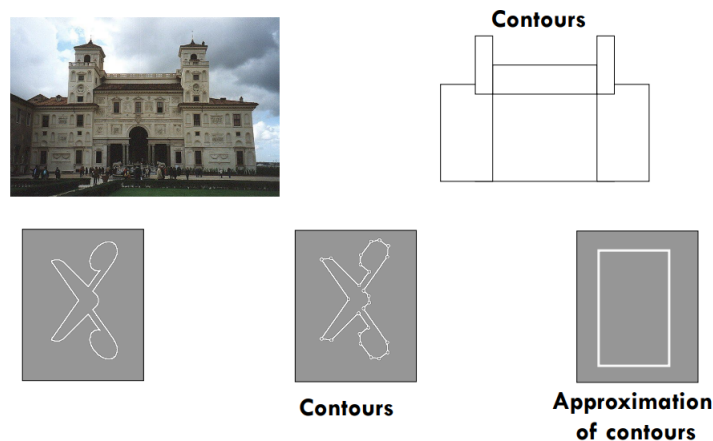
Sono un sottoinsieme di sistemi di multimedia retrieval. Molte cose sono simile al testo. Dobbiamo inizialmente dare una rappresentazione delle immagini, creare un surrogato delle immagini. Cosa c'è nell'immagine? non è facile, è più semplice capire cosa c'è in un testo. Quali sono le proprietà che caratterizzano il surrogato dell'immagine? Le condizioni automatiche o semi automatiche per creare questi surrogati? cosa si intende per misura di similarità?

Le feature di un immagine dipendono dal dominio applicativo. Le più usate sono:

- Forma
- Texture
- Colore

8.12.1 Forma

Concettualmente parlando un'immagine non ha forma. Però possiamo estrarre da un'immagine delle strutture caratteristiche basandoci sulla forma di quello che viene rappresentato al suo interno. Posso usare 16 primitive di forma (quadrato, rettangolo...) per approssimare quello che sto vedendo andando a creare una semplificazione dell'immagine tramite la cattura di queste forme primitive. Ad esempio se sto vedendo la foto di un tramonto, lo posso ricondurre ad un cerchio (il sole) e una linea che separa il mare dal cielo, posso ridurre la rappresentazione grafica in un cerchio e tre rettangoli. La stessa cosa la potrei fare per una persona.



Semantica

Attualmente la forma non ha semantica, non sto dicendo che il cerchio è un sole, ma è solo un cerchio. Quindi al momento sto considerando delle proprietà prive di semantica.

8.12.2 Texture

È una struttura caratteristica che presenta l'immagine, definita da:

- Concentrazione (scala): misura la granularità della texture
- Contrasto: misura la differenza di colore
- Direzionalità: è una proprietà globale dell'immagine

Se c'è uno schema all'interno dell'immagine.

8.12.3 Colore

È un'altra proprietà estraibile dall'immagine. È fondamentale perché è usato come primaria proprietà di discriminante. Possiamo usare qualunque modello (di solito RGB) perché tanto possiamo ricondurci a qualunque altro modello.

Istogramma dei colori

C'è all'interno di questa immagine un uso, nel gamut, di combinazione di colori di un determinato tipo, ma quanto ne sto usando? Data questa gamma di colori sto usando una distribuzione di una gamma di colore all'interno dell'immagine, ad esempio un 70% dell'immagine è bianca.

Con questo istogramma posso capire qual'è la dominante di colore, sono le componenti RGB che superano una particolare soglia.

8.12.4 Surrogati delle immagini

Posso usare queste tre proprietà per creare il primo surrogato dell'immagine, che è la base di partenza su cui farò le query. Attenzione che non posso trattarle tutte e tre nello stesso modo. Il primo set è chiamato descrittori di proprietà, sono informazioni visuali legate a dei set di pixel. Quando faccio retrieval non uso tutti i pixel, si definisce una risoluzione dell'immagine, è la griglia di risoluzione che mi permette di andare ad approssimare la definizione di un'immagine stessa. Considero solo delle celle $n \times m$ con cui andare ad approssimare questa immagine. Perché queste proprietà sono locali, legate al singolo pixel, ma io le considero per cella, faccio la media delle proprietà di tutti i pixel della cella. Queste proprietà locali hanno un nome, un valore e un metodo per poterle estrarre. Sono sempre proprietà di

- Colore: media del colore dei pixel nella cella
- Texture

Qualunque proprietà può essere rappresentata con la tripletta:

- Nome
- Valore
- Metodo

La forma non può essere una proprietà locale, ma è globale.

8.12.5 Database di immagini

Data la definizione di queste proprietà posso definire un database di immagini. È una tripla formata da:

- GI: la griglia delle immagini
- Quali proprietà considero
- Funzioni per estrarre le proprietà

In questa definizione abbiamo tre problemi:

- Compressione:
 - Come ridurre il numero di pixel in un immagine per ridurre lo spazio sul disco

- Come usiamo questa rappresentazione nelle queries
- Segmentation:
 - Come si fa a suddividere l'immagine in un insieme di regioni in modo che ogni regione sia omogenea per quanto riguarda alcune proprietà?
- Queries:
 - Come determiniamo se due immagini sono simili?

Compressione

Non ci interessa tanto dal punto di vista algoritmico, però dovremmo capire come trattare le immagini compresse. Ci interessa dal punto di vista dalla proprietà dell'invertibilità, data un'immagine compressa si può tornare alla sua versione precedente (Image retrieval funziona bene se ha questa proprietà). Anche se c'è una compressione non invertibile che però mantengano la distanze.

Segmentazione

Informalmente, il problema consiste nel prendere in input un'immagine e produrre un insieme di regioni connesse che dividano l'immagine in modo che ogni regione sia omogenea per quanto riguarda alcune proprietà. Si usano colore, texture e contorni per fare due segmentazioni diverse. Possiamo creare delle sezioni di immagini.

Query

Due tipi di query:

- QBF: query by feaytres: sfrutto le feature tecniche e visuali. È una tipologia utile per fare una prima scrematura.
- QBE: query by example: ti do un immagine e restituisce quella che si avvicina di più

8.12.6 Similarità

La distanza tra due immagini è un calcolo matematico della distanza dei vettori che rappresentano le due immagini. Cos'è il feature vector di un'immagine? è un insieme di valori che rappresentano le proprietà locali e globali dell'immagine.

Nell'approccio metrico valgono queste proprietà:

- La distanza tra x e y è uguale a quella tra y e x
- $d(x, y) \leq d(x, z) + d(z, y)$
- $d(x, x) = 0$

La distanza tra due immagini è ottenuta combinando la distanza tra varie proprietà (locali e globali).

Vediamo un esempio pratico, come calcolo la distanza di un colore. La proprietà locale di un colore è un valore RGB associato ad ogni cella, per ogni cella ho un numero (tra 0 e 8). Nel vettore di proprietà avremo $n \times m$ celle e per ogni cella un numero. Dato che per ogni cella ho un numero, date due immagini (con la stessa griglia di risoluzione) posso calcolare la distanza euclidea rispetto al canale Red Green e Blue di ogni cella.

Quindi la distanza della proprietà locale colore è la combinazione dei tre canali RGB.

Quindi ho una griglia di dimensione $n \times m$, per ogni cella avrò un valore rappresentante la proprietà locale. Solitamente non si usano degli spazi ad altissima dimensionalità.

Dato che sono dei numeri posso creare degli indici su queste proprietà locali.

Proprietà globali

La forma non è altro che una forma primitiva, posso creare una rappresentazione numerica perchè ho dei punti in bidimensionale che posso usare per calcolare le distanze. Più la forma è complicata più è difficile calcolare delle distanze, solitamente si usano delle forme semplici.

Fondamentalmente se in due immagini vedo due forme primitive uguali posso calcolare quanto sono distanti, perchè calcolo la posizione in cui sono nell'immagine e se sono in posizioni simili sono immagini vicine.

- Distanza data dalla dimensione delle forme
- Distanza data la posizione delle forme nello spazio

Posso quindi dare una rappresentazione numerica delle proprietà locali e globali, date tutte queste informazioni posso calcolare delle distanze. Questo mi permette di fare le query, perchè posso dire: "dammi tutti gli oggetti tali per cui la distanza sia minore di un epsilon" (epsilon similarity) oppure "dammi tutti gli oggetti che sono più vicini ad un oggetto di partenza" (NN-Similarity).

8.12.7 Feature Vector

Abbiamo un vettore n dimensionale di numeri che posso confrontare ad un altro vettore n dimensionale di numeri, dobbiamo ancora calcolare il coseno dell'angolo α tra i due vettori.

possiamo fare quattro tipi di queries:

- Corrispondenza completa e precisa
- Corrispondenza parziale e precisa
- Corrispondenza completa e imprecisa
- Corrispondenza parziale e imprecisa

Completo/parziale si riferisce agli oggetti che dobbiamo considerare nella query. Preciso/impreciso si riferisce a quante immagini vogliamo trovare in relazione alla distanza.

8.12.8 Precisione e ricerca parziale

La combinazione tra full partial, precise e non precise ci permette di dare una combinazione di ricerche:

- Full precise: tutte le immagini più vicine
- Partial and precise: voglio considerare una porzione dell'immagine (secondo il concetto di segmentazione o secondo concetti miei), è utile ad esempio per cercare le persone simili data una fototessera
- Full imprecise: significa che ti do una soglia, non tutte le immagini vicine ma da una soglia

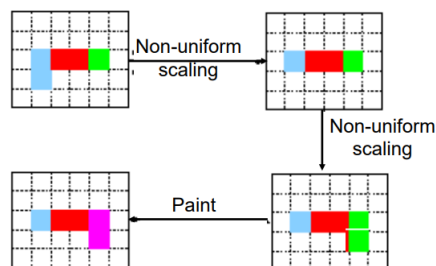
8.12.9 Metodo trasformatore

Cambia le regole rispetto al metodo metrico. Per confrontare due immagini o1 e o2 invece di fare il confronto numerico, non mi interessa catturare le proprietà, ma voglio calcolare quanto costerebbe fare un morphing tra le due immagini. Si basa su operazioni primitive di trasformazione:

- Traslazione
- Rotazione
- Scaling
- Engravings

Ogni operazione ha un costo, per passare da un'immagine all'altra devo sommare diversi costi (ovviamente minimo costo).

Chiaramente il modello transazionale non si basa sui concetti di metriche, si basa sulle operazioni che però devo creare (cosa vuol dire scaling, painting...), dal punto di vista del codice devo creare funzioni che facciano queste cose. Il modello più usato è quello metrico, perché è più efficiente, perché avendo un surrogato posso indicizzarlo, le query quindi diventano più efficienti.



8.12.10 Image DBMS

Un dataset di immagini possiamo considerarlo come un insieme di immagini esterne, o sfruttare la logica di archiviazione di un database, creo degli oggetti che sono memorizzabili in una tabella strutturata.

Un DBMS di immagini è una tripla in cui abbiamo:

- Immagini con una risoluzione
- Proprietà locali
- Proprietà globali

8.13 Oracle Multimedia

Ci permette di sfruttare l'oggetto, ma potrei anche prendere le immagini da un posto diverso (file system o web).

In Oracle esiste il tipo di dato `ORDImage`.

Mi interessa il concetto perchè è vero che l'immagine si definisce un'informazione non strutturata, però abbiamo visto che nei sistemi object relational andiamo incontro a questa problematica sfruttando il concetto di tipo di dato.

Oracle multimedia è un modulo che permette di memorizzare e recuperare immagini.

È necessario fare una distinzione terminologica:

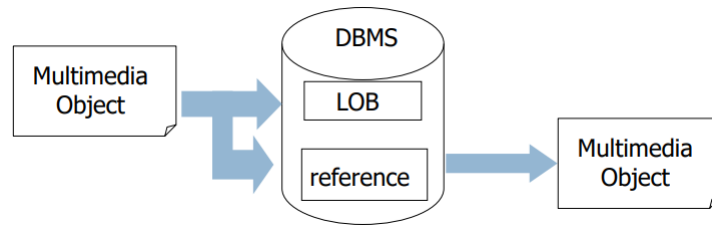
- Media data: si intende l'oggetto, immagine, audio, video, testo...
- Media format data: si intende la caratteristica tecnica del formato dato
- Media keywords data: sono dei descrittori, metadati associati all'oggetto (data, tempo, spazio...)
- Media features data: sono le caratteristiche estraibili visuali, da un particolare media data. Per quanto riguarda le immagini sono:
 - Colore
 - Forma
 - Texture

Useremo dei BLOB messi in locale sul file system.

8.13.1 Storage

Il dato non è strutturato. Abbiamo il dato e il DBMS che lo divide in:

- LOB
- reference



LOB

Ci permette di memorizzare fino a 4GB con gli oggetti BLOB. Si fa indicizzazione, in modo manuale o automatico. Viene usato un'estensione del linguaggio SQL per fare delle query di similarità.

Metadata

Sono tutti rappresentati con dei file XML, così come le keywords.

8.13.2 Storage system

Tipi di dati per rappresentare l'informazione:

- ORDImage
- ORDVideo
- ORDAudio
- ARDDoc

ORDImage

Una volta caricata un'immagine in un oggetto ne estrapola tutte le informazioni che rappresentano il media formato data. L'oggetto ha degli attributi che sono le caratteristiche di formato dell'immagine.

Quindi avremo l'ORDSource più le informazioni che arricchiscono il dato. Potremmo aggiungere anche l'ORDImageFeatures che è il suo surrogato di rappresentazione.

La logica di segmentazione è quella teorica, ci sono degli attributi visuali:

- Colore
- Texture
- Forma
- Posizione: posizione delle texture, colori

8.13.3 Tipo di dato OrdImage

Gli attributi sono la sorgente, che è un altro oggetto (file system, URL...). Una volta che carico un'immagine nell'oggetto tramite metodi va a popolare questi attributi. Chiaramente oltre a questi attributi ho i metodi che mi permettono di popolare questi attributi, interrogarli, cambiarli.

Lo devo creare usando un opportuno costruttore.

8.13.4 ORDSource

È l'oggetto che specifica dove ho preso l'immagine.

8.14 Blob

Cominciamo a creare le tabelle per poter gestire questo tipo di informazioni. Usiamo il blob, ma potremmo usare l'ORDImage o lo StillImage che sono evoluzioni per gestire le immagini. L'unica cosa che il blob ha di differente è che non è adatto a fare confronti di similarità

```
1 create table image_blob_table (id          number primary key,
2                                width        integer,
3                                height       integer,
4                                contentLength integer,
5                                mimeType     varchar2(20),
6                                image_blob   BLOB)
7 lob(image_blob) store as securefile;
```

Creiamo una procedura per caricare le immagini, questa procedura implica che io abbia già un oggetto directory:

```
1 -- imports the file named <filename> in IMAGEDIR into image_blob_table
  in a new
2 -- row. The new row has the id identifier set to <dest_id>
3 create or replace procedure image_blob_import(dest_id number, filename
  varchar2) is
4   img_blob BLOB;
5 begin
6   delete from image_blob_table where id = dest_id;
7   insert into image_blob_table (id, image_blob) values (dest_id,
  empty_blob())
8   returning image_blob into img_blob;
9
10  ORDSYS.ORD_IMAGE.importFrom(img_blob, 'file', 'IMAGEDIR', filename);
11  update image_blob_table set image_blob=img_blob where id=dest_id;
12 end;
13 /
14
15 call image_blob_import(1,'flowers.jpg');
16 call image_blob_import(2,'goats.gif');
```

La funzione di import dice che nella variabile temporanea devi caricare una particolare immagini, a partire dalla directory, che va sotto il nome file name, dopo di che

fai l'inserimento nella tabella con l'ID. Ad ogni ciclo prendo l'immagine e uso la funzione ORDSYS.ORD_IMAGE.getProperties che mi prende delle proprietà tecniche che vogliamo dalla immagine:

```

1  -----
2  -- Populating height, width, contentLength and
3  -- mimeType columns in image_blob_table
4  -----
5  -- sets the height, width, contentLength and mimeType columns for all
   rows
6  -- in the image_blob_table that have null mimetypes using the
7  -- ORDSYS.ORD_IMAGE.getProperties() procedure
8  create or replace procedure image_blob_getproperties is
9      img_mimeType          varchar2(32);
10     img_width              integer;
11     img_height             integer;
12     img_contentLength      integer;
13     unused_fileFormat      varchar2(32);
14     unused_contentFormat   varchar2(32);
15     unused_compressionFormat varchar2(32);
16 begin
17     for rec in (select id, image_blob from image_blob_table where mimeType
   is null) loop
18         ORDSYS.ORD_IMAGE.getProperties(rec.image_blob,
19                                         img_mimeType,
20                                         img_width,
21                                         img_height,
22                                         unused_fileFormat,
23                                         unused_contentFormat,
24                                         unused_compressionFormat,
25                                         img_contentLength);
26         update image_blob_table
27             set width=img_width,
28                 height=img_height,
29                 contentLength=img_contentLength,
30                 mimeType = img_mimeType
31             where id=rec.id;
32     end loop;
33     commit;
34 end;
35 /
36
37 call image_blob_getProperties();
38
39 select id, height, width, mimeType, contentLength from image_blob_table;

```

Oltre ad estrarre proprietà tecniche di un immagine posso modificare queste informazioni, duplicare un immagine modificando il formato di compressione, la risoluzione e così via. Il problema è se cerco di aumentare la risoluzione e la compressione perchè il risultato visivo è stratchato. La prima cosa che si fa quando si ha un catalogo è creare i thumbnail delle immagini che si usano per fare le preview, si gioca sulle dimensioni, prendo due immagini, due blob, e attivo una procedura thumbnails che fa la riduzione dell'immagine stessa. di default rende l'immagine 80x80:

```

1  -----

```

```

2 -- Creating thumbnails
3 -----
4 -- Adds a new row to image_blob_table with id=<dest_id> which includes a
5 -- new image in the image_blob column. The new image is created using
6 -- the
7 -- image from the source row with id=<source_id> and using the
8 -- ORDSYS.ORD_IMAGE.thumbnail() procedure. The destination image will
9 -- contain the output blob will contain an image with a predefined
10 -- size of 80x80 at most.
11 create or replace procedure image_blob_thumbnail(source_id number,
12         dest_id number) is
13     src_blob BLOB;
14     dst_blob BLOB;
15 begin
16     delete from image_blob_table where id = dest_id;
17     insert into image_blob_table(id, image_blob)
18         values (dest_id, empty_blob());
19     select image_blob into src_blob from image_blob_table
20         where id = source_id;
21     select image_blob into dst_blob from image_blob_table
22         where id = dest_id for update;
23     -- Process the data
24     ORDSYS.ORD_IMAGE.thumbnail(src_blob,dst_blob);
25     update image_blob_table set image_blob = dst_blob where id = dest_id;
26 end;
27 /
28
29 -- Convert flowers.jpg to a thumbnail image with a predefined size of
30 -- 80x80
31 -- at most
32 call image_blob_thumbnail(1,3);

```

Possiamo anche cambiare il formato di compressione:

```

1 -----
2 -- Changing Formats
3 -----
4 -- Adds a a new row to image_blob_table with id=<dest_id> which
5 -- includes a
6 -- new image in the image_blob column. The new image is created using
7 -- the
8 -- image from the source row with id=<source_id> and using
9 -- the ORDSYS.ORD_IMAGE.convert() procedure. The destination image will
10 -- be encoded with the file format specified by the fileformat
11 -- parameter.
12 create or replace procedure image_blob_convert(source_id number,
13         dest_id number, fileformat varchar2) is
14     src_blob BLOB;
15     dst_blob BLOB;
16 begin
17     delete from image_blob_table where id = dest_id;
18     insert into image_blob_table(id, image_blob)
19         values (dest_id, empty_blob());
20     select image_blob into src_blob from image_blob_table
21         where id = source_id;
22     select image_blob into dst_blob from image_blob_table

```

```

19     where id = dest_id for update;
20     ORDSYS.ORD_IMAGE.convert(src_blob,fileformat,dst_blob);
21     update image_blob_table set image_blob = dst_blob where id = dest_id;
22 end;
23 /
24 show errors;
25
26
27 --Convert flowers.jpg to flowers.png and store the resulting "image in
28 --a row of the image_blob_table with id=4"
29 call image_blob_convert(1,4,'PNGF');
30
31 -- Adds a new row to image_blob_table with id=<dest_id> which includes a
32 -- new image in the image_blob column. The new image is created using
33 -- the
34 -- image from the source row with id=<source_id> and processing it using
35 -- the ORDSYS.ORD_IMAGE.processCopy() procedure and passing the command
36 -- string specified by <verb>.
37 -- NOTE: assumes <source_id> not equal to <dest_id>
37 create or replace procedure image_blob_processCopy(source_id number,
38     dest_id number, verb varchar2) is
39     src_blob BLOB;
40     dst_blob BLOB;
41 begin
42     delete from image_blob_table where id = dest_id;
43     insert into image_blob_table (id, image_blob)
44     values (dest_id, empty_blob());
45
46     select image_blob into src_blob
47     from image_blob_table
48     where id = source_id;
49     select image_blob into dst_blob
50     from image_blob_table
51     where id = dest_id for update;
52
53     ORDSYS.ORD_IMAGE.processCopy(src_blob, verb, dst_blob);
54     update image_blob_table set image_blob = dst_blob where id = dest_id;
55 end;
56 /
57
58 -- Scale flowers.jpg to 10% into row with id=5
59 call image_blob_processcopy(1,5,'scale=.1');
60
61 -- convert goats.gif to a grayscale jpeg thumbnail with a maximum size
62 -- of 75x100 pixels and store the results into the row with id=6
63 call image_blob_processcopy(2,6,'fileformat=jfif contentformat=8bitgray
64     maxscale=75 100');
65
66 -- update the metadata for the newly created image rows
67 call image_blob_getProperties();
68
69 -- admire our handiwork
70 select id, height, width, mimeType, contentLength from image_blob_table;

```

Potremmo anche voler mettere un watermark sull'immagine:

```

1 -----
2 -- Applying watermark to a BLOB
3 -----
4 -- Adds a new row to image_blob_table with id=<dest_id> which includes a
5 -- new image in the image_blob column. The new image is created using
6 -- the
7 -- image from the source row with id=<source_id> and applying a Text
8 -- watermark
9 -- to the image using ORDSYS.ORD_IMAGE.applyWatermark()
10
11 create or replace procedure image_blob_addwatermark(source_id number,
12             dest_id number) is
13     src_blob BLOB;
14     added_text varchar2(200);
15     dest_blob BLOB;
16     prop ordsys.ord_str_list;
17     logging VARCHAR2(2000);
18 begin
19     delete from image_blob_table where id = dest_id;
20     insert into image_blob_table (id, image_blob)
21     values (dest_id, empty_blob());
22
23     select image_blob into src_blob from image_blob_table
24     where id = source_id;
25     select image_blob into dest_blob from image_blob_table
26     where id = dest_id for update;
27
28     added_text := 'Oracle Multimedia 2013';
29     -- specify properties of text watermark
30     prop := ordsys.ord_str_list(
31         'font_name=Times New Roman',
32         'font_style=bold',
33         'font_size=50',
34         'text_color=red',
35         'position_x=100',
36         'position_y=100',
37         'transparency=0.6');
38
39     -- add text watermark to source image and generate new destination
40     -- image
41     ORDSYS.ORD_IMAGE.applyWatermark(src_blob, added_text, dest_blob,
42         logging, prop);
43     update image_blob_table set image_blob = dest_blob where id = dest_id;
44     commit;
45 end;
46 /
47
48 call image_blob_addwatermark(1,7);

```

Abbiamo la possibilità di estrarre l'immagine:

```

1 -----
2 -- Exporting Images with ORDSYS.ORD_IMAGE.export()
3 -----
4 -- Writes the image data from the BLOB with id=<source_id> in
5 -- image_blob_table

```

```

5 -- to the file named <filename> in the IMAGEDIR directory
6 create or replace procedure image_blob_export(source_id number,
7     filename varchar2) as
8     img_blob BLOB;
9 begin
10     select image_blob into img_blob from image_blob_table where id =
11         source_id;
12     ORDSYS.ORD_IMAGE.export(img_blob, 'IMAGEDIR', filename);
13 end;
14 /
15 call image_blob_export(3, 'flowers_thumbnail.jpg');
16 call image_blob_export(6, 'goats_fixedscale.jpg');

```

Abbiamo la possibilità di estendere il tipo di dato andando a metterci dei metadati in più:

```

1 -----
2 -- Adding Metadata with ORDSYS.ORD_IMAGE.putMetadata()
3 -----
4 -- Embeds XMP metadata into a blob image stored at id=<source_id> and
5 -- saves the resulting blob at id=<dest_id>
6
7 create or replace procedure image_blob_addmetadata(source_id number,
8     dest_id number) is
9     src_blob BLOB;
10    dst_blob BLOB;
11    xmlData XMLType;
12 begin
13     select image_blob into src_blob from image_blob_table where id =
14         source_id;
15     delete from image_blob_table where id = dest_id;
16     insert into image_blob_table(id, image_blob) values(dest_id,
17         empty_blob());
18     select image_blob into dst_blob from image_blob_table where id =
19         dest_id for update;
20
21     -- Create a valid XML packet to embed in the image
22     xmlData := xmltype(
23         '<xmpMetadata xmlns="http://xmlns.oracle.com/ord/meta/xmp">' ||
24         '<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#" ||
25         ' xmlns:dc="http://purl.org/dc/elements/1.1/">' ||
26         '<dc:rights>' ||
27         ' <rdf:Alt>' ||
28         ' <rdf:li xml:lang="en-us">' ||
29         ' Oracle Corporation' ||
30         ' </rdf:li>' ||
31         ' </rdf:Alt>' ||
32         ' </dc:rights>' ||
33         ' </rdf:RDF>' ||
34         ' </xmpMetadata>', 'http://xmlns.oracle.com/ord/meta/xmp');
35
36     -- Insert the metadata and update the image
37     ORDSYS.ORD_IMAGE.putMetadata(src_blob, dst_blob, xmlData, 'xmp',
38         'utf-8');
39     update image_blob_table

```

```

35         set image_blob = dst_blob where id = dest_id;
36     commit;
37 end;
38 /
39
40
41 -- Embed metadata into flowers.jpg
42 call image_blob_addmetadata(1,8);
43
44 -- In order to verify that we have added metadata to the blob image, we
45    retrieve
46 -- it from the database using ORDSYS.ORD_IMAGE.getMetadata().
47 create or replace procedure image_blob_extractmetadata(src_id number) as
48     metav XMLSequenceType;
49     cursor xmlToString(x XMLSequenceType) is
50     select value(list_of_values).getStringval() metadata from table(x)
51         list_of_values;
52     tmp varchar(4000);
53     dest_blob BLOB;
54 begin
55     select image_blob into dest_blob from image_blob_table where id =
56         src_id;
57
58     metav := ORDSYS.ORD_IMAGE.getMetadata(dest_blob,'ALL');
59     open xmlToString(metav);
60     loop
61         fetch xmlToString into tmp;
62         dbms_output.put_line('xmlExtracted: ' || tmp);
63         exit when xmlToString%NOTFOUND;
64     end loop;
65     close xmlToString;
66 end;
67 /
68
69 -- Display extracted metadata for flowers.jpg
70 call image_blob_extractmetadata(8);

```

8.15 ORDImage

É la prima versione di oggetto immagine all'interno di Oracle:

```

1 --Creazione riferimento ORDIMGDIR a directory che contiene immagini da
2 --importare:
3 DROP DIRECTORY ORDIMGDIR;
4 CREATE DIRECTORY ORDIMGDIR AS 'C:\immagini';

```

Creiamo la tabella:

```

1 DROP TABLE stockphotos;
2 CREATE TABLE stockphotos (
3     photo_id NUMBER,
4     photographer VARCHAR2(64),
5     annotation VARCHAR2(255),

```

```

6 photo ORDSYS.ORDImage,
7 photo_sig ORDSYS.ORDImageSignature);

```

e andiamo a popolarla:

```

1 DECLARE
2 Image ORDSYS.ORDImage;
3 ctx RAW(4000) := NULL;
4 BEGIN
5 INSERT INTO stockphotos VALUES (1, 'Fabio Rossi', 'Cerchio nero sfondo
6 bianco', ORDSYS.ORDImage.init(), ORDSYS.ORDImageSignature.init());
7 SELECT photo INTO Image FROM stockphotos
8 WHERE photo_id = 1 for UPDATE;
9 Image.setSource('file','ORDIMGDIR','Img1.jpg');
10 Image.import(ctx);
11 UPDATE stockphotos SET photo = Image WHERE photo_id = 1;
12 COMMIT;
13 END;
14 /
15 CREATE OR REPLACE PROCEDURE MYImage(ident number, photographer
16 VARCHAR2,annotation varchar2) is
17 Image ORDSYS.ORDImage;
18 ctx RAW(4000) := NULL;
19 photo_ident number;
20 photographer_str VARCHAR2 (64) ;
21 annotation_str VARCHAR2 (64) ;
22 filename VARCHAR2 (64) ;
23 BEGIN
24 photo_ident:=ident;
25 LOOP
26   photographer_str :='';
27   annotation_str :='';
28   photographer_str := photographer || '_' || photo_ident;
29   annotation_str := annotation || '_' || photo_ident;
30   filename := 'img' || photo_ident || '.jpg';
31   INSERT INTO stockphotos VALUES (photo_ident, photographer_str,
32 annotation_str, ORDSYS.ORDImage.init(),
33   ORDSYS.ORDImageSignature.init());
34   SELECT photo INTO Image FROM stockphotos
35   WHERE photo_id = photo_ident for UPDATE;
36   Image.setSource('file','ORDIMGDIR',filename);
37   Image.import(ctx);
38   UPDATE stockphotos SET photo = Image WHERE photo_id = photo_ident;
39   photo_ident := photo_ident+1;
40   EXIT WHEN photo_ident>16;
41 END LOOP;
42 END;
43 /
44 exec MYIMAGE(1,'pippo','didascalìa');

```

Andiamo a popolare questa tabella, la cosa comoda dell'ORDImage è che lo posso usare come un oggetto in cui ci sono già caricati i dati tecnici, posso leggere direttamente l'ORDImage. Data la tabella per ogni foto estrai i formati e i dati tecnici che contraddistinguono le varie foto:

```

1 col colonna1 format a20

```

```

2 col colonna2 format a20
3 col colonna3 format a20
4 select photo_id,
5 t.photo.getheight(),
6 t.photo.getwidth(),
7 t.photo.getfileformat() colonna1,
8 t.photo.getcompressionformat()colonna2,
9 t.photo.getcontentformat()colonna3,
10 t.photo.getcontentleng

```

Anche in questo caso potrei usare il thumbnail, non c'è una funzione ad hoc ma ne faccio una copia:

```

1 -- Generates a new ORDImage in row with id=dest_id based on the verb and
2 the
3 -- ORDImage in the row with id=source_id.
4 -- NOTE: assumes source_id not equal to dest_id
5 create or replace procedure MYimage_processCopy(source_id number,
6 dest_id
7 number, verb varchar2) is
8 imgSrc ordsys.ordimage;
9 imgDst ordsys.ordimage;
10 begin
11 insert into stockphotos values (dest_id,'autore',
12 'Texture',ordsys.ordimage.init(),ORDSYS.ORDImageSignature.init());
13 select photo into imgSrc from stockphotos where photo_id = source_id;
14 select photo into imgDst from stockphotos where photo_id = dest_id for
15 update;
16 imgSrc.processCopy(verb, imgDst);
17 update stockphotos set photo = imgDst where photo_id = dest_id;
18 end;
19 /
20 show errors
21 call MYimage_processcopy(1,17,'scale=.1');
22 call MYimage_processcopy(2,18,'fileformat=TIFF contentformat=8bitgray
23 maxscale=100 100');
24 -- look at our handiwork
25 column t.photo.getfileformat() format A20;
26 select photo_id, t.photo.getWidth(), t.photo.getHeight(),
27 t.photo.getFileFormat()
28 from stockphotos t;

```

Vediamo come generare le signature, facciamo un loop in cui per ogni immagine chiamo la generateSignature, estraiamo le informazioni visuali che contraddistinguono il feature vector dell'immagine:

```

1 DECLARE
2 myimg ORDSYS.ORDImage;
3 mysig ORDSYS.ORDImageSignature;
4 x INTEGER;
5 ctx RAW(4000) := NULL;
6 BEGIN
7 FOR x in 1..16 LOOP
8 SELECT S.photo, S.photo_sig INTO myimg, mysig
9 FROM stockphotos S
10 WHERE S.photo_id = x FOR UPDATE;

```

```

11 myimg.import(ctx);
12 mysig.generateSignature(myimg);
13 UPDATE stockphotos S
14 SET S.photo = myimg,
15 S.photo_sig = mysig
16 WHERE S.photo_id = x;
17 END LOOP;
18 END;

```

Possiamo confrontare due immagini, per calcolare la distanza tra due feature vector si usa una particolare funzione (ORDSYS.ORDImageSignature.evaluateScore) che richiede in input le due feature da comparare ed i parametri (colore, texture, shape, location), sono sempre pesi che vanno da 0 a 1:

```

1 DECLARE
2 t_image ORDSYS.ORDImage;
3 c_image ORDSYS.ORDImage;
4 image_sig ORDSYS.ORDImageSignature;
5 compare_sig ORDSYS.ORDImageSignature;
6 score FLOAT;
7 BEGIN
8 SELECT p.photo, p.photo_sig INTO t_image, image_sig
9 FROM stockphotos p
10 WHERE p.photo_id = 3 FOR UPDATE;
11 -- Generate a signature:
12 image_sig.generateSignature(t_image);
13 UPDATE stockphotos p SET p.photo_sig = image_sig
14 WHERE photo_id=3;
15 SELECT p.photo, p.photo_sig INTO c_image, compare_sig
16 FROM stockphotos p
17 WHERE p.photo_id = 4 FOR UPDATE;
18 -- Generate a signature:
19 compare_sig.generateSignature(c_image);
20 UPDATE stockphotos p SET p.photo_sig = compare_sig
21 WHERE photo_id = 4;
22 SELECT p.photo, p.photo_sig INTO t_image, image_sig
23 FROM stockphotos p
24 WHERE p.photo_id = 3;
25 SELECT p.photo, p.photo_sig INTO c_image, compare_sig
26 FROM stockphotos p
27 WHERE p.photo_id = 4;
28 -- Compare two images for similarity based on image color:
29 score:=ORDSYS.ORDImageSignature.evaluateScore(image_sig,
30 compare_sig,'color=1.0,texture=0,shape=0,location=0');
31 DBMS_OUTPUT.PUT_LINE('Score is ' || score);
32 END;

```

Se devo dedicarmi al confronto dato che il sistema di confronto dell'ORDImage non era efficace è stato introdotto lo STILLIMAGE. Proviamo a creare una tabella:

```

1 drop table si_media;
2 CREATE TABLE SI_MEDIA(
3 PRODUCT_ID NUMBER(6),
4 PRODUCT_PHOTO SI_StillImage,
5 AVERAGE_COLOR SI_AverageColor,
6 COLOR_HISTOGRAM SI_ColorHistogram,

```

```

7  FEATURE_LIST SI_FeatureList,
8  POSITIONAL_COLOR SI_PositionalColor,
9  TEXTURE SI_Texture,
10 CONSTRAINT id_pk PRIMARY KEY (PRODUCT_ID));
11 COMMIT;
12 DECLARE
13   lobd blob;
14   fils BFILE;
15   filename VARCHAR2 (64) ;
16 BEGIN
17 FOR x in 1..16 LOOP
18   filename := 'img' || x || '.jpg';
19   fils := BFILENAME('ORDIMGDIR',filename);
20   DBMS_LOB.CREATETEMPORARY(lobd, TRUE);
21   DBMS_LOB.fileopen(fils, DBMS_LOB.file_readonly);
22   DBMS_LOB.LOADFROMFILE(lobd, fils, DBMS_LOB.GETLENGTH(fils));
23   DBMS_LOB.FILECLOSE(fils);
24   INSERT INTO SI_MEDIA (product_id, product_photo)
25   VALUES(x, new ORDSYS.SI_StillImage(lobd));
26   DBMS_LOB.FREETEMPORARY(lobd);
27   COMMIT;
28 END LOOP;
29 END;
30 /

```

È un oggetto utilizzato per memorizzare i dati (come il BLOB e l'ORDIMAGE) ma in più vi ha una gestione e confronto degli attributi visuali più efficace. Abbiamo la possibilità di trattare gli attributi visuali in maniera più dettagliata, c'è all'interno del modulo SI un tipo di dato inventato per estrapolare dall'immagine il colore medio, l'istogramma del colore piuttosto che una lista di feature. Abbiamo anche il concetto di posizione, non più in maniera non disgiunta, ma possiamo pensare anche la posizione solo per il colore, la shape o la texture. Per il caricamento si usa un loop (perchè abbiamo tanti immagini), per la sorgente usiamo il file system (è l'oggetto directory). Anche qui il caricamento avviene tramite un buffer che usai metodo dell'oggetto LOB (large object). Una volta che ha caricato questo oggetto, questo viene utilizzato per istanziare la riga all'interno della tabella.

Vediamo come estrarre delle informazioni. Proviamo a selezionare l'immagine 1, prendiamo la foto con id 1 e la inseriamo in un oggetto myimage, il quale ha un metodo InitFeatures che inizializza le feature visuali:

```

1 DECLARE
2   myimage SI_StillImage;
3 BEGIN
4   SELECT product_photo INTO myimage FROM SI_MEDIA
5   WHERE product_id = 1 FOR UPDATE;
6   myimage.SI_InitFeatures;
7   UPDATE SI_MEDIA
8   SET product_photo = myimage where product_id=1;
9   DBMS_OUTPUT.PUT_LINE('Image feature caching enabled');
10  COMMIT;
11 END;

```

vediamo come cambiare il formato di un'immagine, prendiamo l'immagine 1, avendola inizializzata possiamo estrarre il formato con la funzione SI_Format, chiamo un metodo

ChangeFormat che mi permette di cambiare il formato:

```
1 DECLARE
2   origformat VARCHAR2 (10);
3   newformat  VARCHAR2 (10);
4   myimage SI_StillImage;
5   currentformat VARCHAR2 (10);
6 BEGIN
7   SELECT product_photo INTO myimage FROM SI_MEDIA
8   WHERE product_id = 1 FOR UPDATE;
9   origformat := myimage.SI_Format;
10  DBMS_OUTPUT.PUT_LINE('Original format is ' || origformat);
11  newformat := 'WBMP';
12  myimage.SI_ChangeFormat(newformat);
13  INSERT INTO SI_MEDIA (product_id, product_photo) VALUES (44, myimage);
14  SELECT product_photo INTO myimage FROM SI_MEDIA
15  WHERE product_id = 44;
16  currentformat := myimage.SI_Format;
17  DBMS_OUTPUT.PUT_LINE('New format is ' || currentformat);
18  COMMIT;
19 END;
20 /
```

Lo STILL image lo possiamo usare usando i metodi, usando il get e il set per modificare gli attributi. In realtà supporta anche un approccio procedurale, usa delle procedure ad hoc per fare le stesse cose che fanno le chiamate ai metodi. Vediamo l'esempio di prima con la procedura. Prendo il formato dell'immagine 1, la conversione del formato la facciamo con la procedura SI_ConvertFormat che prende l'oggetto immagine e il nuovo formato:

```
1 DECLARE
2   origformat VARCHAR2 (10);
3   newformat  VARCHAR2 (10);
4   myimage SI_StillImage;
5   currentformat VARCHAR2 (10);
6 BEGIN
7   SELECT product_photo INTO myimage FROM SI_MEDIA
8   WHERE product_id = 1 FOR UPDATE;
9   origformat := myimage.SI_Format;
10  DBMS_OUTPUT.PUT_LINE('Original format is ' || origformat);
11  newformat := 'GIF';
12  SI_ConvertFormat(myimage, newformat);
13  INSERT INTO SI_MEDIA (product_id, product_photo) VALUES (55, myimage);
14  SELECT product_photo INTO myimage FROM SI_MEDIA
15  WHERE product_id = 3 FOR UPDATE;
16  currentformat := myimage.SI_Format;
17  DBMS_OUTPUT.PUT_LINE('New format is ' || currentformat);
18  COMMIT;
19 END;
```

C'è un metodo per fare il thumbnail. Prendo un metodo che ottiene l'altezza e la larghezza e poi uso il metodo Thumbnail:

```
1 DECLARE
2   myimage SI_StillImage;
3   myThumbnail SI_StillImage;
4   height number;
```

```

5 width number;
6 BEGIN
7 SELECT product_photo INTO myimage FROM SI_MEDIA WHERE product_id = 1;
8 width := myimage.SI_width;
9 height := myimage.SI_height;
10 DBMS_OUTPUT.PUT_LINE('Height is ' || height || ' pixels. ');
11 DBMS_OUTPUT.PUT_LINE('Width is ' || width || ' pixels. ');
12 myThumbnail := myimage.SI_Thumbnail;
13 width := myThumbnail.SI_width;
14 height := myThumbnail.SI_height;
15 DBMS_OUTPUT.PUT_LINE('Height is ' || height || ' pixels. ');
16 DBMS_OUTPUT.PUT_LINE('Width is ' || width || ' pixels. ');
17 END;
18 /

```

Lavoriamo con gli attributi visuali. Il primo è l'average color. Il tipo di dato SI_AverageColor i cui attributi sono RGB, lo posso usare per creare un colore, una possibile idea di questa funzione potrebbe essere fare ricerca per colore tra le immagini. Definisco il colore, prendo l'averageColor dal colore e lo andiamo ad inserire in una tabella:

```

1 DECLARE
2 myColor SI_Color;
3 myAvgColor SI_AverageColor;
4 BEGIN
5 myColor := NEW SI_COLOR(null, null, null);
6 myColor.SI_RGBColor(10, 100, 200);
7 myAvgColor := NEW SI_AverageColor(myColor);
8 INSERT INTO SI_MEDIA (product_id, average_color) VALUES (75,
9 myAvgColor);
10 COMMIT;
11 END;
12 /

```

Lo stesso codice posso usarlo per creare l'averagecolor di un'immagine che ho già:

```

1 DECLARE
2 myimage SI_StillImage;
3 myAvgColor SI_AverageColor;
4 BEGIN
5 SELECT product_photo INTO myimage FROM SI_MEDIA WHERE product_id=1;
6 myAvgColor := NEW SI_AverageColor(myimage);
7 END;
8 /

```

Proviamo a fare lo score, uso la funzione SI_Score che viene usato per calcolare le distanze, in questo caso calcolo lo score tra l'averageColor dall'immagine 1 e l'averageColor dell'immagine 2 (tirato fuori automaticamente):

```

1 DECLARE
2 score DOUBLE PRECISION;
3 myimage SI_StillImage;
4 myotherimage SI_StillImage;
5 myAvgColor SI_AverageColor;
6 BEGIN
7 SELECT product_photo INTO myimage FROM SI_MEDIA WHERE product_id=1;
8 myAvgColor := NEW SI_AverageColor(myimage);

```

```

9  SELECT product_photo INTO myotherimage FROM SI_MEDIA
10 WHERE product_id=2;
11 score := myAvgColor.SI_Score(myotherimage);
12 DBMS_OUTPUT.PUT_LINE('Score is ' || score);
13 END;
14 /

```

Estraiamo il colorHistogram e lo andiamo a popolare nell'attributo che avevo nella tabella:

```

1 DECLARE
2   myColorHist SI_ColorHistogram;
3   myimage SI_StillImage;
4 BEGIN
5   FOR x in 1..16 LOOP
6     -- Select a product_photo from the si_media table:
7     SELECT product_photo INTO myimage FROM SI_MEDIA WHERE product_id=x FOR
8     UPDATE;
9     -- Extract the color histogram from the source image:
10    myColorHist := NEW SI_ColorHistogram(myimage);
11    -- Update the color_histogram column with the extracted value:
12    UPDATE SI_MEDIA SET color_histogram = myColorHist WHERE product_id=x;
13    COMMIT;
14  END LOOP;
15 END;
16 /

```

Una volta che abbiamo generato il colorHistogram lo possiamo utilizzare per calcolare gli score. Prendo i due istogrammi e li confronto. Chiamo il metodo Score dall'oggetto che ho creato estraendo l'istogramma dell'immagine 1 e gli passo l'immagine 2, lui dentro estrarrà l'istogramma e ne calcolerà la distanza:

```

1 DECLARE
2   myClrHistConstructed SI_ColorHistogram;
3   myImage SI_StillImage;
4   myImage2 SI_StillImage;
5   score DOUBLE PRECISION;
6 BEGIN
7   -- Get a stored image:
8   SELECT product_photo,color_histogram INTO myimage,myClrHistConstructed
9   FROM SI_MEDIA WHERE product_id=1;
10  SELECT product_photo INTO myimage2 FROM SI_MEDIA WHERE product_id=4;
11
12  score := myClrHistConstructed.SI_Score(myImage2);
13  DBMS_OUTPUT.PUT_LINE('Score is ' || score);
14 END;
15 /

```

Proviamo a combinare alcuni attributi visuali. Vogliamo creare un set di features, da tutte e 16 le foto creo l'averageColor, l'istogramma, la posizione del colore e la texture. Poi creo una feature list in cui gli dico che voglio creare dei confronti pesando le features:

```

1 DECLARE
2   myimage SI_StillImage;
3   myAvgColor SI_AverageColor;
4   myColorHist SI_ColorHistogram;

```

```

5 myPosColor SI_PositionalColor;
6 myTexture SI_Texture;
7 myFeatureList SI_FeatureList;
8 BEGIN
9 FOR x in 1..16 LOOP
10  SELECT product_photo INTO myimage FROM si_media WHERE product_id=x
11 FOR
12  UPDATE;
13  myAvgColor := NEW SI_AverageColor(myimage);
14  myColorHist := NEW SI_ColorHistogram(myimage);
15  myPosColor := NEW SI_PositionalColor(myimage);
16  myTexture := NEW SI_Texture(myimage);
17  myFeatureList := new SI_FeatureList(myAvgColor,0.25,myColorHist,0.35,
18  myPosColor,0.10, myTexture,0.5);
19  UPDATE si_media SET feature_list = myFeatureList WHERE product_id=x;
20  COMMIT;
21 END LOOP;
22 END;
23 /

```

Una volta che abbiamo la lista possiamo usarla per fare i nostri score. Prendo l'immagine 1, ne prendo la feature list, prendo l'immagine 3 e poi uso il metodo Score dell'oggetto feature list passando dentro l'immagine 2:

```

1 DECLARE
2  myimage SI_StillImage;
3  myFeatureList SI_FeatureList;
4  score DOUBLE PRECISION;
5 BEGIN
6  --Check to see if stored feature list is accurate for the stored image:
7  SELECT feature_list INTO myFeatureList FROM si_media
8  WHERE product_id=1;
9  SELECT product_photo INTO myimage FROM si_media
10 WHERE product_id=3;
11 -- The score will be 0 if the feature list is accurate for the stored
12 score := myFeatureList.SI_Score(myimage);
13 DBMS_OUTPUT.PUT_LINE('Score is ' || score);
14 END;

```

La stessa cosa la posso fare nel loop in cui confronto la feature list dell'immagine 1 con la feature list di tutte le altre:

```

1 DECLARE
2  myimage SI_StillImage;
3  myFeatureList SI_FeatureList;
4  score DOUBLE PRECISION;
5 BEGIN
6 FOR x in 1..16 LOOP
7  --Check to see if stored feature list is accurate for the stored image:
8  SELECT feature_list INTO myFeatureList FROM si_media
9  WHERE product_id=1;
10 SELECT product_photo INTO myimage FROM si_media
11 WHERE product_id=x;
12 -- The score will be 0 if the feature list is accurate for the stored
13 score := myFeatureList.SI_Score(myimage);
14 DBMS_OUTPUT.PUT_LINE('Score is ' || score);

```

```
15 END LOOP ;  
16 END ;
```

8.16 IRS

Queste operazioni sono state espanse con un sistema di retrieval content-base. Il concetto di similarità rimane sempre quello, presi due feature vector ne calcola la distanza.

I features vector che abbiamo visto fino ad adesso sono dei metadati che rappresentano l'immagine, potremmo però pensare di usare delle feature visuali.

8.16.1 Semantic Gap

Se faccio le query da un punto di vista visuale se mi appoggio solo sugli attributi visuali un arancia ed un tramonto sono due immagini simili, perchè hanno come colore predominante il rosso, la shape potrebbe essere simile ma in realtà non sono la stessa cosa, rappresentano due cose completamente differenti.

In realtà questo è valido anche in qualunque motore di ricerca in cui dobbiamo andare a recuperare delle immagini. Il confronto tra immagini in google non mi ridà un immagine simile a livello di colore, shape o texture. Non c'è più una ricerca visuale ma una ricerca semantica.

8.16.2 MPE System

Minimum Probability of Error Image Retrieval. Fino all'anno scorso Google funzionava male con le immagini private, questo perchè le immagini erano confrontate anche sulla loro semantica (lì c'erano degli snowboard). Uno dei lavori fondanti sulla logica della ricerca semantica è il Minimum Probability of Error Image Retrieval

8.16.3 Semantic Retrieval

La ricerca si basa sulla semantica dell'immagine, l'immagine deve dirmi qualcosa. Se c'è un addestramento iniziale posso colmare il semantic gap. Per fare un sistema efficace ed efficiente di image retrieval devo partire da un db di immagini la cui semantica sia stato descritta tramite delle annotazioni. Devo avere un db di immagini con un vocabolario già definito che descrive il contenuto dell'immagine, abbiamo un set di lable che descrivono da un punto di vista semantico l'immagine. A questo punto torniamo ai testi, abbiamo delle lable con i pesi. Ritorniamo al text retrieval.

Le query sulla visual extarction non funzionano bene, dobbiamo recuperare la semantica in qualche modo, non è facile perchè è indipendente dal concetto di labeling.

Il lavoro di minimum probability of error retrieval ci dice che ogni immagine da un punto di vista visuale posso creare una griglia, da ogni griglia estraggo i feature vector di caratteristiche tecniche. non sono altro che la distribuzione di vettori nello spazio n-dimensionale. Posso raggrupparli, faccio dei cluster in cui ho delle distribuzioni, questi pattern di queste distribuzioni li posso usare per fare il matching con le nostre query.

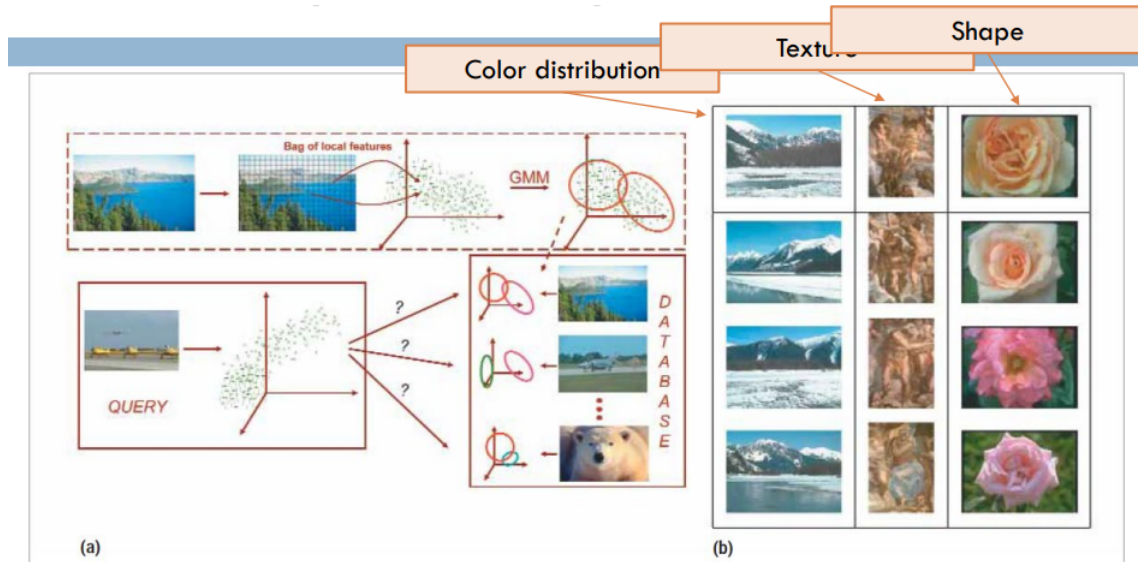


Figure 1. Minimum probability of error retrieval. (a) MPE retrieval architecture. The system decomposes images into bags of local features and characterizes them by their distributions on the feature space. Database images are ranked by posterior probability of having generated the query features. (b) Retrieval results. Each column shows the three best matches (among 1,500) to the query image shown at the top.

Il sistema deve partire dal mapping per capire come legare la label dalle feature visuali. Il corpus di addestramento è debolmente etichettato, vale a dire:

- L'assenza di un'etichetta in una didascalia non significa necessariamente che il concetto visivo associato sia assente dall'immagine,
- Non è noto quali regioni dell'immagine siano associate a ciascuna etichetta.

Ad esempio, un'immagine contenente "cielo" potrebbe non essere annotata esplicitamente con tale etichetta e, quando lo è, non è disponibile alcuna indicazione su quali pixel dell'immagine rappresentino il cielo.

8.16.4 Query by semantic example

Dato il feature vector posso fare un confronto di similarità usando il vettore delle lable, faccio quindi query semantiche non visuali.

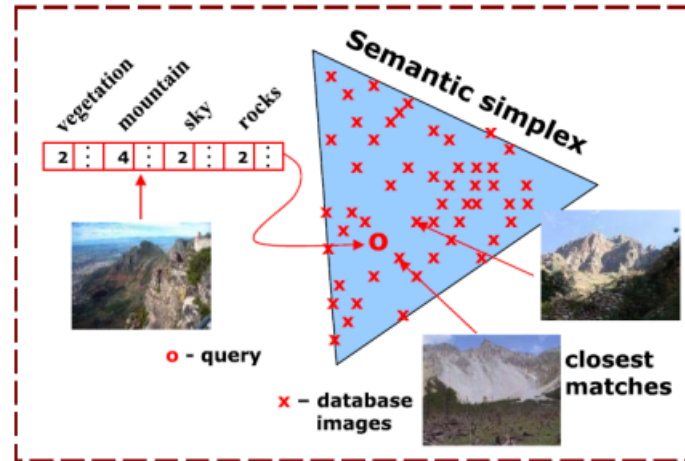


Fig. 1.7 Query by semantic example. Images are represented as vectors of concept probabilities, i.e., points on the semantic probability simplex. The vector computed from a query image is compared to those extracted from the images in the database, using a suitable similarity function. The closest matches are returned by the retrieval system.

8.16.5 Query in natural language

Posso fare una query testuale, prendo tutte le immagini che hanno un feature vector con montagna e cielo. L'annotazione mi ha permesso di integrare la semantica del sistema di text retrieval. Per poi fare confronti tra termini indici.

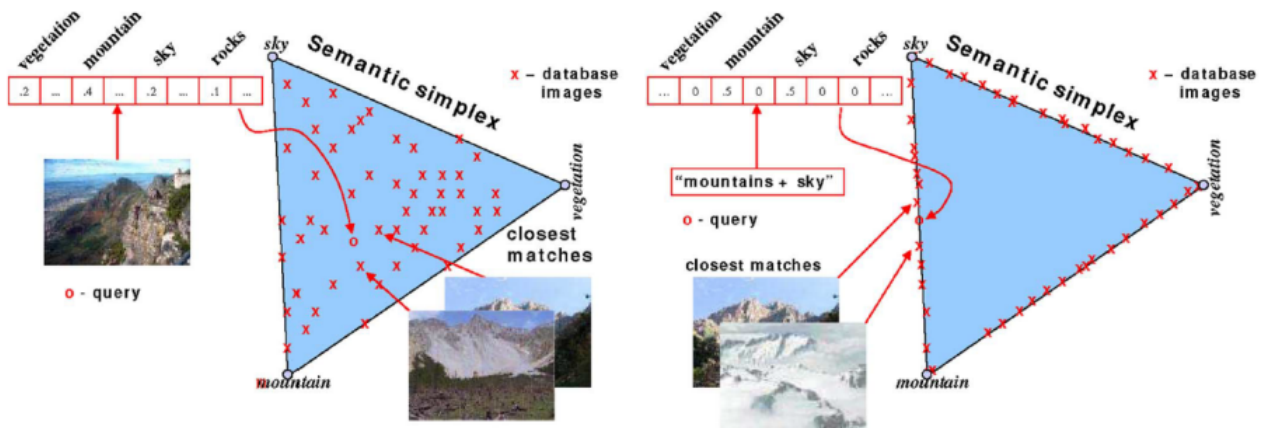


Fig. 2. Semantic image retrieval. Left: Under QBSE the user provides a query image, probabilities are computed for all concepts, and the image represented by the concept probability distribution. Right: Under the traditional SR paradigm, the user specifies a short natural language description, and only a small number of concepts are assigned a nonzero posterior probability.

8.16.6 What next?

É la stessa cosa che si fa ad esempio con Dalle, si va a taggare in maniera testuale delle parti dell'immagine.

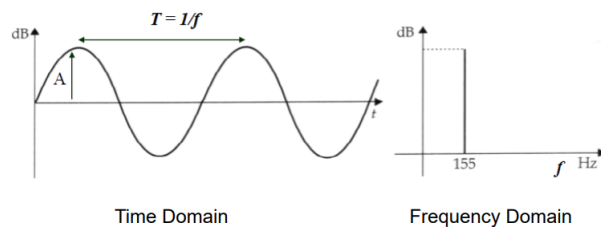
Chapter 9

Digital Audio

Cos'è il suono? A noi interessano le caratteristiche per recuperare l'oggetto suono. È un'onda. È un'onda sonora che viene catturata dall'orecchio, poi tramite l'apparato uditivo viene trasformata in un segnale elettrico che viene mandato al cervello per il riconoscimento del suono.

Le caratteristiche del suono sono importanti per capire come digitalizzarlo e comprimerlo. Il suono è abbastanza soggettivo, molto dipende dall'età e da come abbiamo trattato le nostre orecchie.

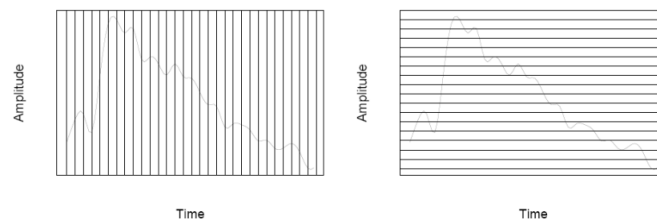
Il dominio di tempo che viene misurato in decibel, mentre il dominio in frequenza che rappresenta gli hertz:



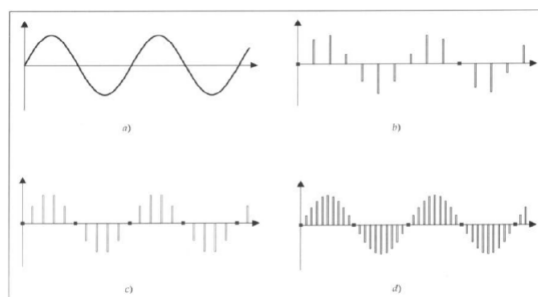
Queste caratteristiche ci servono per capire data l'onda come possiamo digitalizzarla, dobbiamo discretizzarla e sulla base di questa discretizzazione fare le nostre strategie di recupero.

9.1 Campionamento e Quantizzazione

Si dice che di un suono viene fatto un campionamento, dividiamo in intervalli il suono. Il campionamento in frequenza ci dice quando andare a recuperare un campione per poi andarlo a rappresentare. La frequenza si misura in hertz, noi dobbiamo decidere quanti campione prendere al secondo. Sulla base di questo definiamo il teorema di Nyquist che ci dice come campionare il suono in modo corretto. Mentre la quantizzazione ci dice il campionamento in termini di ampiezza, con che precisione preleviamo il campione?

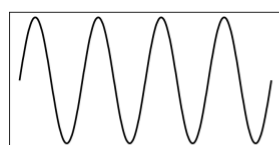


Noi consideriamo il suono puro, non una sovrapposizione di onde, chiaramente il caso reale è una moltiplicazione di onde. Qual'è il campionamento che ci interessa? cosa vuol dire fare una quantizzazione uniforme o meno?

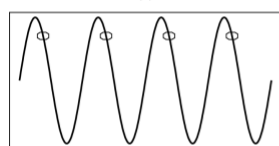


9.2 Teorema di Nyquist

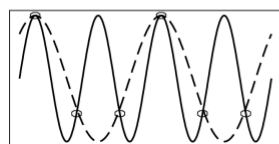
Il suono che vogliamo digitalizzare è una singola frequenza, se facessimo un campionamento alla stessa esatta frequenza dell'audio originale vorrebbe dire campionare sempre alla stessa frequenza, questo risulterebbe in un suono nullo. Se campionassi ad 1.5 otterrei un'onda risultante diversa:



(a): A single frequency.

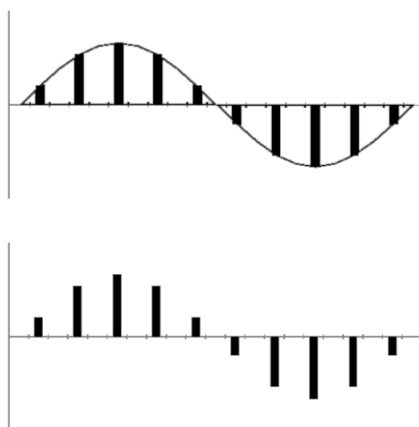


(b): Sampling at exactly the frequency produces a constant.



(c): Sampling at 1.5 times per cycle produces an *alias* perceived frequency.

Quando abbiamo una ricostruzione digitale dell'onda originale sbagliata si dice che abbiamo un effetto alias, ovvero abbiamo fatto un errore di campionamento. Noi vorremmo campionare in questo modo, in modo da poter ricostruire l'onda originale correttamente:



Il teorema di Nyquist dice che il minimo campionamento necessario è 2 volte la frequenza iniziale, $f_c \geq 2f_{max}$. Ad esempio se abbiamo una frequenza iniziale di 4000 Hz noi camperemo ad 8000Hz. Sappiamo che l'uomo sente da 20k Hertz in poi fino a 40k, tant'è che la frequenza di campionamento standard è 44,1k hertz.

- Se campioniamo sopra la frequenza di Nyquist avremo un oversampling, che porta ad uno spreco di banda
- Se campioniamo sotto abbiamo il problema degli alias

La frequenza di nyquist è la metà della rate di nyquist (è la frequenza originale). Poiché sarebbe comunque impossibile recuperare frequenze superiori alla frequenza di Nyquist, la maggior parte dei sistemi dispone di un filtro antialiasing che limita il contenuto di frequenza in ingresso al campionatore a un intervallo pari o inferiore alla frequenza di Nyquist.

Ci dice che l'effetto alias è sempre dato dal campionamento meno la frequenza vera:

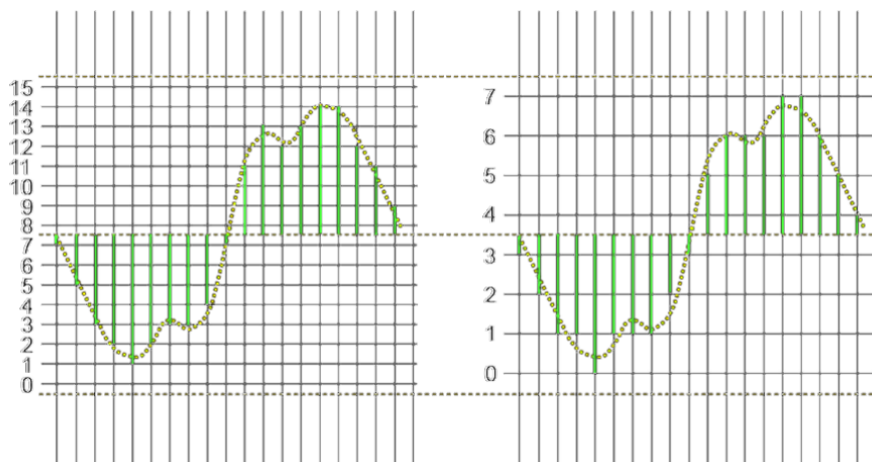
$$f_{alias} = f_{sampling} - f_{true}$$

9.3 Quantizzazione

Questo ci permette di dire quanti campioni prendere dall'onda originale, è la discretizzazione dell'ampiezza. A questo punto ogni campione dobbiamo discretizzarlo, quanti bit usiamo per rappresentare ogni campione. È chiamata quantizzazione in ampiezza.

L'intensità di un suono si misura in decibel, è alla base del Sound Pressure Level che viene espresso in termini logaritmici, perchè il range di ampiezza del suono è molto alto e la rappresentazione logaritmica ci viene in aiuto, $SPL = 20 \log(\frac{p}{p_0})$. p_0 è la minima soglia udibile e viene misurata in decibel.

Chiaramente più bit utilizzo più rappresento correttamente il campione, chiaramente ho sempre un errore di quantizzazione, più bit uso più riduco questo errore.



La precisione di quantizzazione viene misurata tramite la signal to noise ratio che permette di rappresentare il rapporto tra la massima ampiezza propria del segnale e l'ampiezza del rumore presente (statico o bianco) sovrapposto al segnale. È anche definito come il rapporto tra la potenza del segnale e quella del rumore.

Si dice che un suono 10 volte il rumore di sottofondo si misura come $20 \cdot \log(10) = 20dB$.

9.3.1 Merito dei dB

In scala logaritmica i fattori moltiplicativi possiamo trasformarli in somme. La scala logaritmica dice che se duplichiamo l'intensità di un suono aumentiamo di 3dB, l'aumento non è esponenziale, ma è sempre logaritmico.

9.3.2 Signal to noise ratio (SNR)

Come possiamo matematicamente rappresentare l'ampiezza, è una misura della qualità del segnale. È un rapporto tra il volume del segnale e il volume del rumore. 20 per il rapporto tra il volume del segnale e del rumore:

In decibel:

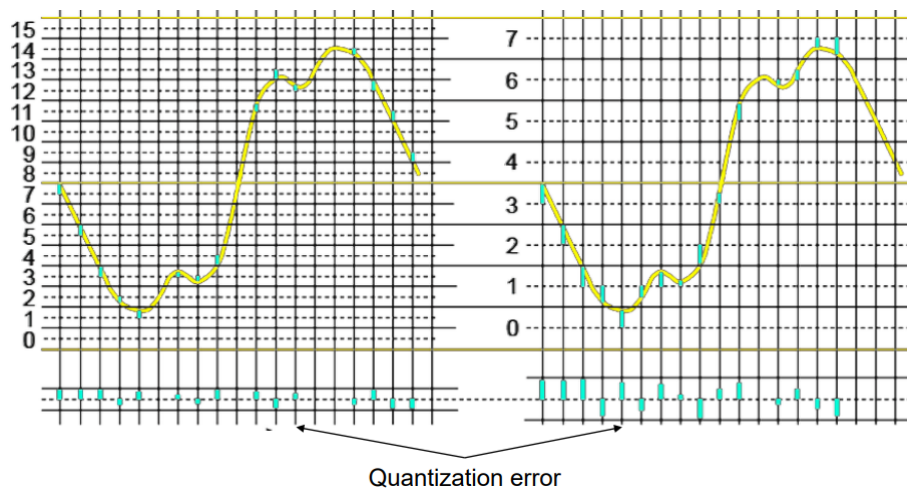
$$SNR = 10 \log_{10} \frac{V_{signal}^2}{V_{noise}^2} = 20 \log_{10} \frac{V_{signal}}{V_{noise}}$$

Fondamentalmente ogni bit in più o in meno che usiamo vuol dire avere un contributo di circa 6dB. Se uso 16 bit abbiamo $16 \cdot 6 = 96dB$.

9.3.3 Signal to Quantization Noise Ratio (SQNR)

Otteniamo anche rumore nel discretizzare il campione, perchè usiamo un tot numero di bit. Questo errore si chiama errore di quantizzazione, viene chiamato rumore.

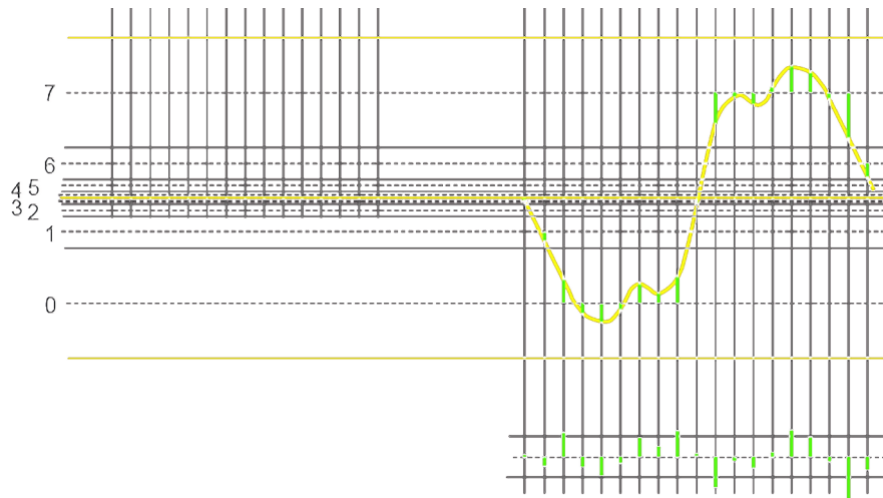
È al massimo il numero di bit che abbiamo utilizzato. La differenza tra il valore effettivo del segnale analogico per il particolare tempo di campionamento e il valore dell'intervallo di quantizzazione più vicino. Al massimo, questo errore può essere pari alla metà dell'intervallo.



9.4 Linear and Non-linear Quantization

Possiamo sfruttare le caratteristiche dell'orecchio umano per garantire una qualità audio migliore usando quei bit? Sì, si può usare una quantizzazione non lineare (uniforme). La quantizzazione lineare usa un valore uniforme di bit per ogni campione.

Potrei dire di quantizzare in maniera differente rispetto alla frequenza del campione, quindi con campioni di frequenza che sono più facilmente udibili dall'orecchio umano uso più bit, per le frequenze più alte o basse uso meno bit.



9.4.1 Storage of audio in file in Kb

La frequenza di campionamento per la durata in secondi per il numero di bit usati per rappresentare ciascun campione, poi se è stereo raddoppio:

$$(f_c \cdot D \cdot N_{bc} \cdot N_c) / (8 \cdot 1024)$$

9.5 Digital Audio Format

Usiamo uno standard, avremo un sampling rate, numero di canale, numero di bit per ogni campione. Poi potremmo avere un header o meno, che contiene informazioni che sono metadati piuttosto che il copyright.

Extension	Source	Parameters	Header
au,snd	NeXT, Sun	Rate, channels, coding	Y
aif(f), AIFF	Apple, SGI	Rate, channels, quantization	Y
aif(f), AIFFC	Apple, SGI	AIFF with compression	Y
wav, WAVE	Microsoft	Rate, channels, quantization	Y
voc	Creative	Rate	Y
iff, IFF/BSVX	Amiga	Rate, channels, instruments info	Y
mp3, mp4	MPEG	Rate, channels, quality samples	Y
sf	IRCAM	Rate, channels, coding	Y
ra	Real	Rate, channels, quality samples	Y
snd, fssd	Mac, PC	-	N
snd	Amiga	-	N

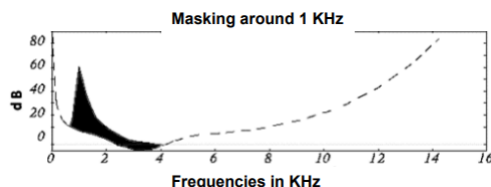
9.6 Compressione

Gli algoritmi di compressione sfruttano la logica dell'orecchio umano. Queste tecniche di compressione si basano sulla suddivisione dello spettro audio in bande di frequenza di dimensioni adeguate alla selettività del nostro sistema uditivo.

Fanno utilizzo di strategie di masking, sono utili a livello di trasformazione di audio analogico in digitale abbiamo dato, se facessimo una quantizzazione non lineare potremmo fare meglio, c'è un'altra logica, psicoacustica, dell'orecchio umano che potremmo utilizzare per migliorare il campionamento, ci sono dei campioni che possiamo buttare via perché tanto non li sentiamo.

9.6.1 Masking

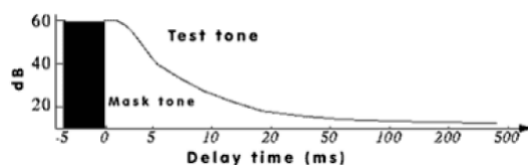
Sono tecniche di frequenza, alcuni suoni ad una determinata frequenza mascherano dei campioni nell'intorno. Mettiamo di emettere un masking tone alla frequenza di 1KHz ad un determinato volume 60dB, è stato dimostrato quel cono nero rappresenta tutto ciò che il suono di mascheramento va a nascondere.



Di conseguenza possiamo non memorizzare tutte le frequenze intorno alla frequenza di mascheramento.

Mascheramento temporale

La procedura fisica (dell'orecchio) che trasforma il suono in segnale elettrico ci mette un po'. Quindi un segnale di 1kHz a 60dB nasconde tutto quello che succede nei successivi 5ms, questo perché deve riassettersi il sistema di percezione meccanico. Chiaramente è inutile andare a campionare sotto questi tempi.



Sovrapporre queste tecniche di masking, possiamo dire che dato un particolare suono potremmo evitare di campionare alcuni campioni perché sono inudibili all'orecchio umano. Questo fenomeno è usato per fare compressione audio, soprattutto nell'MPEG3.

9.7 MPEG

È uno standard di compressione, sta alla base di MP3, ne ha parlato pochissimo, È quello più utilizzato ed è stato migliorato nelle versioni per quanto riguarda la compressione, in quanto utilizza la logica del masking che abbiamo visto.

L'MPEG ha sempre un header, le caratteristiche sono:

- Header
- Quantizzazione non lineare
- Modello psico acustico per la compressione

9.8 MIDI

É un linguaggio di programmazione, il numero di bit rappresentano il suono a livello di linguaggio. É un linguaggio di scripting che serve per rappresentare un suono, lo spazio occupato è bassissimo, perchè è un testo.

Ho una sequenza di bit, ogni gruppo di bit rappresentano qualcosa. I primi 16 rappresentano il canale (che tipi di strumenti vengono utilizzati), i vari bit descrivono il suono, tutto quello che rappresenta la descrizione del suono (frequenze, ampiezze, tipo di strumento utilizzato). Fondamentalmente è un linguaggio che descrive nel dettaglio il suono, abbiamo il:

- sequencer: sintetizza il suono prelevandolo da uno strumento per rappresentarlo da un punto di vista dello scripting

Chapter 10

Digital Video

Un video è un insieme di immagini messe una dopo l'altra.

10.1 Frame

Il problema della discretizzazione del video è il problema della memorizzazione, abbiamo tante immagini, quindi dobbiamo capire come memorizzarle per non occupare troppo spazio. Il framerate sono quanti frame vengono visualizzati per secondo.

10.2 Codifica

Partiamo dal framerate, rappresenta quanti frame abbiamo per secondo, possiamo calcolare qual'è la dimensione. Dobbiamo partire da delle considerazioni fisiche, anche per il video abbiamo una ridondanza delle informazioni, dato un pixel i pixel nell'intorno non cambiano tanto. O anche una cella nei secondi successivi non cambia di tanto (dipende in realtà, se c'è un cambio di scena sì).

10.2.1 La risoluzione video

Cambiano dall HDTV (1920x1080) fino alla VGA (720x480)

10.2.2 Aspect Ratio

Va bene la risoluzione ma possiamo avere anche una rappresentazione da 4:3 a 16:9. Poi esistono anche degli inframezzi che scalano l'immagine o mettono la banda nera.

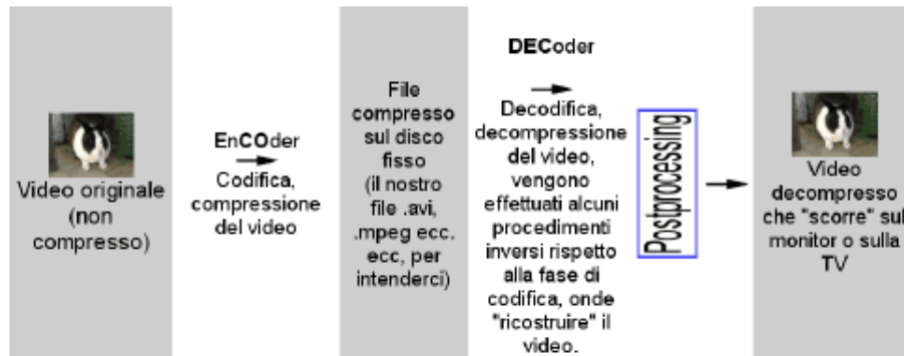


10.2.3 Interlacciamento

Viene utilizzata per presentare il video. Nei televisori vecchi si notava che si aveva un caricamento a fette del video legata all'interlacciamento.

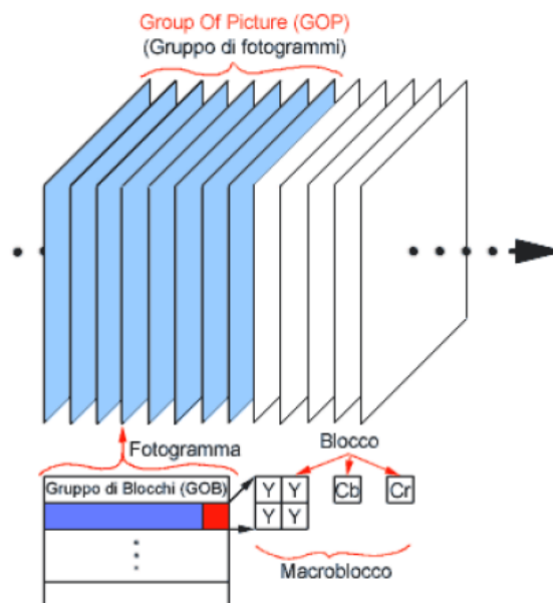
10.3 Compressione

Partiamo da un decoder e encoder che ne deve definire una compressione date le logiche di compressione temporale e spaziale.



La compressione può essere senza perdita o con perdita. Se è con perdita non possiamo invertire completamente il processo ma avremo una compressione migliore, se invece è senza perdita avremo una compressione peggiore ma potremo invertire completamente il processo di compressione.

Le logiche di codifica partono da questi concetti, ogni immagine è divisa in blocchi, poi consideriamo blocchi di frame (che possono fare parte di un'unica scena, un GOP rappresenta un gruppo di frame che posso considerare insieme).



10.3.1 Ridondanza spaziale

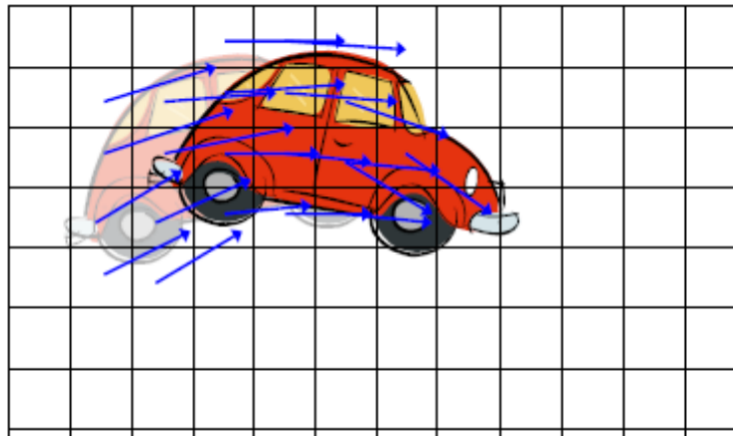
La caratteristica di ridondanza spaziale è legata al fatto che il pixel adiacente non cambia un granchè. L'idea è di tenere soltanto l'informazione utile.

10.3.2 Ridondanza temporale

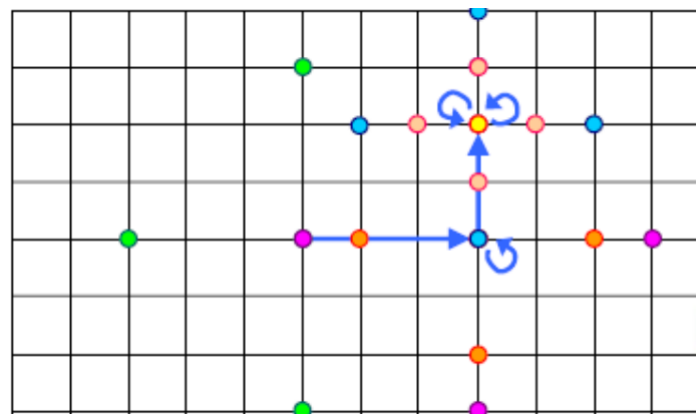
Dato un gruppo di pixel nella stessa scena non cambiano un granchè. Correlazione non solo tra i pixel dello stesso fotogramma, ma anche tra i pixel di fotogrammi adiacenti.

10.3.3 Motion Vector

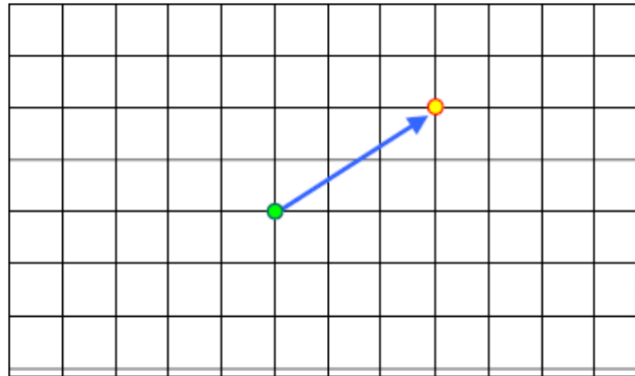
Data una cella, in quelle successive le cose non cambiano tanto perchè andare a codificare tutti i frame? prendo un frame di riferimento (k-frame) e cerco di capire cosa succede nei frame successivi, data questa cella come si andrà a spostare nei frame successivi. La ricostruzione finale non può essere precisa.



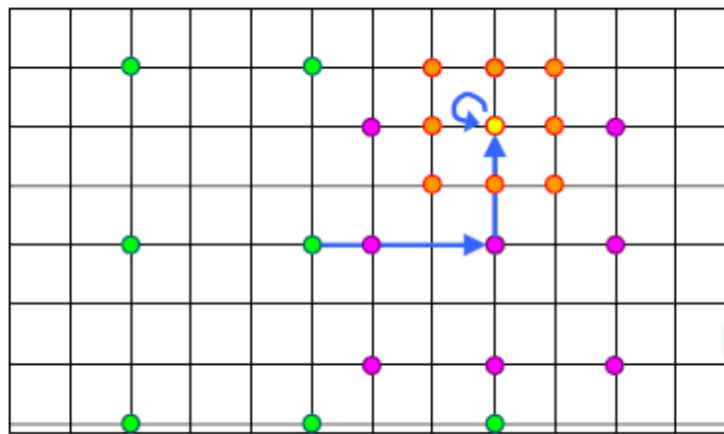
Come faccio a calcolare come cambia il movimento della cella da un frame a quello successivo, lo fa una stima del blocco, l'idea è di cercare il blocco nel frame successivo e stimare il motion vector. Ci sono diverse tecniche per poterlo fare, vediamo quella più semplice, che viene chiamata 2D log. Dato un punto devo stimare cosa avviene nei frame successivi, ovvero dove me lo ritroverò questo punto? faccio delle stime per definire i movimenti, lo faccio solo su alcuni blocchi perchè sfrutto la ridondanza spaziale, parto da una cella di riferimenti e verifico a campione cosa succede nei frame successivi. Considero una griglia 3x3, dai questi 4 punti qual'è quello che mi stima il movimento, devo cercare nei frame successivi dove troverò questo punto (quello viola). A questo punto costruisco un'altra area di ricerca, e anche qui vedo nei frame successivi dove sarà il punto. Uso queste griglie di ricerca per capire dove si muoverà il punto.



A questo punto posso stimare il vettore di movimento del blocco nei frame successivi.



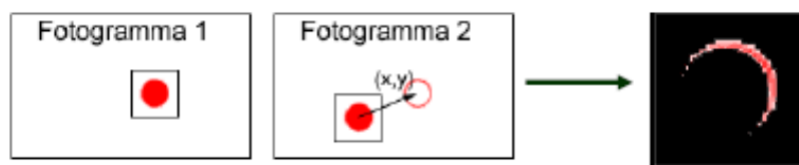
Posso aumentare la griglia di risoluzione, potrei usare 9 punti, aumentando i punti risoluzione aumenterebbe il vettore di movimento.



Sto stimando il movimento all'interno del video, non sto considerando movimenti di camera o cose particolari.

10.3.4 Motion Estimation (ME)

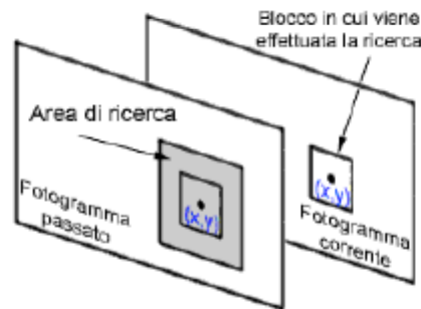
Questo motion vector è associato ad ogni blocco, sono due numeri che ci dicono dove sto andando da un frame 1 ad un frame 2.



10.3.5 Motion Compensation (MC)

Usare il motion vector significa che memorizzo il frame, poi in quelli successivi memorizzo solo i movimenti. Dopo devo compensare tutto, dato il frame di partenza verifico i movimenti e poi faccio l'inverso. Per ricostruire il secondo frame lo faccio dal primo applicando

il movimento. Stiamo guadagnando in termini di spazio, perchè non memorizzo tutti i frame (solo i key frame), tutti gli altri vengono stimati a partire da questi.



10.3.6 Frame I/P/B

Lo standard MPEG classifica i frame in tre tipi:

- Il I-frame (key frame) è quello di riferimento viene chiamato I-frame (independent frame)
- Il P-frame (predicted frame) che viene calcolato a partire dal frame precedente
- Il B-frame (bidirectional frame) che viene calcolato considerando il frame precedente e successivo, è quello più piccolo di tutti dal punto di vista della dimensione

I frame P e B sono dovuti agli algoritmi applicati per il calcolo del motion vector.

I key frame possono essere quelli di inizio e cambio scena, il primo frame di una scena deve essere un I frame. Il P frame è calcolato dal frame precedente ed è legato al motion vector per la stima di movimento e la compensazione, lo devo costruire, quindi c'è un costo computazionale da mettere in gioco per ricostruirlo.

n° fotogramma	1	2	3	4
tipo di fotogramma:	I	B	B	P

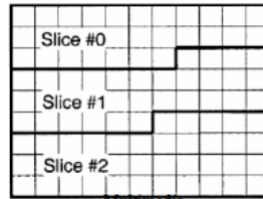
Anche a livello del singolo frame ho questi concetti, ci sono delle cella più rilevanti e altre che vengono calcolate a partire da queste.

10.3.7 MPEG

Come standard abbiamo MPEG con tutte le sue versioni. Dall'MPEG2 la cosa che cambia è che ci sono i profili, posso definire una codifica diversa a seconda del destinatario, codifica ad alto livello e ad alto livello con diverse caratteristiche.

10.3.8 Slice

A volte non vengono trattati i frame nella loro interezza ma vengono divisi a fette, perchè ci sono delle aree più rilevanti di altre, ad esempio il centro del frame è la parte più rilevante. Potrei codificare diversamente queste regioni perchè hanno rilevanza diversa.



10.3.9 Profilo

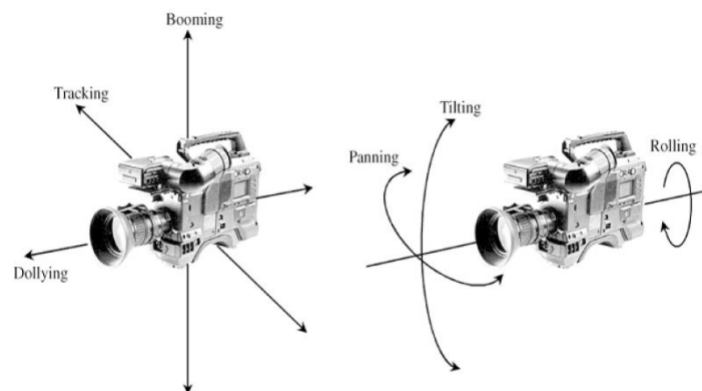
Quali sono le caratteristiche fisiche del video a seconda del device o dell'utente che recupererà il video stesso. Anche a livello di copyright, se pago per un servizio potrei avere diritto a qualità migliore.

Il baseline profile è quello dato a tutti, sono più logiche legate alla trasmissione. L'high profile è dato per maggiore qualità.

10.4 Stabilizzazione

Queste logiche di codifica non considerano che la cattura video può essere soggetta a dei movimenti, la cattura video significa che c'è una videocamera che cattura il video, questa videocamera non era stabile. Questi movimenti sono collaterali, non rappresentano il motion vector, noi dobbiamo partire da un video stabilizzato. La stabilizzazione può essere fatta HW o SW.

Stabilità vuol dire che durante la cattura video sono avvenuti movimenti che rappresentano chi sta catturando il video, potrebbero essere movimenti oscillanti, verticale, rotazione, avvicinamento. Sono movimenti di camera che devo cercare di eliminare tramite un filtering per la stima del mio motion vector, perchè nella compressione non vado a considerare questi movimenti, altrimenti avremmo sicuramente degli errori nel calcolo del motion vector.



10.5 Retrieval

Come faccio a catturare informazioni da un video? vuol dire catturare cosa sto rappresentando. Riesco a catturare:

- Zoom in out

- Cambio prospettiva
- Cambio luce

Perchè sono tutti analisi che posso fare confrontando i frame successivi. Se c'è una grande discrepanza tra un frame e quello successivo è successo qualcosa. Per capire se c'è stato un cambio devo guardare:

- Cambio colore: istogramma colore
- Cambio forme: edge detection
- Cambio macro blocchi o nelle fette con cui ho definito il video

Fare un calcolo di color detection tra un frame e l'altro significa calcolare l'istogramma di colori tra un frame e quello successivo, se sono molto diversi è successo qualcosa.

La stessa cosa con edge detection, se c'è una grande discrepanza di forme tra un frame e quello successivo è successo qualcosa.

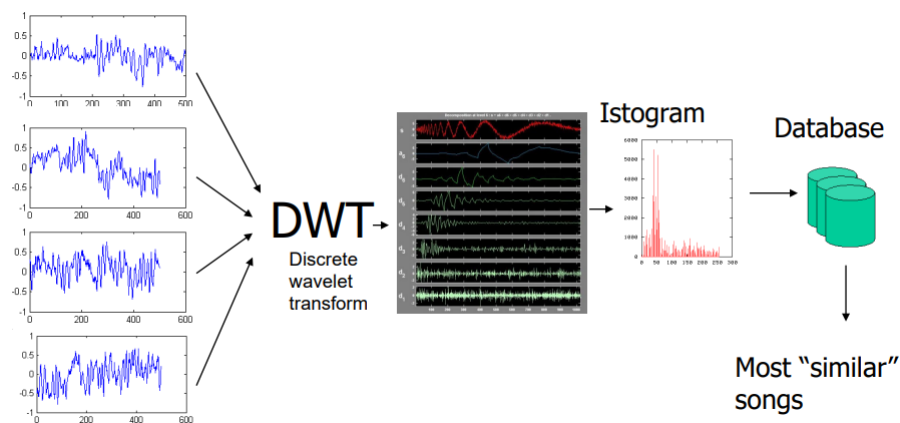
Questo ci permette di capire quando all'interno del video c'è stato un cambio di scena.

Chiaramente devo considerare che la codifica del video non è frame per frame ma c'è la codifica IPB frame, quindi il frame va ricostruito per fare il color detection e l'edge detection. O addirittura il fatto che ho codificato solo delle fette e non tutto il frame.

10.5.1 Audio

Se devo andare a catturare informazioni tecniche è banale (dimensione, frame rate...). Se voglio fare query di contenuto devo basarmi sull'audio retrieval e fare differenze tra frame.

Le tecniche di retrieval audio sono semplici, posso trattare un audio come se fosse un'immagine, ritornerei al problema di confronto di forme. Vengono confrontati le immagini di due suoni (le onde).



Altri sistemi invece segmentavano l'audio per creare dei gruppi all'interno del suono per caratterizzare e segmentare il suono. È più vicino al concetto di contenuto, non riesco a catturare cosa dice una canzone, ma è possibile capire se una canzone è simile ad un'altra. Questo sistema poteva dire:

- Categoria musicale

- Keywords
- Durata
- Altre cose che tira fuori anche Oracle

La cosa carina era quella della segmentazione, trovava i momenti nell'audio (quando non succede niente...).

10.5.2 Oracle

Oracle tratta oggetti audio e video con un sistema semi-automatico di features extraction.

Audio

L'ORDAudio è un oggetto multimediale che ci permette di offrire un insieme di attributi estraibile da un file audio con i relativi metodi di get e set:

```

1 CREATE OR REPLACE TYPE ORDAudio
2 AS OBJECT
3 (-- ATTRIBUTI
4 description VARCHAR2(4000),
5 Source ORDSOURCE,
6 Format VARCHAR2(31),
7 mimeType VARCHAR2(4000),
8 comments CLOB,
9 -- ATTRIBUTI AUDIO
10 Encoding VARCHAR2(256),
11 numberOfChannels INTEGER,
12 samplingRate INTEGER,
13 sampleSize INTEGER,
14 compressionType VARCHAR2(4000),
15 audioDuration I NTEGER,
16 -- COSTRUTTORI
17 STATIC FUNCTION init( ) RETURN ORDAudio,
18 STATIC FUNCTION init(srcType IN VARCHAR2,
19 srcLocation I N VARCHAR2,
20 srcName IN VARCHAR2) RETURN ORDAudio

```

Video

L'ORDVideo è simile, abbiamo una serie di attributi che ci permette di estrarre alcune caratteristiche con tutti i metodi get e set che ne conseguono:

```

1 CREATE OR REPLACE TYPE ORDVideo
2 AS OBJECT (
3 -- ATTRIBUTI
4 description VARCHAR2(4000),
5 source ORDSOURCE,
6 format VARCHAR2(31),
7 mimeType VARCHAR2(4000),
8 comments CLOB,
9 -- ATTRIBUTI VIDEO
10 width INTEGER,

```

```

11 height INTEGER,
12 frameResolution INTEGER,
13 frameRate INTEGER,
14 videoDuration INTEGER,
15 numberOfFrames INTEGER,
16 compressionType VARCHAR2(4000),
17 numberOfColors INTEGER,
18 bitRate INTEGER,
19 -- COSTRUTTORI
20 STATIC FUNCTION init( ) RETURN ORDVVideo,
21 STATIC FUNCTION init(srcType IN VARCHAR2, srcLocation IN VARCHAR2,
    srcName IN VARCHAR2) RETURN ORDVVideo

```

La cosa comoda di questo approccio è che abbiamo un'estrazione di caratteristiche fisiche quasi gratis, caricando un video posso interrogare le sue caratteristiche fisiche. Sono tutte manipolazioni dell'oggetto audio-video che sono facili da fare:

```

1 DROP TABLE media_audio;
2 CREATE TABLE media_audio (
3 media_id NUMBER,
4 annotation VARCHAR2(255),
5 audio ORDSYS.ORDAudio,
6 CONSTRAINT id_media_audio PRIMARY KEY (media_id));
7 COMMIT;
8 DROP TABLE media_video;
9 CREATE TABLE media_video (
10 media_id NUMBER,
11 annotation VARCHAR2(255),
12 video ORDSYS.ORDVVideo,
13 CONSTRAINT id_media_video PRIMARY KEY (media_id));
14 COMMIT;
15 DECLARE
16   lobd blob;
17   fils BFILE;
18   filename VARCHAR2 (64) ;
19 BEGIN
20   FOR x in 1..6 LOOP
21     filename := 'audio' || x || '.mp3';
22     fils := BFILENAME('ORDMEDIADIR',filename);
23     DBMS_LOB.CREATETEMPORARY(lobd, TRUE);
24     DBMS_LOB.fileopen(fils, DBMS_LOB.file_readonly);
25     DBMS_LOB.LOADFROMFILE(lobd, fils, DBMS_LOB.GETLENGTH(fils));
26     DBMS_LOB.FILECLOSE(fils);
27     INSERT INTO MEDIA_AUDIO (media_id, audio)
28     VALUES(x, new ORDSYS.ORDAudio(lobd));
29     DBMS_LOB.FREETEMPORARY(lobd);
30     COMMIT;
31   END LOOP;
32   FOR x in 1..4 LOOP
33     filename := 'video' || x || '.mp4';
34     fils := BFILENAME('ORDMEDIADIR',filename);
35     DBMS_LOB.CREATETEMPORARY(lobd, TRUE);
36     DBMS_LOB.fileopen(fils, DBMS_LOB.file_readonly);
37     DBMS_LOB.LOADFROMFILE(lobd, fils, DBMS_LOB.GETLENGTH(fils));
38     DBMS_LOB.FILECLOSE(fils);
39     INSERT INTO MEDIA_VIDEO (media_id, video)

```

```

40  VALUES(x, new ORDSYS.ORDVideo(lobd));
41  DBMS_LOB.FREETEMPORARY(lobd);
42  COMMIT;
43 END LOOP;
44 END;
45 /
46
47 DECLARE
48  obj ORDSYS.ORDAudio;
49  tempLob CLOB;
50  ctx RAW(64) :=NULL;
51 BEGIN
52  SELECT p.audio INTO obj FROM media_audio p
53  WHERE media_id = 1;
54  DBMS_OUTPUT.PUT_LINE('getting comma separated list of all attributes');
55  DBMS_OUTPUT.PUT_LINE('-----');
56  DBMS_LOB.CREATETEMPORARY(tempLob, FALSE, DBMS_LOB.CALL);
57  obj.getAllAttributes(ctx,tempLob);
58  DBMS_OUTPUT.PUT_LINE(DBMS_LOB.substr(tempLob,
59 DBMS_LOB.getLength(tempLob),1));
60  COMMIT;
61  EXCEPTION
62  WHEN ORDSYS.ORDAudioExceptions.AUDIO_PLUGIN_EXCEPTION THEN
63  DBMS_OUTPUT.PUT_LINE('ORDAudioExceptions.AUDIO_PLUGIN_EXCEPTION
64 caught');
65 END;
66 /
67
68 DECLARE
69  obj ORDSYS.ORDVideo;
70  tempLob CLOB;
71  ctx RAW(64) :=NULL;
72 BEGIN
73  SELECT p.video INTO obj FROM media_video p
74  WHERE media_id = 1;
75  DBMS_OUTPUT.PUT_LINE('getting comma separated list of all attributes');
76  DBMS_OUTPUT.PUT_LINE('-----');
77  DBMS_LOB.CREATETEMPORARY(tempLob, FALSE, DBMS_LOB.CALL);
78  obj.getAllAttributes(ctx,tempLob);
79  DBMS_OUTPUT.PUT_LINE(DBMS_LOB.substr(tempLob,
80 DBMS_LOB.getLength(tempLob),1));
81  COMMIT;
82  EXCEPTION
83  WHEN ORDSYS.ORDVideoExceptions.METHOD_NOT_SUPPORTED THEN
84  DBMS_OUTPUT.PUT_LINE('VIDEO METHOD_NOT_SUPPORTED EXCEPTION caught');
85  WHEN OTHERS THEN
86  DBMS_OUTPUT.PUT_LINE('EXCEPTION CAUGHT');
87 END;
88 /

```

Chapter 11

Media Integration

Non abbiamo visto due aspetti importanti:

- Il problema della sincronizzazione, perchè se abbiamo un progetto multimediale ci aspettiamo più oggetti multimediali insieme
- Rappresentazione dell'oggetto multimediale. Quando andiamo a descrivere qualcosa dobbiamo pensare prima al tool che usiamo e poi la descriviamo. Questa descrizione mi serve per fare l'interrogazione (voglio tutte le immagini con un tramonto), queste descrizioni sono dipendenti dalla piattaforma.

C'è uno standard che possa essere usato per descrivere gli oggetti multimediali in maniera uniforme rispetto a tutte le piattaforme:

- SMIL, all'interno del W3C
- MPEG7

11.1 Sincronizzazione

È importante perchè il progetto multimediale è una combinazione di multimedia. Anche sincronizzare caratteristiche di rete è importante, ad esempio un testo è discreto mentre un video è continuo.

Il problema della sincronizzazione è grosso, perchè porta alla quality of service, perchè l'utente vuole qualcosa di bello, non gli interessa niente delle caratteristiche video.

Il problema classico è quello del lip sync.

La sincronizzazione a volte avviene alla fonte e a volte a valle, sincronizzo e poi spedisco o spedisco e poi sincronizzo? ci sono tutta una serie di logiche che andremo poi a dettagliare nello specifico.

11.1.1 Media Integration in MM Presentation

Il problema di sincronizzazione è stato trattato principalmente nel contesto delle presentazioni. Devo in qualche modo descrivere la sincronizzazione, perchè poi questa servirà (questo audio viene prima del video, il testo si sovrappone, quando si sovrappone, come...). Questa descrizione avviene a livello di codice, si è però cercato di creare un linguaggio di sincronizzazione comune a tante piattaforme.

Ci sono tanti autoring tool, ma ognuno usa il suo, ad esempio PowerPoint ha il suo linguaggio per descrivere la sincronizzazione.

Quello che era stato fatto da W3C era stato creare un markup language per descrivere le sincronizzazioni multimediali, chiamato SMIL, che non ha avuto un grande successo ed è morto. Ora se devo fare sincronizzazione Web uso HTML5 con dei tag appositi.:

```
1 <embed src="path/MyVideo.avi" width="320" height="240" autostart="true"
  loop="true"> </embed>
```

11.1.2 SMIL

Devo integrare vari multimedia e devo anche descrivere la sincronizzazione, molto a basso livello. Cosa che anche usando HTML non è possibile.

Ci permette di definire da un punto di vista sintattico tutte le problematiche di sincronizzazione, e funziona sul DOM della pagina Web.

Era stato pensato solo per descrivere integrazioni multimediali, in realtà poteva andare a creare dei profili di sincronizzazione in base al device o all'utente (se aveva pagato per più qualità glie la do).

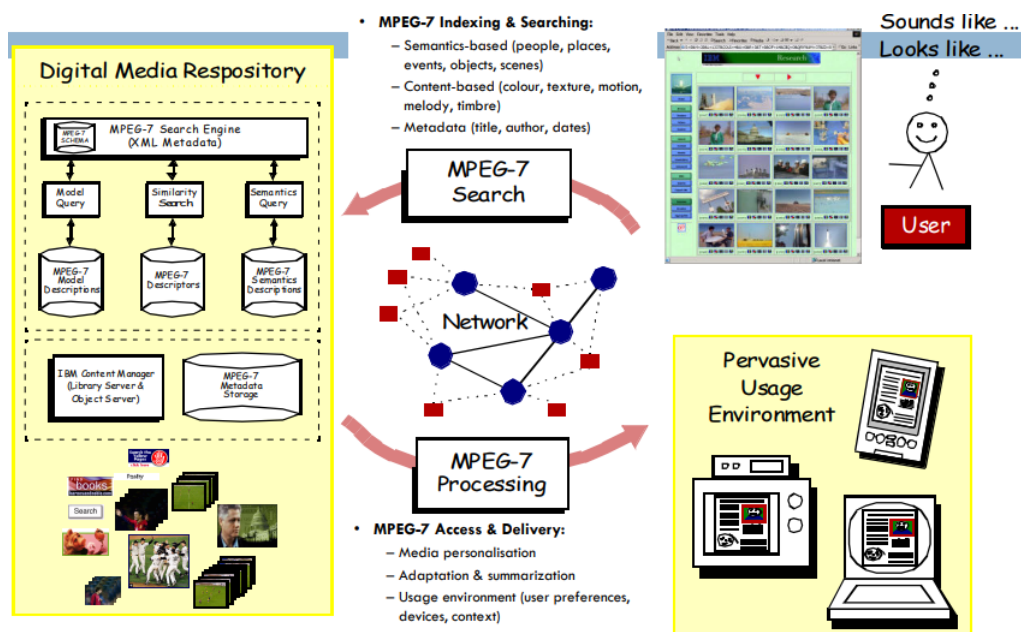
11.1.3 MPEG7

È l'unico standard sopravvissuto per la rappresentazione multimediale e la sua sincronizzazione. È l'unico standard che c'è per la sua descrizione multimediale, vuol dire definire un linguaggio di markup che permette di dare un insieme di attributi che caratterizzano in tutti i livelli (singolo testo, immagine, video) fino ad architetture più complicate la sincronizzazione.

È caratterizzato da:

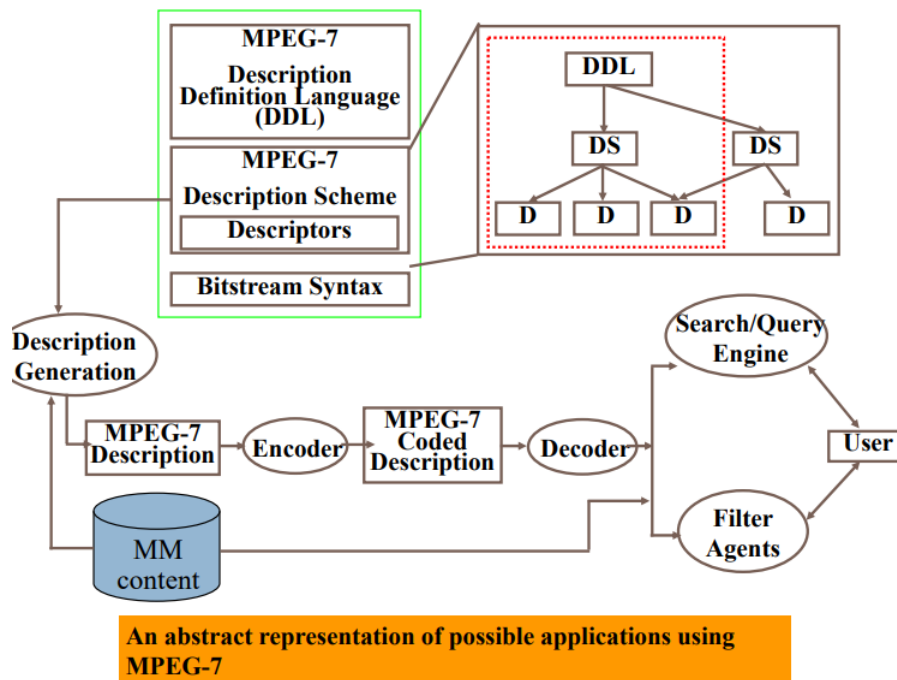
- Schema: che descrittori usare
- Descrittori
- DDL: definisce il linguaggio specifico da utilizzare per un dominio specifico

Abbiamo un datastore di file multimediali, ho gli schemi descrittori (schemi che dicono che descrittori usare), i descrittori. Questi descrittori sono alla base di possibili ricerche, sono tantissimi (tecnici, di framework, semantici). Avendo tanti descrittori posso addirittura fare delle query semantiche. Fare ricerche usando l'MPEG7 permetterebbe di fare ricerche estremamente efficaci in contesti multimediali.



Cominciamo a descrivere gli oggetti singoli, immagini, audio e video. A livello terminologico:

- Dato è l'oggetto
- Feature: caratteristiche che vado a descrivere
- Descrittore: definisce la feature
- Schema descrittore: definisce i descrittori
- DDL: linguaggio per definire i descrittori

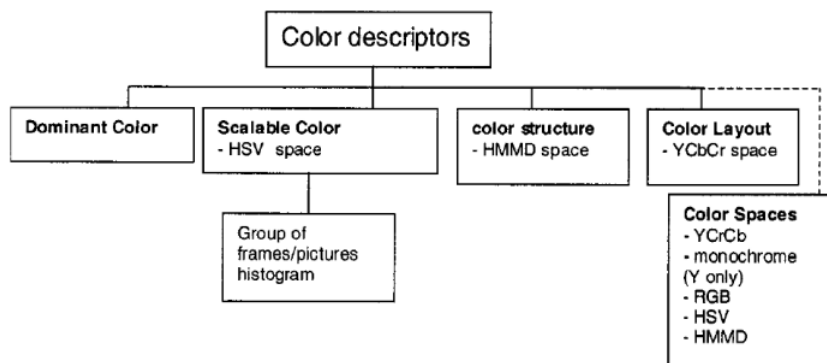


Non è così tanto usato perchè gli schemi ci definiscono una marea di descrittori, ma chi popola i descrittori? in Oracle è il motore che popola gli attributi dell'ORDImage qui non c'è un motore. La descrizione è appannaggio dell'utente finale. Ci sono dei tool che estraggono gli attributi con cui popolare i descrittori, ma sono chiusi e di proprietà di chi lo ha sviluppato.

Descrittori visuali

Usati per le immagini e per i singoli frame dei video.

- Colore
- Texture
- Forma
- Movimento



Addirittura i descrittori di colore si distinguono, ci sono alcuni che descrivono la dominante di colore, la variazione, il modello che uso, il color space:

Vediamo un esempio di descrittore per il colore:

```

<Mpeg7 xmlns="urn:mpeg:mpeg7:schema:2001"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:mpeg7="urn:mpeg:mpeg7:schema:2001"
xsi:schemaLocation="urn:mpeg:mpeg7:schema:2001 Mpeg7-2001.xsd">
  <Description xsi:type="ContentEntityType">
    <MultimediaContent xsi:type="ImageType">
      <Image>
        <MediaLocator>
          <MediaUri>file://red.jpg</MediaUri>
        </MediaLocator>
        <TextAnnotation>
          <FreeTextAnnotation>
            A completely red image :-)
          </FreeTextAnnotation>
        </TextAnnotation>
        <VisualDescriptor xsi:type="DominantColorType">
          <SpatialCoherency>31</SpatialCoherency>
          <Value>
            <Percentage>31</Percentage>
            <Index>255 0 0</Index>
            <ColorVariance>1 0 0</ColorVariance>
          </Value>
        </VisualDescriptor>
      </Image>
    </MultimediaContent>
  </Description>
</Mpeg7>
  
```

Un altro esempio più complicato, per descrivere lo spazio colore quando il file non è piccolo. Descrivo un MediaContent che è un'immagine, ha un determinato formato (descrittori sulle specifiche tecniche), poi ho dei descrittori di contenuto, ad esempio uno che va a descrivere lo spazio colore YChannel con tutti i suoi coefficienti:

```
<?xml version="1.0" encoding="UTF-8"?>
<Mpeg7 xmlns="urn:mpeg:mpeg7:schema:2001"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="urn:mpeg:mpeg7:schema:2001 MPEG7-
2001.xsd">
  <Description xsi:type="ContentEntityType">
    <MultimediaContent xsi:type="ImageType">
      <Image>
        <MediaLocator>
          <MediaUri>http://example.com/image.jpg</MediaUri>
        </MediaLocator>
        <MediaInformation>
          <FileFormat>
            <Format>image/jpeg</Format>
          </FileFormat>
        </MediaInformation>
        <ContentDescription>
          <ColorLayout>
            ...
            .....
            <!-- Y channel DCT coefficients -->
            <YCoeff>
              <Coeff>78</Coeff>
              <Coeff>-22</Coeff>
              <Coeff>34</Coeff>
              <Coeff>12</Coeff>
              <Coeff>-8</Coeff>
              <Coeff>2</Coeff>
              <Coeff>-3</Coeff>
              <Coeff>1</Coeff>
              <Coeff>0</Coeff>
            </YCoeff>
            <!-- Cb channel DCT coefficients -->
            <CbCoeff>
              <Coeff>70</Coeff>
              <Coeff>-10</Coeff>
              <Coeff>20</Coeff>
              <Coeff>8</Coeff>
              <Coeff>-5</Coeff>
              <Coeff>1</Coeff>
              <Coeff>-2</Coeff>
              <Coeff>0</Coeff>
              <Coeff>0</Coeff>
            </CbCoeff>
            .....
            <!-- Cr channel DCT coefficients -->
            <CrCoeff>
              <Coeff>65</Coeff>
              <Coeff>-15</Coeff>
              <Coeff>25</Coeff>
              <Coeff>10</Coeff>
              <Coeff>-7</Coeff>
              <Coeff>2</Coeff>
              <Coeff>-1</Coeff>
              <Coeff>0</Coeff>
              <Coeff>0</Coeff>
            </CrCoeff>
            </ColorLayout>
          </ContentDescription>
        </Image>
      </MultimediaContent>
    </Description>
  </Mpeg7>
```

Abbiamo visto anche i descrittori della shape:

- Region-based Descriptor
- Contour-based Shape Descriptor
- 2D/3D Shape Descriptor
- 3D Shape Descriptor

descrivo il contorno che è fatto da diversi punti e li descrivo:

```
<?xml version="1.0" encoding="UTF-8"?>
<Mpeg7 xmlns="urn:mpeg:mpeg7:schema:2001"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="urn:mpeg:mpeg7:schema:2001 MPEG7-
2001.xsd">
  <Description xsi:type="ContentEntityType">
    <MultimediaContent xsi:type="ImageType">
      <Image>
        <MediaLocator>
          <MediaUri>http://example.com/image.jpg</MediaUri>
        </MediaLocator>
        <MediaInformation>
          <FileFormat>
            <Format>image/jpeg</Format>
          </FileFormat>
        </MediaInformation>
        ...
        .....
        <ContentDescription>
          <Shape>
            <ContourShape>
              <Grid>
                <Width>100</Width>
                <Height>100</Height>
              </Grid>
              <Contour>
                <Point>
                  <X>30</X>
                  <Y>40</Y>
                </Point>
                <Point>
                  <X>35</X>
                  <Y>42</Y>
                </Point>
                <Point>
                  <X>40</X>
                  <Y>45</Y>
                </Point>
                .....
                <!-- Additional points representing the shape contour -->
                </Contour>
              </ContourShape>
            </Shape>
          </ContentDescription>
        </Image>
      </MultimediaContent>
    </Description>
  </Mpeg7>
```

I movimenti posso descriverli come movimenti di camera o come movimenti di scena (traiettoria):

- Motion Activity Descriptors
- Camera Motion Descriptors
- Motion Trajectory Descriptors: esempio tipico delle partite di calcio
- Parametric Motion Descriptors

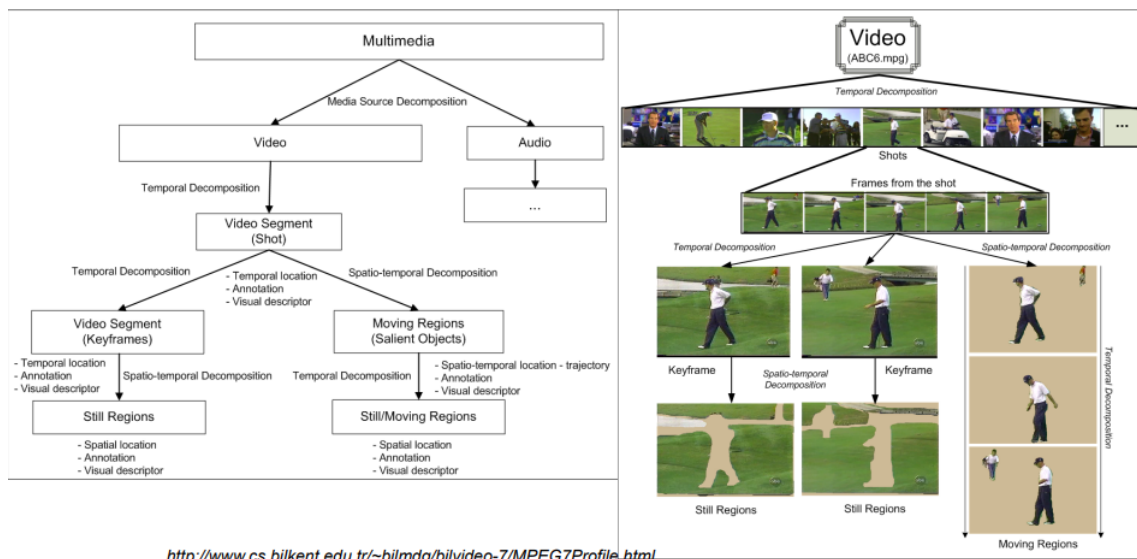
```

<?xml version="1.0" encoding="UTF-8"?>
<Mpeg7 xmlns="urn:mpeg:mpeg7:schema:2001"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="urn:mpeg:mpeg7:schema:2001 MPEG7-
  2001.xsd">
  <Description xsi:type="ContentEntityType">
    <MultimediaContent xsi:type="VideoType">
      <Video>
        <MediaLocator>
          <MediaUri>http://example.com/video.mp4</MediaUri>
        </MediaLocator>
        <MediaInformation>
          <FileFormat>
            <Format>video/mp4</Format>
          </FileFormat>
          </MediaInformation>...
        </Video>
      </MultimediaContent>
    </Description>
  </Mpeg7>>
  ....
  <ContentDescription>
    <MotionTrajectory>
      <Trajectory>
        <Point>
          <Time>0</Time>
          <X>50</X>
          <Y>100</Y>
        </Point>
        <Point>
          <Time>1</Time>
          <X>52</X>
          <Y>98</Y>
        </Point>
        <Point>
          <Time>2</Time>
          <X>55</X>
          <Y>96</Y>
        </Point>
        <!-- Additional points representing the trajectory over time -->
      </Trajectory>...
    </MotionTrajectory>
    <BoundingBox>
      <TopLeft>
        <X>30</X>
        <Y>90</Y>
      </TopLeft>
      <BottomRight>
        <X>70</X>
        <Y>110</Y>
      </BottomRight>
    </BoundingBox>
  </ContentDescription>
  </Video>
</MultimediaContent>
</Description>
</Mpeg7>>

```

Video

I descrittori video sono interessanti, perchè mi permettono di segmentare il video, posso descrivere cosa c'è in foreground e in background e cosa c'è nel video. Sono molti di più e posso descrivere la decomposizione temporale e spaziale nel video.



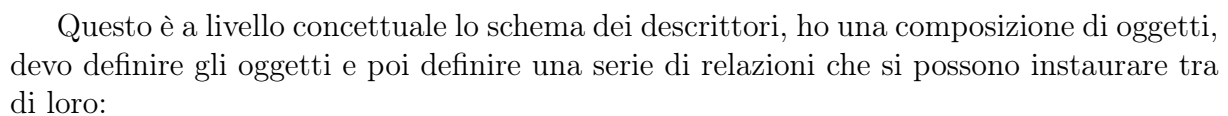
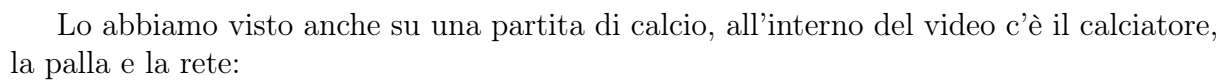
Posso avere dei descrittori che descrivono il contenuto, descrivere la semantica del video. Posso descrivere gli aspetti strutturali, se è una collezione come si combinano gli

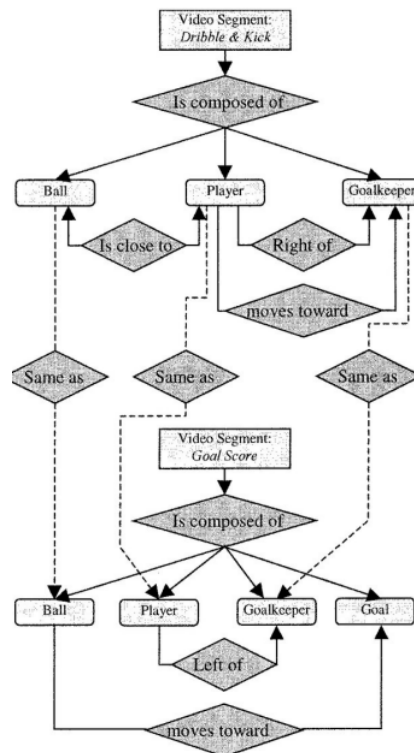
Posso usare vari descrittori per descrivere il video a vari livelli:

- A livello 1 posso descrivere gli attributi fisici, ad esempio la dominante di colore
- A livello 2 voglio descrivere una decomposizione del video, cosa c'è in primo piano (shape, histogram dei colori e luce), e quello che c'è sullo sfondo

Potrei creare uno schema di descrittori che mi permette di definire descrittori ad alto e basso livello.

- Potrei creare uno schema di descrittori che mi permette di definire descrittori ad alto e basso livello.





A livello più alto posso descrivere delle caratteristiche di tutto un componente multimediale (summary, spazi frequenza, interazione con l'utente).

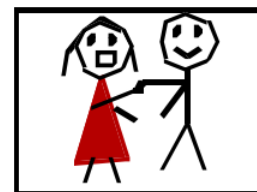
Un primo esempio di descrittori sono gli annotatori, servono per descrivere degli elementi all'interno di un audio o video per descrivere la semantica di quello che c'è nell'immagine:

The following example gives an MPEG-7 description of the event of handshake between people:

```

<Mpeg7>
  <Description xsi:type="SemanticDescriptionType">
    <Semantics>
      <Label>
        <Name> Shake hands </Name>
      </Label>
      <SemanticBase xsi:type="AgentObjectType" id="A">
        <Label href="urn:example:acs">
          <Name> Person A </Name>
        </Label>
      </SemanticBase>
      <SemanticBase xsi:type="AgentObjectType" id="B">
        <Label href="urn:example:acs">
          <Name> Person B </Name>
        </Label>
      </SemanticBase>
      <SemanticBase xsi:type="EventType">
        <Label><Name> Handshake </Name></Label>
        <Definition>
          <FreeTextAnnotation> Clapping of right hands by two people </FreeTextAnnotation>
        </Definition>
        <Relation type="urn:mpeg:mpeg7:cs:SemanticRelationCS:2001:agent" target="#A"/>
        <Relation type="urn:mpeg:mpeg7:cs:SemanticRelationCS:2001:accompanier" target="#B"/>
      </SemanticBase>
    </Semantics>
  </Description>
</Mpeg7>

```

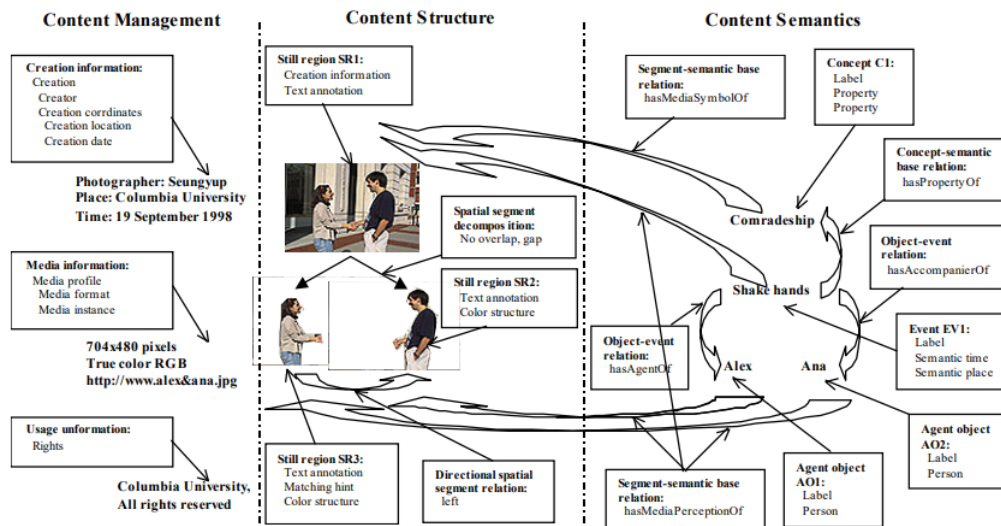


Tag semantici

Sono gli ultimi descrittori che sono stati aggiunti e sono utili per fare query semantiche. Sono stati integrati nei vari schemi audio video...

Ha introdotto 3 classi di descrittori:

- **Contenuto:** descrittori che descrivono il video nelle sue caratteristiche gestionali, chi è il fotografo, l'ora in cui è stato fatto, dove, caratteristiche fisiche, diritti di autore
- **Content Structure:** c'è una region con delle caratteristiche visuali
- **Content Semantic:** se c'è una decomposizione a livello di struttura c'è una relazione semantica? questa persona chi è? le due persone interagiscono tra di loro? perchè? descrivo la regione con degli attributi semantici



Strutture e tool semantici

Se è complicato estrarre i valori da attribuire ai descrittori tecnici è ancora più difficile costruire un agente che crei i descrittori semantici. Ad oggi dove è usato si usano persone che descrivono la scena.

Quindi l'MPEG7 sarebbe pronto per descrivere la semantica dei file multimediali su vari livelli, ma è un casino tirare fuori le descrizioni.

Chapter 12

Quality of Service

Ho recuperato tutti i file che interessano all'utente, ora devo mandarli all'utente. Abbiamo popolato il dataset, lo abbiamo descritto, abbiamo fatto information retrieval ora dobbiamo dare il risultato finale all'utente. Questa cosa crea dei problemi, perché dobbiamo considerare le problematiche della trasmissione finale.

A seconda di qual'è la fruizione dell'utente possiamo classificare le applicazioni multimediali:

- Streaming di informazioni già memorizzate
- Streaming di un video live
- Real-time in cui c'è un'interazione (video conferenza)

Ci sono diverse classi ognuna con delle caratteristiche proprie, ad esempio di un video memorizzato in un dataset potresti mettere pausa ed andare avanti, in una partita non potresti andare avanti, in una videocall mettere in pausa non avrebbe senso.

Caratteristiche da considerare:

- Trasmissione di grossi volumi
- Trasmissione real-time ed interattiva, potremmo avere problemi se perdiamo qualcosa
- Doppia comunicazione
- Trasmissione in rete, in alcuni momenti la rete è vuota in altri è piena. Non è predicibile ed ho dei picchi

Abbiamo qualche esempio di qualità di trasmissione:

	<i>Data rate (approximate)</i>	<i>size</i>	<i>Sample or frame frequency</i>
Telephone speech	64 kbps	8 bits	8000/sec
CD-quality sound	1.4 Mbps	16 bits	44,000/sec
Standard TV video (uncompressed)	120 Mbps	up to 640 x 480 pixel x 16 bits	24/sec
Standard TV video (MPEG-1 compressed)	1.5 Mbps	variable	24/sec
HDTV video (uncompressed)	1000–3000 Mbps	up to 1920 x 1080 pixel x 24 bits	24–60/sec
HDTV video (MPEG-2 compressed)	10–30 Mbps	variable	24–60/sec

La nostra sfida è quella di superare la logica del best effort offerta da TCP/UDP/IP in cui non c'è nessuna garanzia sull'aspettativa o sulla varianza del ritardo dei pacchetti, devo considerare le esigenze che definiscono la logica del servizio.

Ci sono due qualità del servizio:

- Tecnica
- Percepita: quello che l'utente vuole

Le caratteristiche tecniche sono:

- Data rate: misurazione della velocità di trasmissione
- Latenza: rapporto tra il frame massimo e i pacchetti persi
- Perdita dei pacchetti o errore dei pacchetti: una percentuale dei pacchetti persi
- Jitter: la continuità di trasmissione
- Sincronizzazione

È un attributo di continuità nel tempo, un alto jitter lo abbiamo quando il video si continua a bloccare basso quando non si blocca oppure non me ne accorgo.

12.1 Multimedia Service Classes

Queste dimensioni di qualità del servizio caratterizzano determinate classi di trasmissione multimediali:

- Real-time: doppia via di comunicazione, bassa latenza, poco jitter ed eventualmente un servizio di priorità
- Priority data: quello che abbiamo nelle applicazioni di e-commerce, c'è una perdita dei pacchetti bassa, latenza bassa, jitter poco importante
- Silver: una via di comunicazione, moderata accettabile latenza, moderato jitter, web surfing
- Best effort: non è real time ma è buono nel download
- Bronze: nessuna garanzia

Quali sono i valori accettabili in termini di bitrate in base ai servizi:

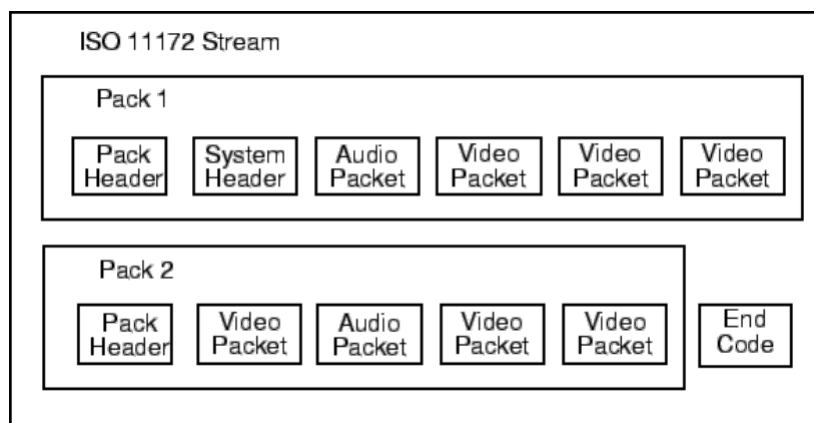
Così come possiamo avere una tolleranza della latenza e del jitter in un video:

Application	Speed Requirement
Telephone	16 kbps
Audio-conferencing	32 kbps
CD-quality audio	128–192 kbps
Digital music (QoS)	64–640 kbps
H. 261	64 kbps–2 Mbps
H. 263	< 64 kbps
DVI video	1.2–1.5 Mbps
MPEG-1 video	1.2–1.5 Mbps
MPEG-2 video	4–60 Mbps
HDTV (compressed)	> 20 Mbps
HDTV (uncompressed)	> 1 Gbps
MPEG-4 video-on-demand (QoS)	250–750 kbps
Videoconferencing (QoS)	384 kbps–2 Mbps

Application	Avg Latency Tolerance (msec)	Avg Jitter Tolerance (msec)
Low-end videoconf. (64 kbps)	300	130
Compressed voice (16 kbps)	30	130
MPEG NTSC video (1.5 Mbps)	5	7
MPEG audio (256 kbps)	7	9
HDTV video (20 Mbps)	0.8	1

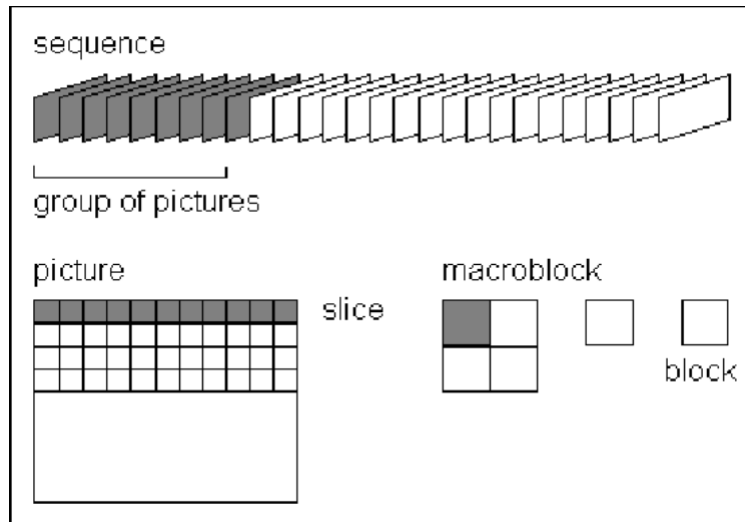
12.2 MPEG System Streams

Mettiamo di voler trasmettere un MPEG, che come abbiamo visto, è formato da pacchetti che devono essere trasmessi:



Ogni singolo di frammento video è una sequenza di frame e ogni frame è una griglia di riferimento con slice o macroblocchi:

Nel caso in cui dovessi fare una trasmissione come tratteremmo il problema del jitter



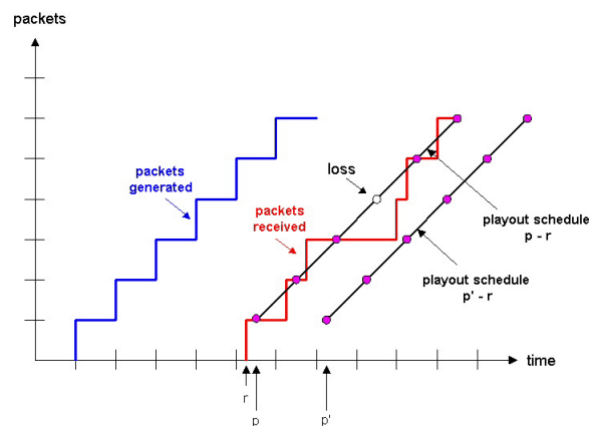
in modo da essere tolleranti sul jitter. Sono interessato ad avere continuità a discapito di altre dimensioni. Posso adottare delle tecniche.

12.2.1 Packet Jitter

In alcuni contesti non è possibile, ma un classico modo per moderare il problema del Jitter è quello di moderare il delay. Se quando spedisco la rete è talmente congestionata ricevo i pacchetti in un ordine diverso rispetto a quello in cui li ho mandati, questo implica un Jitter:



La classica strategia è creare un delay, sarebbe meglio posticipare la ricezione in modo da ricostruire una sequenza uguale a quella che è stata inviata:

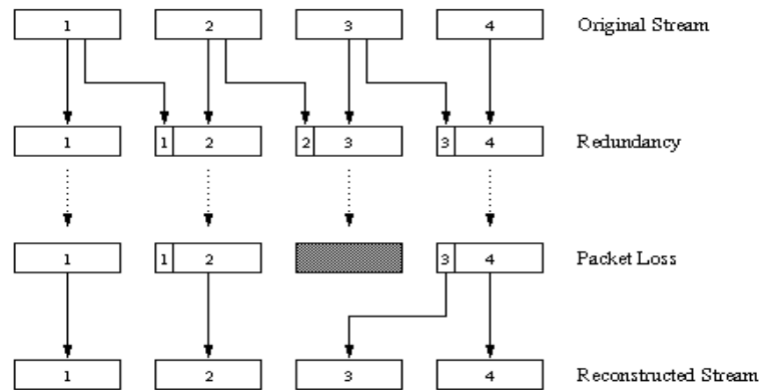


C'è una formula matematica che ci permette di calcolare di quanto ritardare la trasmissione, a volte funziona a volte no.

12.2.2 Perdita dei pacchetti

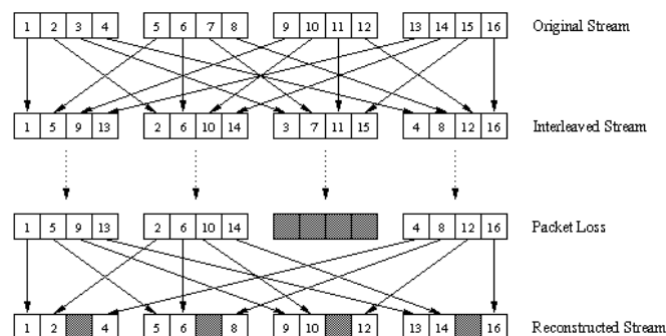
È un altro problema grosso che dovrei riuscire a risolvere.

Una prima soluzione è inserire nello streaming originale una ridondanza, il pacchetto 1 te lo rispedisco con risoluzione più bassa nel pacchetto 2. Questo garantisce che se perdo il pacchetto te lo ricostruisco, con una delay e un downgrade della qualità video:



Il problema è di avere un overhead della trasmissione.

Un altro modo è quello che viene chiamato interleaving streaming, invece che trasmettere come è nella trasmissione originale trasmetto ricombinando i pacchetti:



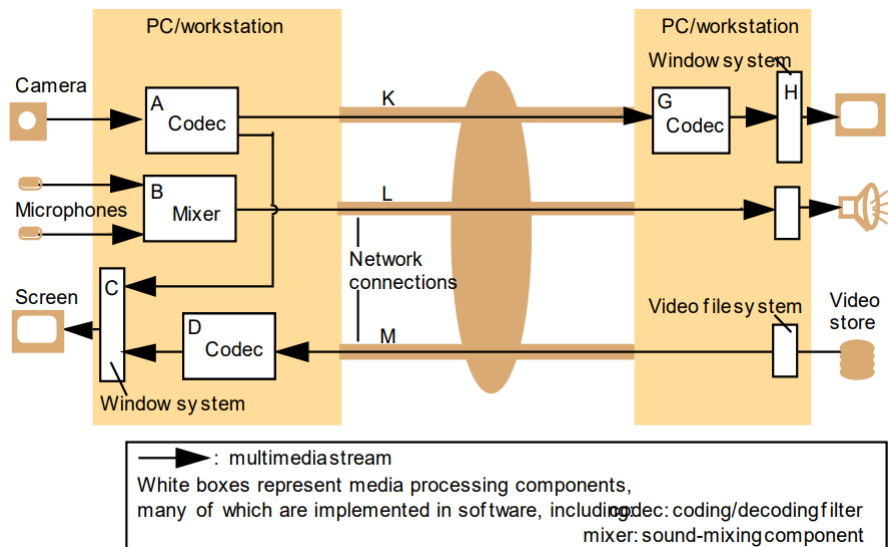
È comodo perchè nel momento in cui perdo un pacchetto quando ricostruisco un pacchetto ho perso solo piccoli pezzi impercettibili di informazione. Non si sposa bene con il real time perchè devo avere tutta l'informazione già arrivata, però evita il downgrade e l'overhead.

Per applicazioni non realtime c'è anche la possibilità della ritrasmissione del pacchetto.

12.3 The windows of scarcity

Oggi abbiamo un abbondanza di risorse, quindi possiamo garantire una qualità minima andando a superare il best-effort in cui andavamo a garantire l'arrivo dei pacchetti ma senza garantire una qualità in termini di larghezza di banda, jitter.

L'idea di base è questa, in un lato trasmettiamo l'informazione e nell'altro la riceviamo. L'informazione multimediale viene trasmessa sopra una rete che viene catturata da parte dell'utente finale dagli apparati di decodifica (casce, monitor):



Ogni risorsa ha delle sue specifiche tecniche che la vanno a contraddistinguere, ad esempio una videocamera ha una particolare larghezza di banda. Nonchè abbiamo tutta una serie di specifiche che caratterizzano la rete, dovrei considerare che in tratti diversi della rete potrei avere caratteristiche diverse.

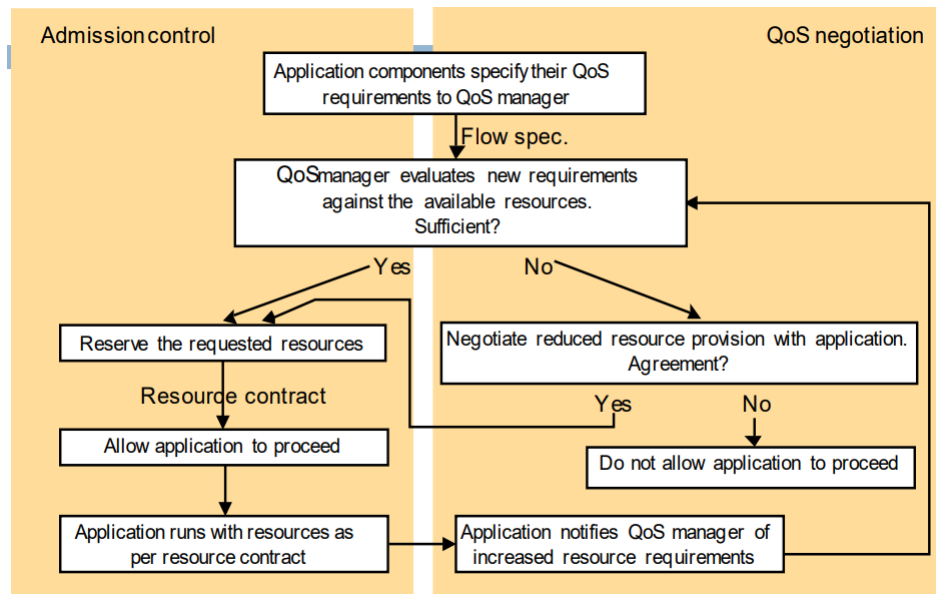
Component		Bandwidth	Latency	Loss rate	Resources required
Camera	Out:	10 frames/sec, raw video - 640x480x16 bits		Zero	-
A Codec	In:	10 frames/sec, raw video	Interactive	Low	10 ms CPU each 100 ms;
	Out:	MPEG-1 stream			10 Mbytes RAM
B Mixer	In:	2 × 44 kbps audio	Interactive	Very low	1 ms CPU each 100 ms;
	Out:	1 × 44 kbps audio			1 Mbytes RAM
H Window system	In:	various	Interactive	Low	5 ms CPU each 100 ms;
	Out:	50 frame/sec framebuffer			5 Mbytes RAM
K Network connection	In/Out:	MPEG-1 stream, approx. 1.5 Mbps	Interactive	Low	1.5 Mbps, low-loss stream protocol
L Network connection	In/Out:	Audio 44 kbps	Interactive	Very low	44 kbps, very low-loss stream protocol

12.4 Gestione QoS

Avviene in due fasi principali:

- **Negoziiazione:** negoziare una qos significa che chi emette o riceve un informazione deve definire delle specifiche, verificare che le specifiche offerte in rete siano adeguate, verificare se la trasmissione è fattibile alle mie specifiche. Se non è garantita una trasmissione alle mie specifiche negoziiamo un downgrade delle specifiche.
- **Admission control:** le applicazioni devono rispettare le risorse contrattate, bisogna riciclare le risorse quando si liberano

Nello schema abbiamo una prima fase in cui ogni componente specifica cosa ho bisogno al manager della qos, questo va a matchare con le specifiche del flusso, a questo punto il manager valuta se le esigenze possono essere supportate dalle risorse in rete. Se sono sufficienti devo passare alla fase di admission control, devo preservare le risorse al fine di garantirti quello che tu hai richiesto, procedo alla trasmissione e va avanti finchè le risorse non vengono liberate. Se il matching non va a buon fine devo fare una negoziazione e trovare un accordo, se non lo trovo non trasmetto nulla:



12.4.1 Negoziazione

Devo specificare le caratteristiche delle risorse e della trasmissione, devo dare un numero alle dimensioni della QoS:

- Larghezza di banda: cosa ho bisogno di trasmettere
- Latenza: un tempo accettabile di latenza
- Perdita di pacchetti: di solito dato in percentuale (è accettabile la perdita del 2% dei pacchetti)

Poi devo descrivere le risorse presenti per vedere se matchano la domanda.

La larghezza di banda non deve essere per forza un valore, può anche essere un minimo e un massimo.

Linear Bounded Arrival Process Model (LBAP)

Poi devo specificare un picco di trasmissione, burst, come faccio a sapere se ci sono momenti di congestione? devo garantire una preservazione della trasmissione anche in casi di congestione. Per poter gestire questo problema viene adottato il modello LBAP, rappresenta come la rete è in grado di gestire i picchi di trasmissione per verificare se la gestione è adeguata alla mia trasmissione. Questo modello si basa su tre variabili:

- M: la dimensione massima del messaggio che posso garantire
- R: il rate, il massimo messaggi per secondo
- B: quant'è il massimo dei messaggi che superano il massimo che sono in grado di gestire

Grazie a questi 3 valori sono in grado di definire il modello con altri tre parametri:

- Numero di messaggi massimi che arrivano da una particolare risorsa: $N = B + R \cdot IntervalloDiTempo$
- Massima velocità media: $R^1 = M \cdot R$, messaggi per secondo che riesco a gestire
- La dimensione massima del Buffer: $BS = M \cdot (B + 1)$

Vediamo un esempio (perchè lo mette di sicuro all'esame), consideriamo di avere le tre variabili del LBAP:

- $M = 1176$ Bytes per messaggio
- $R = 75$ messaggi al secondo
- $B = 10$ messaggi

Ora calcoliamo la previsione dell'utilizzo delle risorse:

- Durante un intervallo di tempo di 1 secondo, il numero massimo di messaggi che arrivano a una risorsa non deve superare: $N = 10 + (75 \cdot 1) = 85$ messaggi
- La massima velocità media di trasmissione dei dati in byte al secondo è: $R^1 = 1176 \cdot 75 = 88200$
- La dimensione massima del buffer in byte è: $BS = 1176 \cdot (10 + 1) = 12936$

Poi ci sono altri valori che è possibile specificare:

- Latenza: è un numero temporale che varia a seconda del tipo di trasmissione sfruttando la percezione umana. Il Jitter solitamente è da portare a 0.
- Perdita dei pacchetti: presa con una percentuale o una probabilità

12.4.2 Traffic shaping

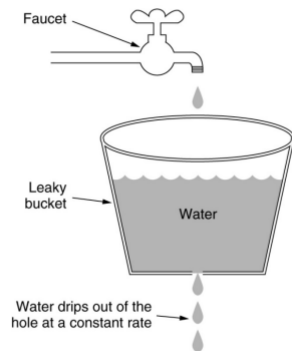
Oltre a modellare le specifiche, devo modellare come viene gestito il traffico. Data una richiesta e una domanda cosa avviene quando mettiamo in atto una trasmissione. Si usano due algoritmi:

- Leaky Bucket
- Token Bucket

Leaky Bucket

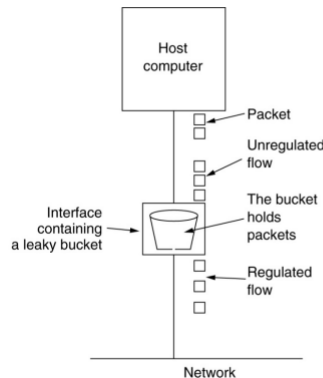
È usato per garantire una costanza di trasmissione ed evitare picchi, non abbiamo Jitter. Le risorse sulla rete gestiscono le trasmissioni mantenendo una certa regolarità, devono avere un buffer interno (nel router), all'arrivo dei vari pacchetti questi vengono rispediti ad un constant rate, la trasmissione avviene regolarizzando il flusso. Chiaramente se arrivano troppi pacchetti vengono persi.

(a) A leaky bucket with water



(a)

(b) a leaky bucket with packets



(b)

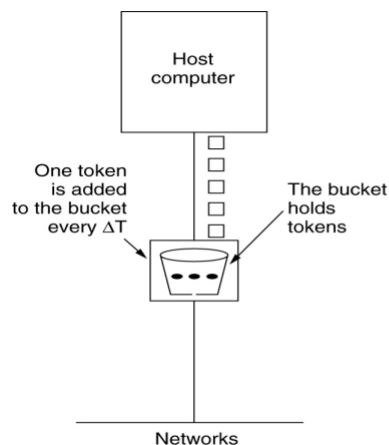
Garantisco una regolarità di trasmissione dei pacchetti ma c'è una probabilità di perdere i pacchetti.

Questo modellatore è adeguato per evitare Jitter ma non è adeguato per supportare una bassa perdita dei pacchetti.

Token Bucket

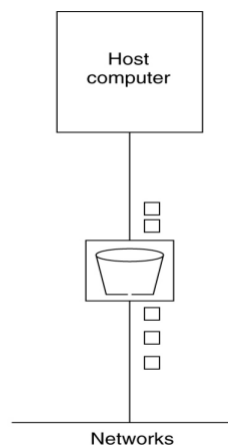
Non trasmettiamo ad una constant rate, ma in base a quanti token ho presenti nel secchiello, i token vengono generati ad intervalli regolati e tendenzialmente non possono accumularsi all'infinito.

(a) Before



(a)

(b) After

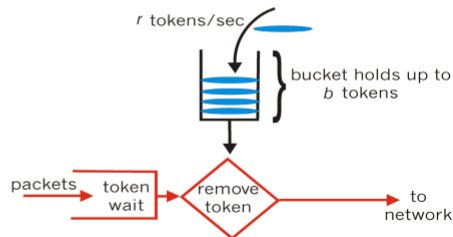


(b)

Mettiamo che posso trasmettere pacchetti solo se ho un numero adeguato di token, la generazione di token nel router è di 3 al secondo. Quindi tengo i pacchetti e li spedisco solo quando ho i token.

arrival	queue	Token bucket	sent
p1 (5)	-	0	-
p2 (2)	p1	3	-
p3 (1)	p2	6-5=1	p1
		4-2-1=1	p3,p2
		4	
		6	

parameters:
MaxTokens=6
 $1/r=3$ (=3 token/time)



Così facendo ho la possibilità di mettere in coda se non ho abbastanza token e di gestire un picco se ho abbastanza token (perchè accumulo token e poi quando arrivano tanti messaggi li posso spedire tutti).

Modella LBAP perchè ho un buffer che posso definire (è la B del modello).

Leaky Bucket vs Token Buket

Leaky Buket:

- Perdita:
 - Pacchetti
- Rate:
 - Fisso (perfetto)
- Arriving Burst:
 - Aspettano nel secchio

Token Bucket:

- Perdita:
 - Token
 - Gestione dei pacchetti separata
- Rate:
 - Velocità media
 - Burst consentiti
- Arriving Burst:
 - Possono essere spediti

12.4.3 Flow Specification

Esiste uno standard in cui posso specificare:

- Larghezza di banda: andando a definire la massima unità di trasmissione e il massimo transmission rate
- Delay: tempo
- Lost rate: una percentuale

12.4.4 Procedura di negoziazione

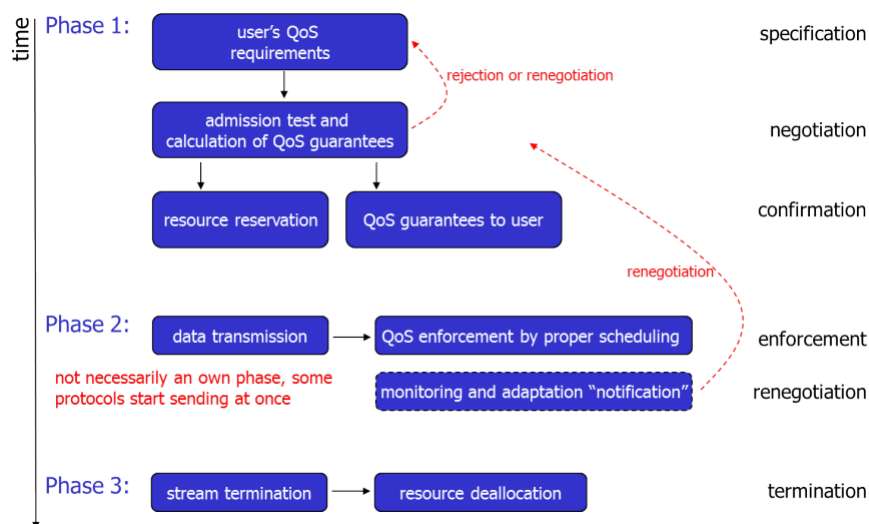
Il secondo step che devo fare è quello di andare a fare un matching tra l'offerta e la domanda. Approccio semplice:

- Colui che spedisce definisce un flusso di specifica e il QoS manager controlla se tutto quello che c'è sulla rete garantisce il servizio

Il risultato finale non è altro che un sì o un no.

12.4.5 Reservation delle risorse

Se la risposta è sì blocco tutte le risorse affinché il flusso ammesso sia garantito. C'è il reservation protocol, abbiamo un'operazione di reservation in cui abbiamo il calcolo delle risorse in base alle specifiche accordate e poi facciamo uno scheduling e ne consento l'uso.



12.4.6 Scheduling

Dipende come è gestita la rete, potrebbe esserci un round robin (pacchetto per pacchetto). Altrimenti viene garantito un real time scheduling, non viene applicato il round robin, ma quello che ha una deadline più prossima deve essere servito per prima. A tutti verrà garantita la qos negoziata.

12.4.7 Stream adaptation

É il caso in cui non posso garantire quello che mi hai chiesto, devo quindi rinegoziare. Lo faccio tramite un stream adaptation, ti chiedo se posso fare un downgrade delle tue richieste (ad esempio ti do l'audio in mono), piuttosto che fare un incremento del delay.

Cosa posso scalare:

- Scaling temporale: meno frame per secondo
- Scaling spaziale: riduco la qualità
- Scaling frequenza: compressione
- Amplitudinal scaling
- Spazio colore scaling

Attenzione che lo scaling non viene negoziato come soluzione globale su tutta la rete, ma viene offerta anche in base a delle strategie di filtering. Vuol dire che dipende un po' da chi è il destinatario, dico che posso degradare il servizio in base alle caratteristiche del ricevente (ad esempio se il ricevente non può ricevere una larghezza di banda superiore ad un tot non te la do). Potrei avere anche delle caratteristiche basate sugli utenti finali, chi ha pagato deve avere un servizio migliore rispetto a chi ha un servizio free.

12.5 IP Protocols

Attualmente possiamo implementare le modalità di gestione della QoS? É ancora una cosa non garantita, se non in particolari contesti è difficile andare a contrattare una qos perchè non è ancora gestita. Avere una qos significherebbe avere una classe di qos garantita.

In realtà la rete è pronta per supportare la qos, perchè nel protocollo IP ci sono delle parti in cui specificare la qos.

12.5.1 Approcci di Qos

Ci sono due approcci:

- Integrated Services
- Differentiated Services

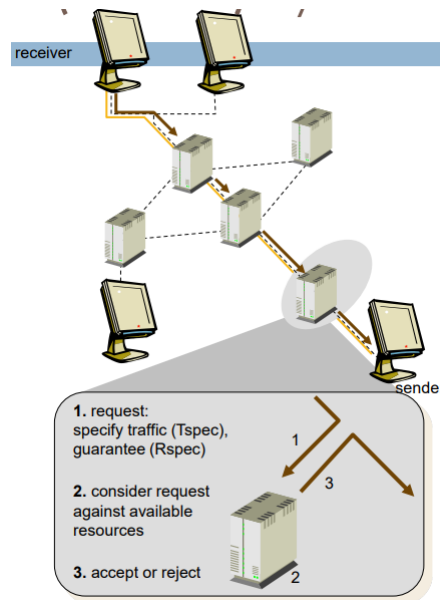
Che ci permettono di gestire la qos sulla rete attuale.

Integrated Services

É definito da uno standard IETF. Si basa sulle caratteristiche del protocollo IP più un altro servizio che si chiama resource reservation protocol che si occupa di fare la prelazione delle risorse in modo che la qos sia garantita (implementa la reservation delle risorse). Poi devo fare l'admission control per garantire la qos.

Funziona in questo modo, ci deve essere un gestore della qos che cattura le informazioni su tutta la rete per capire le caratteristiche del traffico sia per capire le specifiche delle

esigenze di chi spedisce e riceve. Quindi gli utenti devono specificare di cosa hanno bisogno. Il gestore deve mandare un segnale (admission control) con le specifiche (risorse e di rete) da rispettare a tutti i router, ogni router deve controllare se può garantire queste specifiche, da una risposta di accettazione o di rejected.



Per poter fare questo definisce nuovi servizi a livello di rete:

- Guaranteed service: permette di garantire una larghezza di banda e delay
- Controlled-load: garantisce un adeguato caricamento per gestire più richieste contemporaneamente

Per poter modellare questa strategia di trasmissione utilizziamo dei servizi di classe e la logica del token bucket, non facciamo altro di regolarizzare il traffico in modo tale da garantire che l'offerta venga rispettata. Quindi ogni router ha informazioni che riguardano il token rate e il bucket size per poter implementare l'algoritmo.

Attualmente è gestibile questo integrated service, perché ogni router è in grado di fare l'admission test, per tutti i OS. Non viene fatto perché:

- grosso overhead, prima della trasmissione c'è una serie di trasmissioni non banali.
- Grossa richiesta di memoria
- Poco flessibile
- Neutralità della rete, tutti sanno tutto, bello ma pericoloso
- Poco commerciale

IETF Differentiated Services

È un'altra proposta che è un po' più scaltra per garantire la qos nella rete, perchè permette di avere un'implementazione meno demanding per le risorse e per la naturalità della rete. Si differenzia per il fatto che ho una gestione delle funzionalità diverse, tutti i router non sono uguali. Sicuramente è utile che risolve problemi di scalabilità e non ho tutto quell'overhead.

L'idea di base è che non voglio che ci sia il multicast a livello di rete (il signal a tutti i router che mi devono rispondere, questo admission test è troppo demanding), però voglio andare a differenziare cosa succede ai bordi della rete (router che comunica con chi emette e riceve) rispetto a quello che succede nei punti core (quelli che non comunicano nei punti estremi), perchè fanno due cose diverse, uno deve capire quali sono le specifiche, l'altro deve trasmettere (può fregarsene delle specifiche), addirittura potrei fare un aggregazione dei flussi per garantire scalabilità.

Le functional components sono legate alle funzionalità del router, le cose che devono fare i router al bordo sono diverse dalle funzionalità che devono fare i router nel cuore. Le edge functions devono capire le specifiche, prendono il pacchetto e lo classifica in base alle condizione del traffico. Questo è il compito del router sul bordo. Il core function deve fare solo instradamento.

L'edge router tagga il pacchetto con il DS-mark, un marchio che lo contraddistingue in base alle sue esigenze di qos (lo marca leggendo nell'IP le sue esigenze). A questo punto i router core in base al marchio instradano i pacchetti.

Fondamentalmente le funzionalità dell'edge router sono quelle di classificatore.

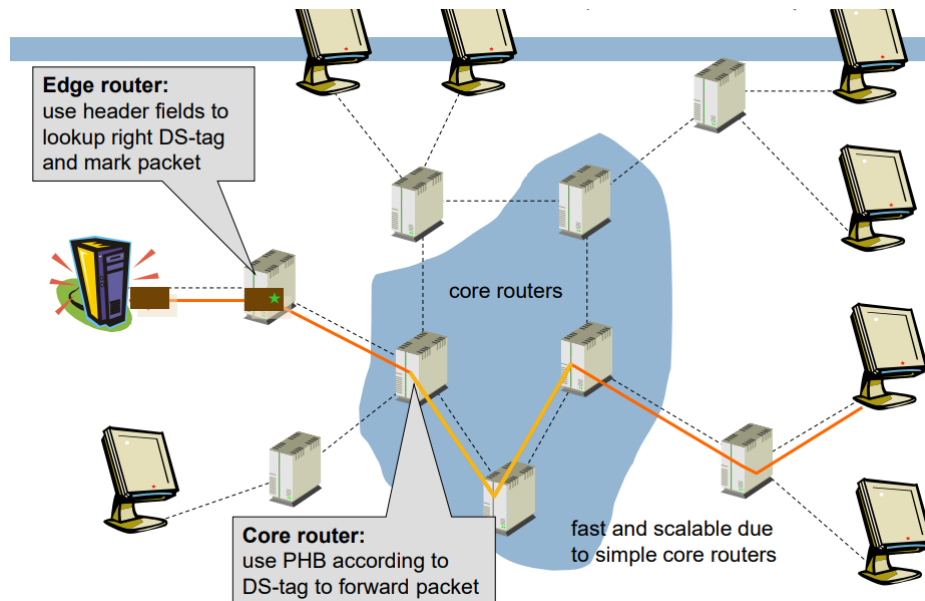
Questo classificatore ha delle regole che sono legate alle edge function, si cerca già a livello di bordo di evitare che ci siano dei picchi, congestione della rete. C'è una metric function che cerca di evitare che ci siano congestioni sicure a livello di core router. Cerco di bloccare se ho trasmissioni troppo veloci, scartando pacchetti o declassandoli a livello di mark.

A questo punto andiamo sui core router, che non fanno altro che instradare tramite la logica PHB leggendo il marker dei pacchetti. Il loro comportamento dipende dal marker con cui sono stati classificati i pacchetti. Instradano secondo un aggregazione di traffico che dipende dalla rete.

I core router usano una politica di token bucket per regolare il traffico.

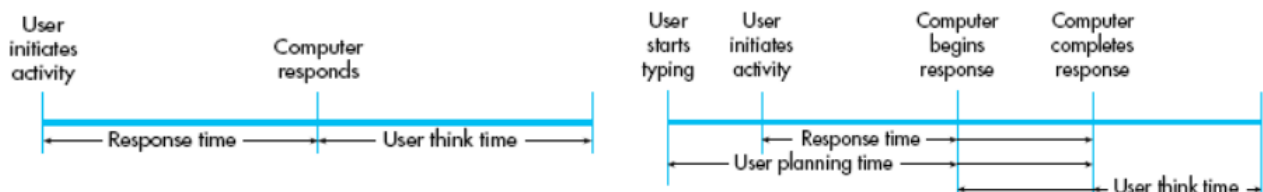
Come si comporta l'instradamento? non ci sono delle specifiche a livello di standard ma degli esempi (tutti i pacchetti di classe A devono avere una percentuale di larghezza di banda).

Quindi anche in questo caso siamo in grado di garantire un controllo di qos.



12.6 Human computer interaction

La qos della trasmissione non basta per l'utente, se faccio streaming sì, chiaramente se c'è un'interazione le logiche cominciano a cambiare perché abbiamo una doppia via di comunicazione.



L'utente interagisce con i sistemi in maniera soggettiva e non possiamo prevederlo. Noi possiamo scomporre l'interazione, per fare il task l'utente deve pensare, poi c'è il planning time in cui capisco cosa fare e lo faccio, nel momento in cui inizio ad interagire la macchina deve iniziare a rispondere con un tempo di risposta adeguato, a questo punto quando l'utente ha presentato la risposta devo pensare se era quella che volevo. Con la qos abbiamo garantito solo il response time della macchina, ma non abbiamo detto nulla del resto.

Il response time e il display time hanno effetto sulla soddisfazione dell'utente in termini di costo, performance, error rate. È stato dimostrato che agli utenti piace un'interazione rapida. L'utente quando interagisce con un sistema deve risolvere un problema, quindi ha delle aspettative. Quando interagisce è importante sottolineare che ci possono essere degli errori, che dipendono dai task a cui dobbiamo rispondere, se sono tanti complessi un'interazione molto rapida rende difficile risolvere i problemi e sarebbe meglio che la macchina rispondesse con un rate più basso, viceversa se stiamo facendo un task semplice. Quindi la qualità non dipende solo dalla trasmissione, ma anche dal task. I response time devono variare in base al task, per fare alcune cose c'è bisogno di tempo. Deve variare anche in base all'esperienza dell'utente, se è novizio non tanto pressante, quindi un'altra

variabile da considerare è l'utente.

L'utente fa fatica ad adeguarsi ad una macchina che continua a cambiare, la variabilità del comportamento della macchina da fastidio.

Per concludere va bene garantire la qos a livello tecnico, ma non basta, dobbiamo capire come l'utente interagirà con la macchina.